

Centroid CNC PLC and CNC functions Programming Manual

CNC12 v5.x+

Oak, Hickory, Allin1DC, Acorn, AcornSix, and MPU11 platforms.

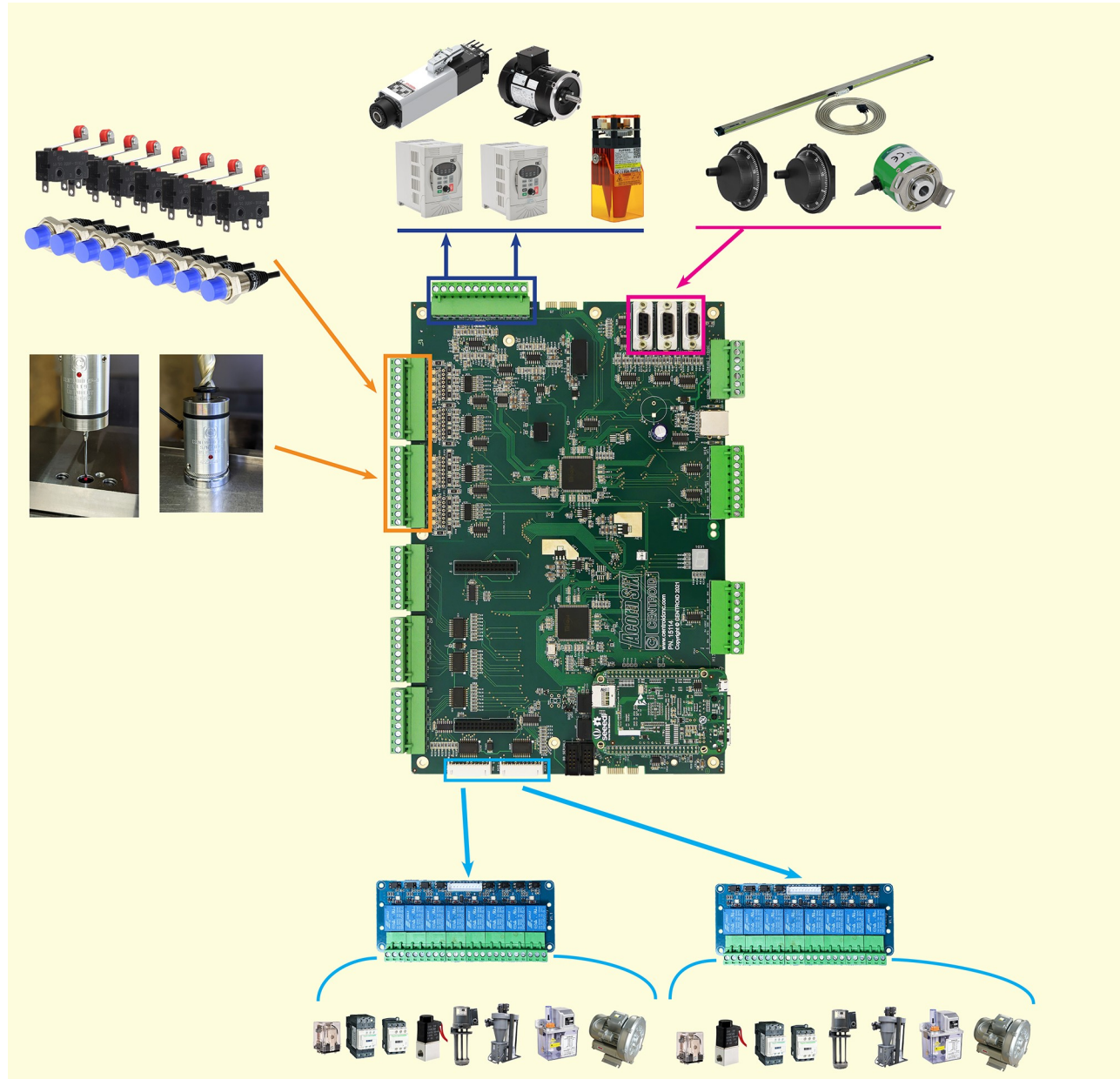


Table of Contents

Introduction.....	4
Resources and Tools.....	10
Conventions Used in this Manual.....	11
Compiling a PLC Program.....	11
PLC Program Statistics.....	12
Language.....	13
Programming Conventions.....	13
Defining Variables.....	14
Data Types.....	15
Keywords.....	19
Operators.....	23
Standard PLC Program Layout.....	31
Initial-Condition Setup.....	31
Internal PLC Fault and Software Running Checking.....	31
Jog Panel and Keyboard Jogging.....	33
Axis Enable.....	33
Fiber/Wire Connection Checking.....	33
LubeTimers.....	35
Feedrate Override.....	38
Spindle Functionality.....	38
MPG Operation.....	39
Probe Protection.....	41
PLC Optional Sections.....	43
Debounce or Invert Inputs.....	43
Setting Inputs High or Low for Testing.....	48
Compiler Errors.....	51
Warnings.....	51
General Errors.....	51
Syntax Errors.....	52
Application Examples.....	59
Toggle an Output Every Second.....	59
Aux Key Jogging.....	59
Aux Key Override of M-Code.....	59
Wait One Second Before Jogging on Key Press.....	59
Interpret Enter Key as Cycle Start in MDI*.....	59
Count Machine On Time.....	59
Custom M-Codes.....	60
Using M94/M95 Bits.....	60
Using One M94/M95 Bit and a Parameter.....	60
Customizing Standard M-Codes.....	60
Troubleshooting and Changing PLC Programs.....	61
Write Down and Think Through Changes to the Program.....	61
PLC Diagnostic Screen.....	61
PLC Bit-State Dump.....	61
DUMP.....	61

Echo to a Memory Bit.....	61
Use Stages.....	61
Communication In/Out Faults.....	62
PLCBus.....	62
Appendix A: Example PLC program.....	63
ALLIN1DC DC system example.....	63
Appendix B: Jog Panel Mapping.....	98
JogPanel Inputs and Outputs.....	98
Appendix C: Keyboard Jog Mapping.....	99
Notes on Keyboard Jogging.....	99
Keyboard Key Numbering Table.....	100
Appendix D: System Variables.....	101
System Variable Types.....	101
Notes on Certain System Variables.....	101
Appendix E: PLC I/O Location.....	120
ALLIN1DC.....	120
DC3IOB.....	120
GPIO4D.....	121
PLC Expansion.....	121
Appendix F: What's New in CNC11.....	123
There is Only One PLC Program.....	123
The PLC Program has the Final Word.....	123
Spindle Speed DAC Command.....	123
Direct Control of and Responsibility for Jogging.....	123
Compiler/Language Differences.....	123
Appendix G: Definitions of nonobvious Words.....	125
Bit.....	125
Integer Number.....	125
Floating-point Number.....	125
Range.....	125
Precision.....	125
Data Type.....	125
Define/Declare.....	125
Variable.....	125
Constant.....	125
Appendix H: Cyclone & MCU Status SV Information.....	127
Appendix I: Drive Status SV Information.....	128
Appendix J: Error Counter Information.....	130
Appendix K: Drive Control SV Information.....	131

Introduction

The Centroid PLC or Programmable Logic Controller is responsible for controlling outputs primarily based upon the state of inputs, but also for controlling outputs based upon time. The Centroid PLC Program itself is the logic that is written to describe how the Centroid CNC PLC reacts to the Inputs and when to cause Outputs to turn on or off. Centroid's PLC programming Language and Features allow much more control and capability than just a basic i/o ladder logic setup. Centroid's PLC programming language supports control of advanced CNC control functions which allow extensive customization of the CNC12 software to meet a wide variety of applications. Centroid PLC programming language, compiler and its related real time Logic Analyzer the "PLC Detective" and PLC diagnostic menu are free tools provided and maintained by Centroid for our customers.

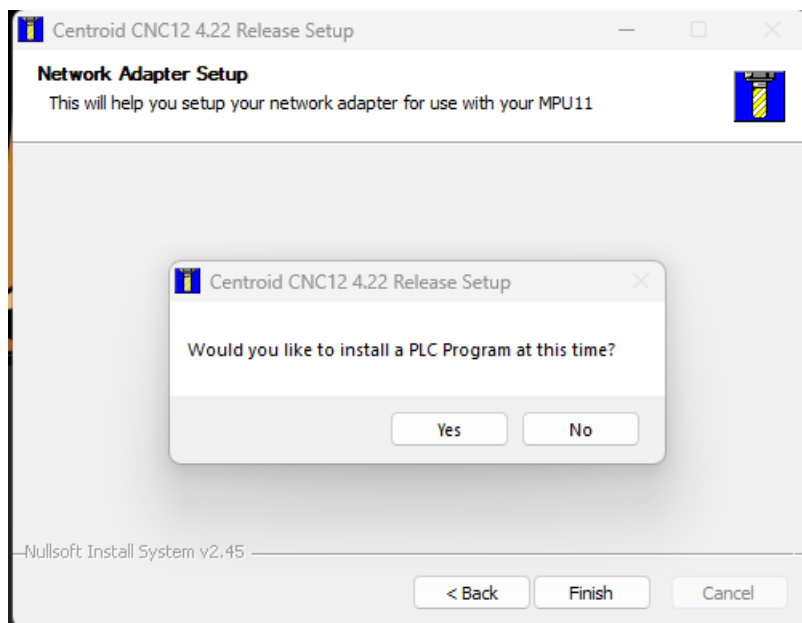
Note: Many common CNC functions and actions can be performed with simple CNC Macro programming that do not require any PLC program editing to do these function(s). Newbies often confuse or assume that the CNC function that they want to create or modify must be done in the PLC program so, before jumping to that conclusion be sure to review Centroid's advanced Macro feature set by reading this ["Introduction to Centroid CNC12 Macro Programming"](#) guide and reviewing the relevant sections in the Mill and Lathe operator manuals

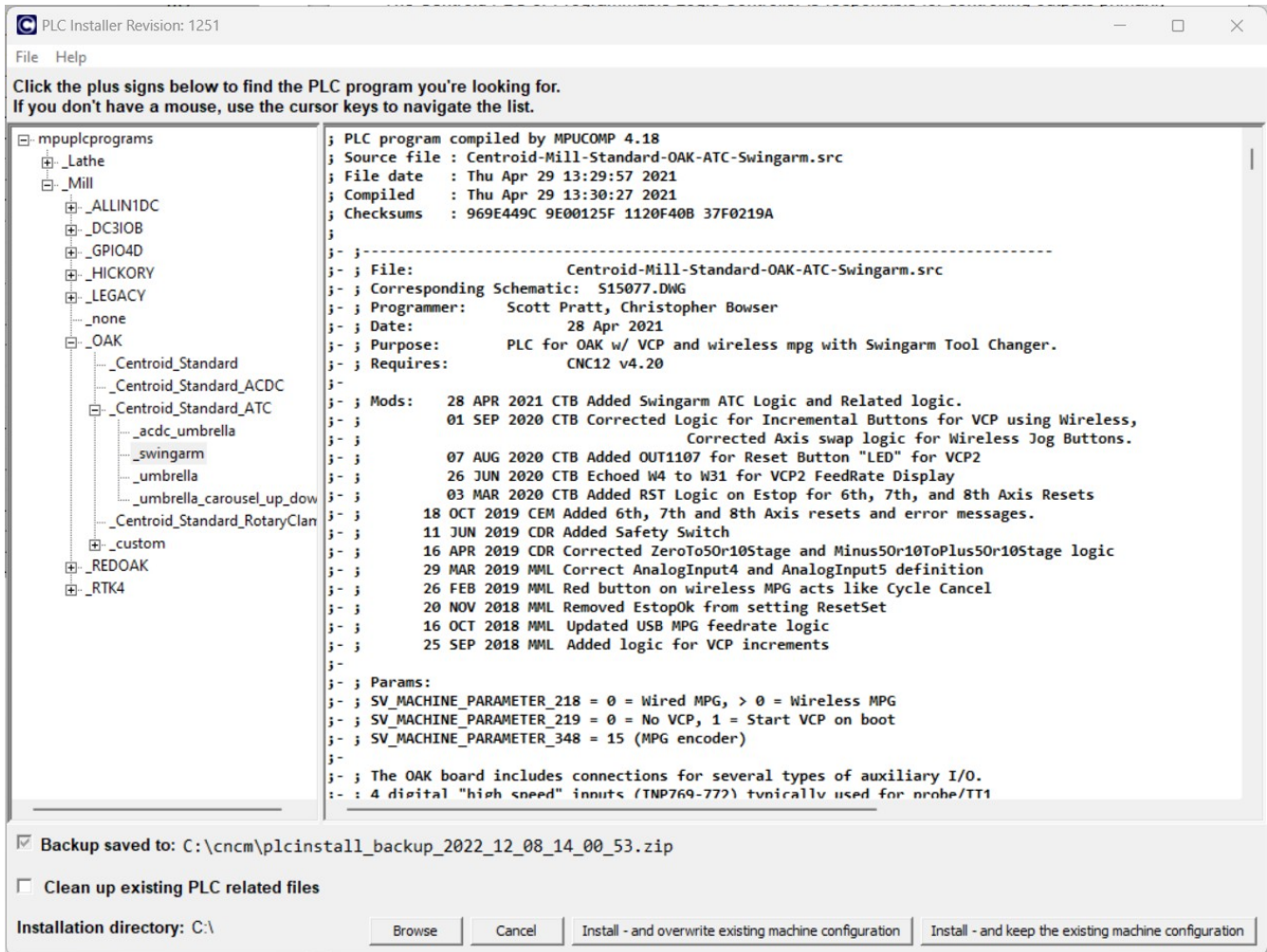
"Stock" tested and working PLC programs for common CNC controller configurations are provided by Centroid and are included with the CNC12 software installer. Each "Stock" PLC Program has a corresponding CNC control system Hookup Schematic. These schematics can be found on the schematics download page. [Here. https://www.centroidcnc.com/centroid_diy/schematics/pbrowse.php](https://www.centroidcnc.com/centroid_diy/schematics/pbrowse.php)

Use the search to find and sort, for example type in "OAK ATC" to see all the Oak ATC schematics.

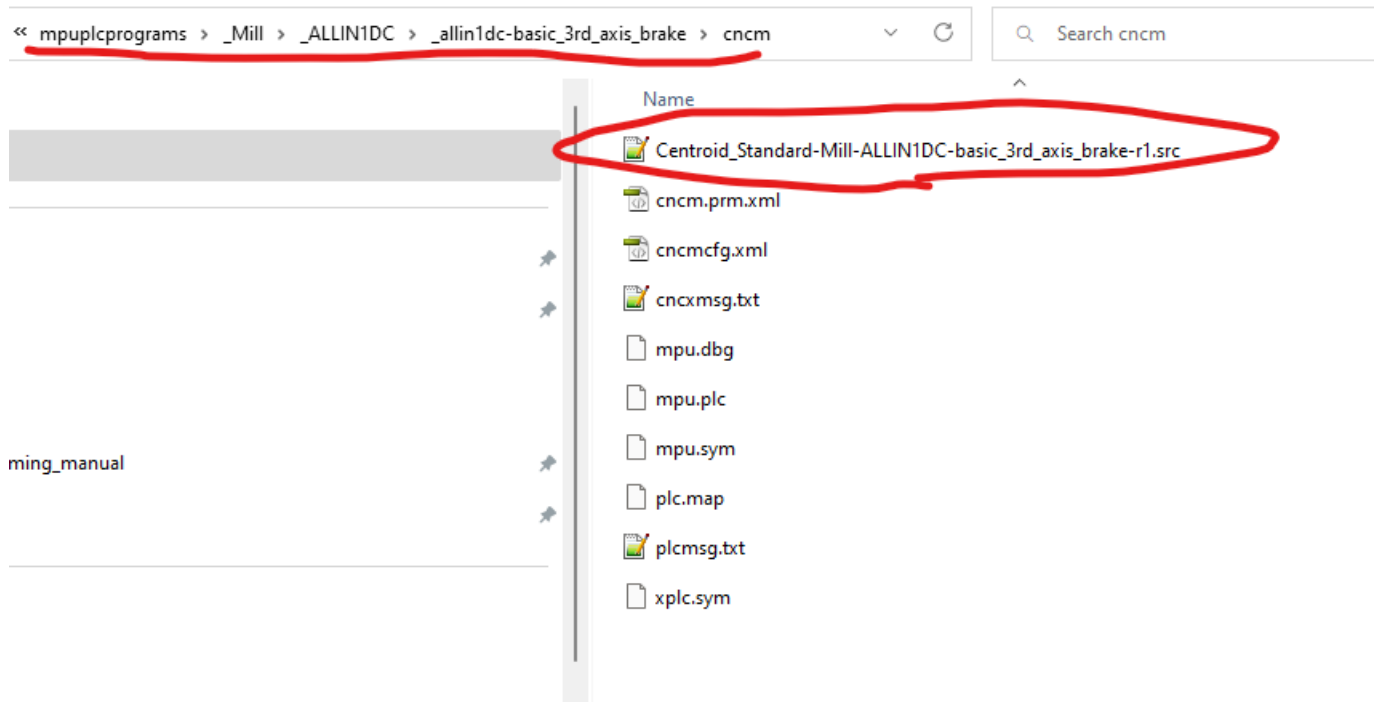
When running the CNC12 software installer it will ask you to choose a PLC program to install from a list of Centroid provided PLC programs.

These "stock" PLC programs are tested working programs are for use with Oak, Allin1DC, MPU11, RedOak, Hickory systems. These common machine configurations PLC programs allow for easy fast CNC control setup and commissioning. Load the PLC program you want, wire it according to the schematic, configure the machine parameters and you are up and running. Follow the instructions in the Oak and Allin1dc installation manuals for step by step setup information when using a stock PLC program.





If you wanted to view or edit any of the source files for the stock Centroid provided PLC programs they are included with the CNC12 installer in a folder called "mpuplcpprograms"



Acorn and AcornSix CNC Control systems use the same Centroid PLC programming language as all Centroid CNC controllers but, the Acorn/AcornSix CNC12 uses a CNC control configuration Wizard which creates the PLC program for the CNC control based on the Input / Output selections made by the user in the Wizard input and output menus! In essence the Wizard creates a 'custom' PLC program based on user selections from a list of canned PLC functions. A wide variety of 'canned' PLC functions are select-able from a list in the Wizard which are drag and drop assigned to any input or output, the Wizard will then create a PLC program using the users selections.

Input Type:
All

Axis1DriveOk
Axis2DriveOk
Axis3DriveOk
Axis4DriveOk
ChuckFootPedal
ChuckIsClosed
ChuckIsOpen
CutOffIsDown
CycleCancel2
CycleStart2
DrawBarReleased
DriveOk
FeedHold2
FirstAxisHomeLimitOk
FirstAxisHomeOk
FirstAxisMinusLimitOk
FirstAxisPlusLimitOk
FourthAxisHomeLimitOk
FourthAxisHomeOk
FourthAxisMinusLimitOk
FourthAxisPlusLimitOk
HomeAll
LimitAll
LubeOk
Ohmic Sensor

Acorn Integrated Inputs 1-8

NC	NO		Definition
1		IN1	SafetyDoorSwitchClosed
2		IN2	ProbeTripped
3		IN3	ProbeDetect
4		IN4	BackGear
5		IN5	ZriHomingAll
6		IN6	VFDUpToSpeed
7		IN7	VFDZeroSpeed
8		IN8	EStopOk

Click and Drag an Input function definition from list to the Input number Definition box to assign a function to an input.

Click the Input number circle to toggle the input state from NC to NO. Note: Probe Input states are determined in the Probe setup menus.

Output Type:

Mill

▼

AirBlowNozzle

AmberLight

Axis1BrakeRelease

Axis2BrakeRelease

Axis3BrakeRelease

Axis4BrakeRelease

ChargePump

DustCollectionOn

Flood

G540SpinFwdOff

G540SpinRevOff

GreenLight

LaserEnable

LaserReset

Mist

OrientSpindle

PWM Output

PWMSelect

RedLight

SafetyDoorLockOpen

SpindleCooling

SpindleCoolingFan

SpinREV

TurnClampOn

UnclampTool

VFDDirection

VFDEnable

VFDResetOut

Acorn Integrated Outputs 1-8

Definition

1 OUT1 ATCAirBlowActivate

2 OUT2 DriveResetOut

3 OUT3 SpindleBrakeRelease

4 OUT4 SpinFWD

5 OUT5 VacuumOn

6 OUT6 WorkLight

7 OUT7 LubePump

8 OUT8 NoFaultOut

Click and Drag an Output function definition from list to the Output number Definition box to assign a function to an output

The Acorn Wizard uses a Universal Template PLC source program “acorn_universal_template.src” and creates the ‘custom’ Acorn PLC program based on the user selections in the Wizard and the PLC code in the universal template. Source code for the Wizard generated PLC programs appear in the root cncm or cncd directory: ‘acorn_mill_plc.src” etc.. The Acorn Template PLC source code is located cncm\resources\wizard\default\plc\acorn_universal_template.src

When hand editing an Acorn PLC program be sure to turn off the automatic PLC program generation feature of the Wizard so the Wizard does not overwrite your work. The “Custom PLC” in use selection can be found in the Wizard preferences menu.

This manual assumes basic familiarity with Centroid's PLC programming. Not every detail is explained. Centroid has created Centroid PLC programming Video series that is available on YouTube for newbies. If you are just starting out with learning Centroid PLC Programming start with this video series. Below is the link to the YouTube Centroid PLC programming playlist.
https://www.youtube.com/playlist?list=PLXhs2C5No0_gFS_RmKNo7hii2WKledQIQ

Videos in the PLC programming playlist.

Video 1 – Notepad++ Setup Downloading and installing Notepad++.
<https://notepad-plus-plus.org/downloads/>
Following TB294 to setup Notepad++
https://www.centroidcnc.com/dealersupport/tech_bulletins/uploads/294.pdf

Video 2 – PLC Introduction What is a Centroid PLC program? What does a Centroid PLC program do? What kind of stuff can you control with it? Centroid PLC program language. Files associated with Centroid PLC.

Video 3 – Installing CNC Software and PLC Program Installing/Updating CNC12 software. Installing PLC program. Standard Centroid PLC programs.

Video 4 – PLC Definitions – Part 1 PLC program header. Adding comments/commenting lines of logic. Constant definitions used for custom PLC messages.

Video 4 – PLC Definitions – Part 2 Inputs, outputs, and memory bits. Probe, MPG, and jog panel connector I/O. System variables CNC-to-PLC PLC-to-CNC. Skin events Custom macros – M94/M95 Words, floating words, one-shots, timers, and stages

Video 5 – PLC Program Logic IF THEN statement. Logic AND, OR, and XOR. Logic Not(Inverse), Is equal to, and Is not equal to. SET, RST, and coils. Bit-based parameters.

Video 6 – PLC Tools PLC Diagnostic menu. Inverting inputs. Forcing outputs on and off.

https://www.centroidcnc.com/centroid_diy/downloads/acorn_documentation/cnc12_PLC_diagnostic_screen.pdf

PLC Detective.
https://www.centroidcnc.com/downloads/centroid_PLC_detective_quickstart.pdf Logic analyzer.

Video 7 – Turning Output On-Off Using Input Logic to turn an output on and off based on the state of a physical input Using SET and RST commands Using the coils - ()

Video 8 – Turning Output On-Off Using AUX button – Part 1 and 2 Logic truth table. Using one-shot. Logic to turn an output on and off whenever an auxiliary button is pressed. If the output is off and the button is pressed, it will turn the output on. If the output is on and the button is pressed, it will turn the output off. Have the LED associated with auxiliary button follow the state of the output.

Video 9 – Turning Output On-Off Using Custom Macro– Part 1 Macro definition. Macro header. Skip macro when in Search or Graph mode. System variable table in operator's manual. MDI. Use 1 macro to turn output on and another macro to turn it off. Tie macro to auxiliary button. Use 1 macro to turn output on, wait some time, and then turn the output off. Use 1 macro to turn output on, wait some time, then wait for an input to close, and then turn the output off. Chapter 9 – Turning Output On-Off Using Custom Macro– Part 2 Use timer that is defined in the PLC program to place the system in a fault state if the input isn't set. Use timer that is based off of a parameter value to place the system in a fault state if the input isn't set. Creating custom PLC message.

Video 10 – PLC Logic For Safety Items Door Interlock. Light curtain. Place system into a fault state when running a program. Prevent a program from running.

Resources and Tools

Notepad++

<https://notepad-plus-plus.org/downloads/>

Configure Notepad++ for PLC editing and CNC Use

https://www.centroidcnc.com/dealersupport/tech_bulletins/uploads/294.pdf

Introduction to PLC programming Video Series

https://www.youtube.com/playlist?list=PLXhs2C5No0_gFS_RmKNo7hii2WKledQIQ

PLC Detective

https://www.centroidcnc.com/downloads/centroid_PLC_detective_quickstart.pdf

PLC Diagnostic menu (virtual input/output LEDS)

https://www.centroidcnc.com/centroid_diy/downloads/acorn_documentation/cnc12_PLC_diagnostic_screen.pdf

Free Tech Support

<https://centroidcncforum.com/index.php>

Paid one-on-one Factory Technician Tech Support / PLC Programming.

https://www.centroidcnc.com/centroid_diy/purchase_tech_support.html

Links the latest Mill (Router) CNC12 Operator Manual and the Lathe CNC12 Operator Manual can be found here. <https://centroidcncforum.com/viewtopic.php?f=60&t=3397>

This manual is for anyone trying to change or write a PLC program on a Centroid CNC11 or CNC12 system. A CNC11 system is one that is based on MPU11 hardware, while a CNC12 system is one that is based on MPU12 hardware. The Centroid CNC11 software works with a CNC11 system. The Centroid CNC12 software works with both a CNC11 and a CNC12 system.

Centroid's PLC programming Language is the same for all of Centroid's current CNC platforms including but not limited to: Oak, Allin1DC, MPU11, Acorn, AcornSix, Hickory. Some PLC functionality and capability differences between these platforms is defined by the hardware feature set of that CNC control board there is no limitation or differences in the PLC programming language itself.

There are some definitions in [Appendix H](#), but it is intended to clarify where there may be confusion between CNC11/12 PLC program meaning and a more general meaning. If you are unsure of what you are doing, please contact Tech. Support at support@centroidcnc.com or make a post on the Centroid Tech Support forum and ask questions.

When modifying PLC programs, it is considered good practice to regularly make backups. Always make a report.zip ('create report' in the CNC12 utility menu) and store it in at least one location in case the program change needs to be reverted.

The manual explains the components that can be used to make a PLC Program and goes over the standard parts of PLC programs. There are several appendices including sample programs, detailed key mappings, differences in CNC10 and CNC11/12 PLC programs, and errors associated with the compiler. If you are experienced with CNC10 PLC programs be sure to read [Appendix H](#) about the differences between CNC10 and CNC11/12.

Conventions Used in this Manual

There are several text conventions used in this manual. The follow list explains the most common ones.

- Code from PLC programs including System Variables and the various data types are in Consolas font at 10pt size. An example is `SV_PC_VIRTUAL_JOGPANEL_ACTIVE`.
- Keyboard Keys are in Arial font at 10pt. Size and bold. An example is **ALT-Q**
- On and SET are interchangeable, as are Off and RST.
- [System Variables](#) may be written as SV.
- PLC program and program are used interchangeably and mean the same thing.
- [Data Type](#) and type are used interchangeably and mean the same thing.
- When a specific name for a [Data Type](#) is helpful it is used, but typically the direct name is used to remove confusion about what type is being used in a given example.
- Names of [Data Types](#) such as [Memory Bits](#) and [Outputs](#) are capitalized.
- Binary data is written in this document from Most Significant Bit (Msb) to Least Significant Bit (Lsb) on a Left to Right order when explaining how bits are moving around. This follows the typical convention in programming. **Note:** that the PLC program reads and writes pure binary data from left to right as well, but goes from Lsb to Msb. See [BTW](#) and [WTB](#) for more information.
- From a general PLC programming viewpoint, the term CNC11 software is the same as CNC12 software. The term "CNC software" denotes either CNC11 or CNC12. The same applies for MPU11, MPU12, and MPU13 with the term "MPU hardware" denoting either MPU11, MPU12, or MPU13 based hardware.

Compiling a PLC Program

The source code which you write or change must be converted to the format that CNC software uses internally. This is accomplished by compiling the program. The compiler is called mpucomp which is short for MPU compiler. The syntax for compiling a program is `mpucomp.exe ProgramName.src mpu.plc`. If the filename contains spaces, it must be surrounded with quotes. The first file is the source code (a text file) and the second file is the name of the output file (a text file). CNC software looks for a file named mpu.plc to load and execute as the PLC program, so the output file must be named exactly `'mpu.plc'`, including the fact that it is all lower

case letters. Once the program is compiled successfully, the system should be powered off completely and powered back on again for the changes to take full effect. CNC software will display a warning when it detects the loading of a changed PLC program. Whether or not you can proceed without side effects depends upon what has been changed in the PLC program and the current state of the PLC system. When in doubt, power off.

PLC Program Statistics

The PLC program is constrained by certain factors and limits. The following list enumerates some of the more important ones. Pressing **ALT-I** on the main menu causes the Live PLC Diagnostic screen to appear. It shows the state of Inputs, Outputs, Stages, Memory Bits and Words as well as the Time that is being taken by execution of the PLC program.

- Presently there is a limit of about 80 Inputs and Outputs using a GPIO4D and four PLC1616ADD boards. DC3IOB systems can achieve a bit more than this. There is a hard system maximum of 768 Inputs and 768 Outputs in CNC software and MPU hardware. This limit does not apply to IO related to the Jog Panel.
- There are many powerful features that take a long time to execute and thus should be used sparingly. Usage of them is detailed below and recommendations against using them are included in the [Operators](#) section.

Language

Data Types

The kind of information that a variable can hold is defined at compile time by the data type when it was declared. All of the types below can be used in a PLC program. Typically Words and Floating-point Words have enough precision to achieve the desired results versus Double Words and Double Floating-point Words.

Data type	Range	Notes
INP	INP1-INP1312	Physical inputs mapped to hardware
OUT	OUT1-OUT1312	Physical outputs mapped to hardware
MEM	MEM1-MEM1024	Internal memory bits
STG	STG1-STG256	STG1 is set on at startup
FSTG	FSTG1-FSTG256	Fast Stages
T	T1-T128	32-bit timers, only T1-T64 available prior to v4.22
PD	PD1-PD256	One-shot (positive differential)
W	W1-W128	32-bit signed integers W1-W88 available for PLC Detective and G-code variables
DW	DW1-DW128	64-bit signed integers DW1-DW22 available for PLC Detective and G-code variables
FW	FW1-FW128	32-bit floating point variables FW1-FDW44 available for PLC Detective and G-code variables
DFW	DFW1-DFW128	64-bit floating point variables DFW1-DFW22 available for PLC Detective and G-code variables

Important Note

It is critical to understand when specific Data Types are updated during the execution of a PLC program. The execution of a PLC program occurs approximately 1000 times a second, but only PLC program code inside a fast stage (FSTG type) or outside of any stage is actually executed 1000 times a second. PLC program code inside a regular stage (STG type) is executed 50 times a second. The execution of a PLC program from top to bottom as it has been written is referred to as a PLC “pass”.

A question that quickly emerges when writing PLC programs is, “If I change the value of a variable in the PLC program, when does it actually take effect?” The answer to this question is that it depends upon the data type being changed. Timers, Inputs and Outputs are all buffered at the beginning of the program. This means that a snapshot is taken of the state of them and that image **does not change during the pass of the PLC program**. In other words, Timers have the same value at the start of the pass as they do at the end. The same is true for Inputs and Outputs as far as the real-world physical state is concerned. The snapshot of the Inputs never change, but the image of the Outputs can be changed on any line and that brings us to the other category of when things update. Memory Bits, all Words, One-Shots, both kinds of Stages, System Variables, and the *image* of the Outputs are changed immediately and on the next line of the program will be at the value they were assigned to on the previous line.

Another important thing to note is that the Live PLC Diagnostic screen (**ALT-I**) does not show every transition of every variable. What you see is a snapshot of PLC data that is updated about 20 times a second.

Constant Definitions

It is often easier to remember a name for an error rather than the number associated with it. Defining a Constant allows for this ease of use. By convention Constants are put before Variable Definitions, but they can be put

anywhere before the first IF statement. Typically, Constants are used for PLC message numbers, but they can be used for anything. Math is allowed when defining constants to help avoid mistyping a number. Parentheses are allowed to force correct math, but only for Integer values. Floating-point numbers cannot have math done on them or be used to create them. See the [Syntax Errors](#) section.

```
PI_C          IS 3.1415926535897932384626433832795
ASYNC_C       IS 2
SYNC_C        IS 1
MULTIPLIER_C  IS 256
FAULT_C       IS (SYNC+5*MULTIPLIER)
```

Input – INP

Inputs are physical switches or buttons that can be either on or off. When defining an Input, it is written as Limit_Switch_I IS INP2 or Aux10Key_I IS JPI18. Inputs can be logically combined with [Outputs](#), [Memory Bits](#), [Timers](#), [One-Shots](#), [Stages](#), [Fast-Stages](#) and [System Variables](#) that are bits using [Logical Operators](#), but not [Relational Operators](#). Limit switches, Jog Panel keys and Inverter signals such as fault or at speed are all examples of inputs. Analog voltage inputs are a range of inputs typically 12 or 16 inputs and must be converted to Word type variables.

Output – OUT

Outputs are physical, real-world outputs such as relay contacts, relay driver signals, or analog signals that can be on or off. Analog voltages are converted from Bits in the PLC program to analog voltage at the header. They can be used to control other relays, lube pumps, spindle enable, inverter analog speed control, coolant, Jog Panel LEDs, etc. The standard way of specifying an output is Lube_0 IS OUT2 or AutoCoolantLED_0 IS JP021. Outputs can be logically combined the same way as [Inputs](#).

Memory Bit – MEM

Memory Bits can be on or off and cannot directly affect anything outside the PLC program. These Bits are often used to make logic easier to read by combining many repetitive checks or keystrokes into one variable. Memory Bits are also used for debugging to store whether transient signals have occurred, whether to invert inputs based on parameter 178, etc. Memory Bits can be logically combined to other Bit type Variables with [Logical Operators](#), but not [Relational Operators](#).

Word – 32-bit – W

Words are Integer numbers that can be used to store error codes, parameter values, and System Variables that are Word sized themselves. Words are defined like Error_Code_W IS W10. Only the 12 words are visible on the Live PLC Diagnostic screen at one time. They can be compared with other Word types, System variables that are Integer numbers rather than one bit, and Timers. [Relational Operators](#) are allowed on any Word type variable, but [Logical Operators](#) are not. Doing a Relational comparison produces a result that can be used in a Logical Operator, however. An example of this is IF (W1 > W2) || MEM1 THEN SET OUT6. This statement tests whether W1 is greater than W2 first. If that is true or MEM1 is true then the output is turned on. Words can hold values from -2147483648 to 2147483647.

Double Word – 64-bit – DW

Double Words are Integer numbers just like Words, but can hold values from -9223372036854775808 to 9223372036854775807. A Double Word is defined like BigNumber_DW IS DW3. For most PLC programs, Double Word types are not necessary. Double Words are otherwise exactly the same in usage as Words.

Floating-point Word – 32-bit – FW

Floating-point Words are real numbers that can store fractional values from 2^{-149} to 2^{129} . It is typically precise enough for any operation in a PLC program. It is defined like SpindleDAC_FW IS FW1. Precision problems can

occur if comparing very very large and very very small numbers, but typically this is not a concern. Floating-point Words have the same comparison ability as Words.

Double-Floating-point Word – 64-bit – DFW

Double-Floating-point Words are real numbers that can store fractional values from 2^{-1074} to 2^{1022} . They are more precise and can compare bigger numbers with smaller compared to Floating-point Words. In general is is not advised to use this type at all due to significant time required to do any calculations. The definition of a Double-Floating-point Word is `PreciseNumber_DFW IS DFW1`. DFWs are compared exactly like Words.

Timer – 32-bit – T

Timers are counters with the special ability to be compared with both [Relational](#) and [Logical Operators](#). This means that you can check `IF T1 THEN SET OUT1` to see if the Timer has reached its set point or `IF T1 > 1000 THEN SET OUT2` to see if the Timer has counted past 1000 milliseconds (one second). Timers are initialized with a 32-bit positive Integer number that is interpreted as the number of milliseconds to count before evaluating to true when checked with Logical Operators.

The value is typically stored in the Timer during the `InitialStage`. To start a Timer counting use `SET T1`. To reset the Timer so that it is waiting to count again use `RST T1`. Note that you do not need to store a value into a Timer each time it is RST unless you want to change the value to which it counts.

Timers evaluate to true after reaching the stored time until they are reset. The actual count of the Timer continues to climb until it gets to about 10 seconds before the max value that Timers can hold. This means that Timers can be checked to see if they are over a certain value with Relational Operators.

Timers can count up to 2,147,473,646 milliseconds (24 days, 20 hours, 31 minutes, and 13.646 seconds). Be aware that if you are trying to time something longer than that you will need to come up with a scheme for tracking the rollover. A simple option for counting days is to set the Timer to 1 day worth of ms (24 hrs. * 60 min. * 60 s = 86400000 ms). When the Timer expires, increment a Word day counter and then RST and SET the Timer to begin counting again. See [Application Examples](#) for how to setup a Timer for counting days. The following example sets the Timer to one second then SETs the Timer if MEM1 is closed. When T1 counts to 1000, an Output is turned on and the Timer is RST. If the Memory Bit is set next time through the PLC program the Timer will be SET again. The first time the Timer value has counted past 10 a Memory Bit will be turned on.

```
T1 = 1000
IF MEM1 THEN SET T1           ;Set the value that the timer counts to before evaluating to
                               ;true.
IF T1 THEN RST OUT2, RST T1   ;If 1 second has elapsed turn off the output and reset the
                               ;timer so it can be started again.
IF T1 > 10 THEN SET MEM50     ;If the timer has counted past 10ms SET MEM50
```

One-Shot – PD

One-Shots or Positive Differentials are used to detect the first rising edge of an event occurring. A One-Shot can only be turned on and off using a [Coil](#). Because of this a One-Shot should only ever be SET/RST on one line of the PLC Program. It can be checked anywhere, but not SET/RST. Once the PD has been SET by the IF test, the conditional section must evaluate to false and thus RST the PD before it can be used again. This means that you cannot hold a button down and cause the One-Shot to trigger more than once. Using this in combination with Debounce allows safer detection of key presses to prevent multiple actions when only one is intended. One-Shots are often used on Jog Panel keys and M-Codes. One-Shots may not be desired in certain circumstances such as the Override +/- buttons for Spindle Speed. Typically one wants to hold down the button and have the Override value change as long as the button is held down. An example definition and usage of a One-Shot follows.

```
KeyPress_PD IS PD1           ;Define the One-Shot
IF JPI1 THEN (KeyPress_PD)    ;If jog panel Input 1 is pushed, set the One-Shot
IF KeyPress_PD THEN SET MEM300 ;If the One-Shot is set, turn on memory bit
```

Stage – STG

Stages are useful in many ways. First, they are used to break up different sections of the program to allow easier debugging and testing. Conventional programming dictates that there is at least an InitialStage and a MainStage in any PLC program, but Stages are not required to be explicitly used at all. Turning a Stage on and off is just like the process for Outputs.

```
IF 1==1 THEN SET STG1           ;Turn on a Stage all the time
IF M6_SV THEN SET ATC_Main_Stage ;Turn a Stage on when a tool change is called
```

When a Stage is RST the PLC executor does nothing with the logic inside the Stage. This means that whatever Variables are SET or RST when the Stage itself is RST maintain that state unless they are modified somewhere else in the program.

Warning: No checks are made against resetting any stages. It is possible to RST every stage and have nothing executed in the PLC program. This requires a system power off and reboot.

The InitialStage is often used to setup any timers used and the state of tools for ATC machines. At the end of the InitialStage a JMP should be called to the MainStage. The InitialStage should never be SET again. To troubleshoot a problem, Stages could be turned off to narrow down the problem section. Also, Stages are usually used to break up the steps of a tool change sequence and various M-Codes so that they are not executed or checked every time through the PLC program and for debugging. Stages are executed at the standard rate of PLC program execution which is 50 times per second.

Warning: Typically Coils should not be used in Stages that are turned on and off, especially if you come from a CNC10 PLC programming background. This is because CNC10 PLC typical coil behavior will not occur if a Stage is off. This means that One-Shots should also not be used in Stages that may be turned off. Odd effects can occur with One-Shots in Stages if you do not know what to expect. If, for example, a One-Shot is used to RST a Stage, the One-Shot will **not** be reset the next time through the PLC program, but will instead wait for the next time the Stage is SET and the Coiled One-Shot is scanned over. This will, instead of setting the One-Shot as might normally be expected, cause the One-Shot to be RST finally and then require the One-Shot to be triggered again in the **next** PLC Program pass, if the Stage is still SET.

Timer counting is not affected by a Stage being RST. This means that if you SET a Timer in a Stage and then RST the Stage the Timer still counts and can be checked outside of the Stage and the expiration of the Timer will be accurate.

If you want to use a One-Shot to exit a stage, say from a button push, then you should put the One-Shot in coils again to make sure it is turned off on the line that resets the Stage. The following example illustrates the concept. With this example, if the Jog Panel key is pressed when the Stage is executed, the Stage will always be reset on the first pass through.

```
;=====
STG1
;=====
IF JPI1 THEN (PD1)           ;push a jog panel button to trigger a One-Shot
IF PD1 THEN (PD1), RST STG1  ;accessing the PD again causes it to get RST, Stage is RST
```

Fast Stage – FSTG

Fast Stages behave exactly like Stages except that they are executed at up to 1000 times per second. These should be used only when extremely precise timing is necessary.

Warning: Fast Stages do not interrupt the operation of normal speed stages, so if the normal PLC program takes longer than 1 ms to execute completely, then the Fast Stages will not execute at the stated rate. Make sure that the standard timings are below 1 ms to get the fastest execution time. The average time can be found in the PLC Diagnostic screen.

System Variable – SV_*

These are System Variables that are SET or RST by CNC software or PLC Program. There are both Word and Bit type SVs. Both CNC software and the PLC program can read these System Variables. Convention states that

they should be typed in all caps. For a complete list of System Variables and their uses see [Appendix D](#).

Keywords

Keywords are reserved words in the PLC programming language that cannot be used except for the defined purpose. Often attempting to use the keyword in an undefined way will cause the program to fail compilation. In example code they will always be capitalized by convention and should not be confused with constant defined variables or System Variables.

Defining variables – IS

This keyword is used to setup labels for all the data types. It is only used in the definition section at the top of the program. Any definition after the first IF...THEN statement will cause an error at compile time. It is not used in the actual PLC program that gets executed. When the program is compiled all the labels are replaced with what they refer to from the definition section of the PLC program. For example the E-stop Input is defined like:

```
EStopOk_I IS INP11
Lube_0     IS OUT2
```

In this example, EStopOk_I is the name given to the INP data type. Similarly, Lube_0 is the name given to the OUT data type. In both cases, the keyword IS is part of the syntax for defining a name.

IS is used on every data type to define a name for that variable. There is also a built in functionality for defining constant data to have a name. Math can be done in the definition and previous definitions of constants can be used as long as the entire assignment is in parentheses. An example is:

```
DEFINED_CONSTANT_C IS (1+2+5*7)
SECOND_CONST_C     IS (DEFINED_CONSTANT_C*10)
```

Conditional Statement – IF/THEN

The Conditional Statement is used for every line of the PLC Program that can be executed and is synonymous with a rung in ladder logic. The first IF/THEN in a Program defines the start of the PLC program and the end of the Variable Definitions. The part of the line between the IF and the THEN must evaluate to a boolean value (true or false). If it cannot be resolved to a boolean value then an error is thrown at compile time. This means that Words must be checked with [Relational Operators](#) and a Word itself cannot be the test for truth. IF W1 > 10 THEN (OUT2) is a valid test whereas IF W1 THEN (OUT2) is not. There can be as many conditions on the truth of a Conditional Test as you need. They are all separated by [Logical Operators](#) such as OR(| |) and AND(&&).

There is no ELSE Keyword, but the functionality can be achieved by copying the previous test and putting parentheses around it with a NOT symbol in front of it.

The following table defines what can be done in the test part of the Conditional Statement. While the statements are legal it does not mean that they are typically used. For example, typically Stages are not checked in IF/THEN statements.

Data Type	Usage
Input	IF INP50 THEN (OUT50)
Output	IF OUT50 THEN (MEM50)
Memory Bit	IF MEM40 THEN SET STG2
Stage	IF STG2 THEN (OUT4)
Fast Stage	IF FSTG1 THEN (OUT1)
Word	IF W1 > 156 THEN (OUT30)
Double Word	IF DW2 > 4000000000000 THEN (MEM300)
Floating-point Word	IF FW1 > 5.432 THEN SET OUT3

Double-Floating-point Word	IF DFW1 > 8900.983201293 THEN (OUT80)
SV bit	IF SV_ENABLE_AXIS_1 THEN (MEM70)
SV Word	IF SV_PC_CYCLONE_STATUS_2 == 4 THEN (MEM65)
One-Shot	IF PD2 THEN SET OUT7
Timer	IF T1 THEN SET OUT 5 IF T1 > 4000 THEN SET OUT 6

Print Message – MSG

Printing a message is useful when the user needs to know about status changes. It is often used in ATC PLC programs to indicate what part of the tool change process is being executed. Error reporting is the other key usage of the PLC messaging functionality. There are two types of messages that can be used with the PLC Message functionality, Synchronous and Asynchronous. Printing synchronous messages is done in its own Stage that is SET only when a message needs to be printed. This Stage should be the last Stage in the PLC program. Asynchronous messages are usually printed right inline with the rest of the code. Synchronous messages are displayed only when the SV_STOP System Variable is SET and Asynchronous messages are printed immediately. There is not a practical limit to the number of messages that can be defined for usage.

The command to send a message is IF 1==1 THEN MSG W1 where W1 stores a correctly setup message value. A Word variable must be used with the MSG command.

Only one message can be displayed per pass of the PLC program. This explicitly does not mean that one each Async and Sync message can be displayed per pass. There is no queue of messages that get buffered and sent out eventually. If a new message is sent, it overwrites the previous one. Effectively, the last MSG command in the program is what is displayed on screen, if it has changed.

Worth noting is the fact that once a particular Synchronous or Asynchronous message has been sent, a different message number must be sent of the same type (Sync or Async) before that original message can be sent again. For Synchronous messages, in addition to sending a different message number, it must be a different nonzero message number. For example, if you send a Lube Fault Synchronous message, when that clears you must send another different (and nonzero) Synchronous message before the Lube Fault message can be displayed again. This means that if the Lube Fault message is displayed and then the fault is cleared, but no new message is sent, if Lube Fault occurs again, the PLC program will be in the fault state, but there will be no message displayed. It is very important to avoid this case as it will look as though there is no error, but CNC11 will not start a job due to SV_STOP being SET. It can be debugged by putting the Word variable in the first twelve words so that they are displayed on the first PLC Diagnostic screen.

CNC11 takes the value sent via the MSG command and looks in the plcmsg.txt file for the selected message and prints it to the screen. The method of formatting this value is to start with a 1 or 2 for Synchronous or Asynchronous respectively and then add the message number times 256. An example table is shown below. The Word Value column in the following table is what should be stored in the Word to be sent out with the MSG command.

Message Number	Type	Word Value	Notes
1	Synchronous	257	1 + 1*256
2	Asynchronous	514	2 + 2*256
25	Synchronous	6401	1 + 25*256
50	Asynchronous	12802	2 + 50*256

It is easiest and less error prone to write the message number once by defining a [Constant](#) to store to a word before messaging it out. Below are the defines used in both Asynchronous and Synchronous messages for the example program.

```
INP13_GREEN_MSG IS (2 + 1*256) ;258
INP13_RED_MSG IS (2 + 2*256) ;514
```

```

INP14_GREEN_MSG IS (1 + 3*256) ;769
NO_SYNC_MSG     IS (1 + 99*256) ;25345
NO_ASYNC_MSG    IS (2 + 100*256) ;25602

```

```

EStopOk_I       IS INP11
Async_I         IS INP13
Sync_I          IS INP14

```

```

Async_O         IS OUT13
Sync_O          IS OUT14

```

```

Sync_Cleared_M  IS MEM1
Stop_M          IS MEM2

```

```

Sync_W          IS W1
Async_W         IS W2

```

```

InitialStage    IS STG1
MainStage       IS STG2
SetErrorStage   IS STG3

```

```

;=====
InitialStage
;=====

```

```

;setup default Word values

```

```

IF 1==1 THEN Sync_W = NO_SYNC_MSG, Async_W = NO_ASYNC_MSG,
             RST InitialStage, SET MainStage

```

```

;=====
MainStage
;=====

```

```

IF !EStopOk_I THEN SET SV_STOP
IF SV_STOP THEN (Stop_M)

```

```

;prevent sync messages from showing up by resetting SV_STOP

```

```

IF !EStopOk_I && Sync_Cleared_M THEN RST Sync_Cleared_M, RST Sync_O, Sync_W = NO_SYNC_MSG
IF EStopOk_I && !Sync_Cleared_M THEN RST SV_STOP

```

```

;sync

```

```

;--if the Input is green, set the Sync message and override the Async messages

```

```

IF Sync_I THEN Sync_W = INP14_GREEN_MSG, SET SV_STOP, SET SetErrorStage, SET Sync_O
IF !Sync_I && Sync_O THEN SET Sync_Cleared_M

```

```

;async

```

```

;--show a message if the Input changes

```

```

IF Async_I THEN Async_W = INP13_GREEN_MSG, MSG Async_W, SET Async_O
IF !Async_I THEN Async_W = INP13_RED_MSG , MSG Async_W, RST Async_O

```

```

;=====
SetErrorStage
;=====

```

```

IF 1==1 THEN MSG Sync_W

```

```

;if the message has been cleared, reset this stage to allow Async messages

```

```

IF Sync_W == NO_SYNC_MSG && Sync_Cleared_M THEN RST SetError

```

The sample plcmsg.txt file use for the above example is:

```

1  9001 Input 13 Green

```

```
2  9002 Input 13 Red
3  9003 Input 14 Green
99 9099 No SYNC Message
100 9010 No ASYNC Message
```

plcmsg.txt

The plcmsg.txt file contains a list of all the messages that the PLC can send to CNC11. This facility is used to notify the user of status changes and fault conditions. The typical messages should not be overwritten by new custom messages, rather new numbers should be added. The format for each line of the plcmsg.txt file is as follows.

MessageNumber MessageLogNumber Message

There are three fields separated by one space each that must be setup for a line to be valid and usable. If the line is not formed correctly, you will not know it until the message is trying to display. The MessageNumber field is exactly the same number as the Message Number in the above table. The MessageLogNumber causes the printed message to be put in the msglog.txt file so that problems can be diagnosed by Tech. Support. The 9xxx series messages are reserved for PLC program usage. Make sure the Log Level is set to 4 in parameter 140 to ensure the messages are logged. Message is the useful text that will be printed along with the MessageLogNumber. It should contain a pithy message that informs the user about the change that occurred. All text to the end of the line is printed so no comments are allowed in this file. The standard plcmsg.txt file is listed below.

System Variables – SV_...

System Variables are Keywords with specific purpose and can find the full list in [Appendix D](#). System Variable names cannot be used verbatim as constant value labels, variable names, or as the beginning of either. Removing the underscores from the name, for example, gets around this problem. Attempting to do so will cause the program to fail at compile time.

Data Type Name

None of the names of Data Types used in the PLC program are allowed to be used as constant value labels or variable names. This means that INP1 is invalid as is just INP. Attempting to do so will cause the program to fail at compile time.

Indexes – Data Type[Data Type or Constant]

Using the Index ability is useful when writing generic programs that allow users to set an Input or Output that they want a function to occur at in parameters. It is very important to check for [Index Out of Range](#) errors if this functionality is used. The default value for the PLC Program parameters is 0.0 and all Data types start at 1, so an Index Out of Range error would occur right away. This cannot be used with System Variables. In the following example Parameter 171 is checked to make sure it is within a certain range, then an Output number is set based on the information it finds.

```
P171_W      IS W1
LubeOut_W   IS W2

;read the parameter
IF 1==1 THEN P171_W = SV_MACHINE_PARAMETER_171

;make sure the parameter is in range, if not default value is used
IF (P171_W <= 0) || (P171_W >= 10) THEN LubeOut_W = 2
IF !((P171_W <= 0) || (P171_W >= 10)) THEN LubeOut_W = P171_W

;set output based on parameter
IF SV_PROGRAM_RUNNING || SV_MDI_MODE THEN (OUT[LubeOut_W])
```

Range Selection – '..'

This functionality allows setting or resetting of Bit type Variables including System Variables. Two dots are placed a space apart from the beginning variable and ending variable. All variables between and including the two book-end variables are either SET or RST, regardless of whether they are being used for anything else or if they have a label. This can be useful in error conditions if you want to turn off all Outputs or Stages. Usage is as follows.

```
IF JPI1 THEN (PD1)
IF PD1 THEN RST OUT1 .. Out80
```

```
IF JPI2 THEN (PD2)
IF PD2 THEN SET OUT80 .. OUT1
```

DUMP

The DUMP command makes a “dump” of the values stored in the first 64 [Words](#), [Double Words](#), [Floating-point Words](#), and [Double-Floating-point Words](#) to debug_dump0.txt in the main cncx directory. DUMP should only be used sparingly and then only when debugging the initial implementation of a PLC program because of the cost of writing data to a file on the hard disk. This is useful for making sure Floating-point variables are correct. The PLC detective utility and the Live PLC IO display in CNC software have the ability to monitor and display the first 44 W and FW types and the first 11 DW and DFW types. In CNC12 v4.14+, the first 88 W types can be displayed.

An example of using this is as follows.

```
IF JPI1 THEN (PD1)
IF PD1 THEN DUMP
```

Operators

The operators in a PLC program are used to compare data, to set one piece of data equal to another, invert the data, etc. There are both unary and binary operators. The unary operators only require one variable or piece of data to operate on. An example of this is inverting a memory bit like this !MEM1 or turning a Memory Bit on like SET MEM1. A binary operator requires two pieces of data. Examples of using a binary operator are W1 >= W2 and W3=W4. There are no Bitwise Operators at present. This means that bit masking cannot be done by ANDing or ORing constant values with a Word. It can be done by [setting and resetting bits](#) and [shifting](#) as well. The following operators are all valid in a CNC11 PLC program.

Assignment – =

The equals sign is used to set a Word or Timer on the left of the equals sign to a Word, Timer or numerical value on the right of the equals sign. Some examples are listed below.

```
IF 1==1 THEN W1 = 10 ;Word1 is set to an integer value of 10
IF 1==1 THEN T1 = W1 ;Timer1 is also set to an integer value of 10 representing 10 ms
IF 1==1 THEN FW1 = 2.5 ;Floating-point Word1 is set to 2.5
```

Set – SET

SET turns on any of the Bit variables. The bit value is set to 1 and evaluates to true when checked. That is to say that any of the [Outputs](#), [Memory Bits](#), [Timers](#), [Stages](#), [Fast-Stages](#) and [System Variables](#) that are bits can have this keyword used on them. One-Shots cannot be SET. An example of using this is IF 1==1 then SET MEM1.

Data Types that can be used with SET	Example of using SET
Memory Bits	IF 1==1 THEN SET MEM2
Outputs	IF 1==1 THEN SET OUT2
Inputs	IF 1==1 THEN SET INP2

Timers	IF 1==1 THEN SET T2
Stages	IF 1==1 THEN SET STG2
Fast Stages	IF 1==1 THEN SET FSTG2
One-Shots	IF 1==1 THEN SET PD2

Reset – RST

Reset turns off any of the Bit variables. The bit value is set to 0 and evaluates to false when checked. That is to say that any of the [Outputs](#), [Memory Bits](#), [Timers](#), [Stages](#), [Fast-Stages](#) and [System Variables](#) that are bits can have this keyword used on them. One-Shots cannot be RST. An example of using this is IF 1==1 then RST MEM2.

Data Types that can be used with RST	Example of using RST
Memory Bits	IF 1==1 THEN RST MEM2
Outputs	IF 1==1 THEN RST OUT2
Inputs	IF 1==1 THEN RST INP2
Timers	IF 1==1 THEN RST T2
Stages	IF 1==1 THEN RST STG2
Fast Stages	IF 1==1 THEN RST FSTG2
One-Shots	IF 1==1 THEN RST PD2

Output Coil – ()

Output Coils are used to SET or RST any bit output based on the conditions before the THEN. If the test in the IF is true the output is SET, whereas if the test is false then the output is RST. The use of parenthesis does not constitute Output Coils unless they are used to the right of THEN on any program line. Output Coils cannot be used on Words. Be careful when using Coils because they can cause logical problems in your program. Do not put a variable in Coils on one line and then try to SET or RST it somewhere else in the program as well because it will change while moving through the PLC program and may have surprising results. If you are going to use a variable in Coils, all of the logic to turn it on or off must be on the same line to avoid trouble. Coils cannot be used on Timer Data types to start them counting because they are generally guaranteed to be turned off again on the very next pass of the PLC program. Some examples are shown below to illustrate some Coil concepts.

;basic Coil usage

IF 1==1 THEN (OUT1) ;always turn on OUT1

IF MEM1 THEN (OUT2); SET OUT2 if MEM1 is SET and RST OUT2 if MEM1 is RST

;potentially problematic Coil usage

IF MEM1 || INP5 THEN (OUT4) ;OUT4 is *guaranteed* to be SET or RST by this line

IF MEM3 THEN SET OUT4 ;OUT4 *may* be SET by this line if MEM3 is SET

IF INP10 THEN RST OUT4 ;OUT4 *may* be RST by this line if INP10 is SET

;better usage of Coil

IF (MEM1 || INP5) || MEM3 && !INP10 THEN (OUT4) ;all logic combined

Jump – JMP

Jump RSTs the current Stage and SETs another one. See [Stages](#) for the effects of turning on or off a stage. Execution does not jump around in the PLC program as in Assembler, but typically the stages are written one after the other so in essence it will move to that one. A sample usage of Jump is IF 1==1 THEN JMP STG3. If a JMP is called such that the current Stage is the stage being jumped to, then the Stage will still be set next time through the PLC program. Jump can only be used on Stage variables. It is not advised to JMP out of the MainStage, whereas it should be done out of the InitialStage. If you jump out of the MainStage and no other

stages are set, you must exit CNC11 and reboot the MPU11.

;typical usage

InitialStage

IF 1==1 THEN JMP MainStage ;InitialStage is RST and MainStage is SET.

Basic Math Operators – *, \, +, -, %

These are the basic four math functions, multiplication, division, addition and subtraction plus the Modulus operator. They are all binary operators. These operators are allowed to be used on Word types only. The modulus operator, %, is used to find the remainder of the division of two numbers.

Note that the calculation is done and then changed to the appropriate type when the value is assigned to a variable. Assignment of floating-point values to Integer Words results in the decimal point value being truncated. This means that IF 1==1 THEN W1 = 2.5*1 will result in W1 being set to 2. The following examples illustrate usage of some of the operators.

;multiplication

IF 1==1 THEN W1 = 15*2; W1 is set to 30

IF 1==1 THEN FW1 = 0.5*; FW1 = half of current encoder counts for axis 1

;division

IF 1==1 THEN W2 = 128 / 2; W1 is set to 64

;Modulus

IF 1==1 THEN W3 = 15 % 2; W3 is set to 1 because 15/2 = 7R1

IF 1==1 THEN W4 = 128 % 15; W4 is set to 8 because 128/15 = 8R8

Relational Operators – <, >, <=, >=, !=, ==

Relational Operators are only valid when comparing Words, Word type System Variables and Timers. Bit type variables cannot be compared with these operators. A relational operator compares two variables of the same type and results in a 1 or 0 as the output.

Be aware that checking Timers with Relational Operators does not indicate anything about whether the Timer has expired or not. It merely compares against the current count of ms since the Timer was Set.

Less Than or < checks to see if the variable on the left is less than the value on the right to evaluate to true.

Less Than Or Equal or <= checks to see if the variable on the left is less than or equal to the variable on the right to evaluate to true.

Greater Than or > checks to see if the value on the left is greater than the value on the right to evaluate to true.

Greater Than Or Equal checks to see if the value on the left is greater than or equal to the value on the right to evaluate to true.

Not Equal or != checks to see if two variables are not exactly equal to evaluate to true. Note that Floating-point != checks will most likely fail on calculated values due to rounding error. >= or <= should be used instead to check Floating-point numbers.

Is Exactly Equal To or == checks to see if two variables are exactly equal to each other to evaluate to true. The same warning applies to == as != about Floating-point numbers.

The following examples show how a sample PLC program line looks with a Relational Operator on it.

IF 1==1 THEN W1 = 10, W2 = 10

IF W1 > W2 THEN (OUT1) ;Out1 is RST because W1 is not greater than W2

IF W1 == W2 THEN (OUT2) ;Out2 is SET because W1 is equal to W2

Logical Operators – !, &&, ||, XOR or ^

Logical Operators are used to compare or change the state of Bit type variables including System Variables.

&& is the operator for AND. This means that both sides of the operator must be true to have the statement evaluate to true. The following truth table shows all the options for evaluating a binary expression.

|| is the operator for OR. This means that only one side of the operator needs to be true for the statement to evaluate to true.

XOR or ^ are the operators for Exclusive OR. This means that one of the sides of the operator must be true and the other false to have the statement evaluate to true.

A	B	A && B	A B	A ^ B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

The ! is a unary operator for NOT that inverts the state of a bit whether it is true or false.

Bit Value	Result
0	1
1	0

Logical Operators cannot be used on Word Type variables. The results of multiple Relational checks can be combined with Logical checks for more complex statements, however. All of the following lines are valid PLC code. Often it increases readability to use parentheses around conditions to ensure correct interpretations.

```
IF MEM1 && INP2 THEN (OUT1)
IF (W1 > W2) || !MEM4 THEN (OUT5)
IF !MEM1 && INP2 || STG1 || FSTG1 && OUT3 XOR PD1 && T3 XOR SV_PC_POWER_AXIS_1 THEN (OUT6)
```

Convert to Word – BTW

BTW takes a range of Inputs, Outputs or Memory Bits and does a binary to decimal conversion, storing the result in a Word. The default number of bits is 8, but a value of 1 to 32 is allowed. The first Bit is treated as the Least Significant Bit and the next bit is the next highest number of Input, Output, Memory Bit. For example MEM1 to MEM8 are set, from lowest to highest, as 0110 0100 where 0 is red and 1 is green. When BTW is executed the number is converted as $2+4+32 = 38$ decimal. Note that the number of bits specified is a count number to use rather than a specific Bit number to convert up to and including. This means, starting at Bit 0, how many of the bits should be converted. The following example will setup the bits as the example above in code.

```
;setup the binary information
IF 1==1 THEN RST MEM1, SET MEM2, SET MEM3, RST MEM4
, RST MEM5, SET MEM6, RST MEM7, RST MEM8
;evaluate the Memory Bits as a Binary number and convert it to Decimal, storing in W1
IF 1==1 THEN BTW W1 MEM1
```

Convert to Binary – WTB

WTB converts a Word type variable to Binary and writes to Outputs or Memory Bits. By default the lowest 8-bits of Binary are written. The lowest bit goes into the first bit and so on up to the highest Bit. An Integer number from 1-32 can be specified to print from 1 to 32 of the Binary bits to either Memory or Output locations.

Note that the number of bits specified is a count number to use rather than a specific Bit number to convert up to and including. This means, starting at Bit 0, how many of the bits should be converted.

Note that the Binary bits will write to the next consecutive bits regardless of any other meaning in the PLC program. If you do a default WTB in a Memory section that only has 5 free bits, three of them will change something you likely do not want to change.

For example the Word W1 has a value of 12345 in decimal notation and 11000000111001 in Binary notation. A standard WTB W1 MEM1 will print 00111001 with the Least Significant Bit written to MEM1 and the Most Significant Bit written to MEM8 as seen on PLC Diagnostics. In the first example the default WTB is used and 8 bits will be written out. In the second example 12 bits will be written to OUT17 to OUT28.

Data Types that can be used with WTB	Example using WTB
Memory Bits	IF 1==1 THEN WTB W5 MEM10
Outputs	IF 1==1 THEN WTB W5 OUT17 12

Convert to Binary Coded Decimal – BCD

BCD is used primarily for interfacing with external devices that need information in Binary Coded Decimal format. BCD is different than Binary in that each number in a decimal number is converted to a 4-bit binary number.

This means that to represent the decimal number 125 in BCD the number must be broken up into the ones, tens and hundreds positions and each of those numbers is converted to a Binary number. The Binary numbers for 1, 2, and 5 are 0001 0010 0101 which is the BCD version of 125.

The following table shows the conversion for the Decimal and BCD values. To use this feature on outputs, four outputs must be used for each place in decimal that the external device will be looking for. In the case of an ATC with 24 tools two sets of four outputs must be used to represent all possible numbers. An example of using the BCD command is: IF 1==1 THEN BCD W2. The Word can then be sent to the correct Outputs with the WTB command.

Note: The largest Decimal value that can be converted is 8 digits because only 32-bit Integer Words can be used for the BCD command. This gives 32-bits divided by 4 bits gives 8 decimal places. This means that the BCD of a number greater than 99,999,999 is invalid and undefined. For reference, the bits higher than 8 places are truncated and cannot be recovered.

Decimal	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Convert from Binary Coded Decimal – BIN

BIN converts a Word value that is in BCD format to standard Binary format. When the Word is displayed on the PLC Diagnostic Screen it is shown as a Decimal number. The WTB command can be used to then print the Binary version of the number to Memory Bits or Outputs if desired. A simple use case of the BIN command is IF 1==1 THEN BIN W1.

Note: There is no checking on the BCD number to determine if it is a valid BCD number. There are many invalid BCD numbers in each group of four digits, which could be checked in a PLC program. In every group any BCD number greater than 1001 or 9 is invalid because in decimal notation each place can only hold 0-9. For example a BCD number 57004 is completely invalid because in Binary it is 1101 1110 1010 1100 all of which are above 1001.

One way to test the BCD number is outlined below.

```

;read the 4 bits in from inputs for one BCD number
IF 1==1 THEN BTW W1 INP1 4
;create a copy of the word
IF 1==1 THEN W2 = W1
;convert the BCD number to Binary
IF 1==1 THEN BIN W2
;convert the now Binary number back to BCD
IF 1==1 THEN BCD W2
;check to see if they are the same BCD again
IF W2 != W1 THEN SET BCD_Fault_MEM, AsyncW = 258, MSG AsyncW

```

Set or Reset a Bit in a Word – BITSET / BISTRST

When manipulating binary data it is often desirable to SET or RST only certain bits in the data. BITSET and BISTRST can be used on Words or Word Type System Variables. Depending on the command used, the specified bit from 0-31 will be changed in the specified Word. Some System Variables that are Integer Word values are actually 16 to 32 different Binary Bits that can be checked, for example SV_PC_CYCLONE_STATUS_1. Note that the Bit number range is different than the one used in WTB and BTW. The Bit number is the actual Bit that is checked. Because Bit numbering starts at zero, when you reference a Bit by number you are checking the “Bit number + 1” Bit.

```

IF 1==1 THEN BITSET SV_PLC_DEBOUNCE_1 5 ;Turn on Bit 6 in the first Debounce SV
IF 1==1 THEN BISTRST W2 20 ;Turn off Bit 21 in Word 2

```

Check if a Bit is Set in a Word – BITTST

BITTST checks the specified bit in the specified Word or System Variable and SETs a Memory Bit if true and RSTs the Memory Bit if it is false. The range of bits that can be tested is 0-31. Note that the Bit number range is different than the one used in WTB and BTW. The Bit number is the actual Bit that is checked. Because Bit numbering starts at zero, when you reference a Bit by number you are checking the “Bit number + 1” Bit. In the example below Word number one, Bit 17 is checked and the state of it is copied to Memory Bit one.

```

IF 1==1 THEN BITTST W1 16 MEM1

```

Left / Right Shift Bits in a Word – LSHIFT/RSHIFT

Left and Right Shift are used on Words to move bits to the left or right. Right Shift is a Logical Shift rather than an Arithmetic Shift. This means that, as data is shifted, upper bit positions will always be filled with zero.

A pictorial example of shifting is in the following table with the same commands shown in code afterwards. The left shifting adds zeroes to the right-hand side to bump the number to the left first. The decimal value is 3735928559 originally and then after the left shift it becomes 4009754624. Finally, the right shift adds zeroes to the left-hand side to bump the number to the right and generate a decimal 61184. Notice that this acts as a mask and removes all except the second 8-bits.

Operation	Binary 32-bit Word value
Original Value	1101 1110 1010 1101 1011 1110 1110 1111
Left Shift 24 bits	1110 1111 0000 0000 0000 0000 0000 0000
Right Shift 16 bits	0000 0000 0000 0000 1110 1111 0000 0000

```

IF 1==1 THEN LSHIFT W1 24
IF 1==1 THEN RSIFT W1 16

```

Trigonometric Functions – SIN, ASIN, COS, ACOS, TAN, ATAN2

Trig. Functions Sine, ArcSine, Cosine, ArcCosine, Tangent and ArcTangent2 are available in the PLC program.


```
IF 1==1 THEN JMP MainStage
```

```
;=====
MainStage
;=====
IF JPI1 THEN (PD1)
IF PD1 THEN DUMP, RST MainStage
```

The following list shows what the results are from the above calculations. You can follow along on your PC with the calculator as long as you enter the first three values in Radians.

PLC Dump Start

```
FW1: 0.52359879   FW2: 1.04719758
FW3: 0.78539819   FW4: 0.50000000
FW5: 0.49999997   FW6: 1.00000012
FW7:30.00000000   FW8:60.00000000
FW9:45.00000000
```

Square Root – SQRT

The square root of a number is a different number that, when multiplied by itself results in the given number. Numbers that have the square root taken of them should be stored into a Floating-point Word or Double-Floating-point Word because they will most likely be non-Integer numbers. SQRT results can be stored into a Word, but the non-Integer portion will be truncated. This operation should rarely be used due to the time intensive nature of the operation.

An example of using the SQRT function follows.

```
IntSqrt_W IS W1
Sqrt_FW   IS FW1
;take the square root of 2 and store to a Floating-point Word
IF 1==1 THEN Sqrt_FW = SQRT (2) ;returns 1.41421354
;take the square root of 17 and store to a Word
IF 1==1 THEN IntSqrt_W = SQRT(17) ;returns 4.12310562, but 4 stored into the Word
```

Raise Number to a Power – POW

Raising a number to a power means multiplying the number by itself some number of times. The POW command takes two arguments, the first being the number to raise and the second being the number of times to multiply the first by itself. Both numbers can be floating-point values. If the second number is relatively small, it is usually faster to just multiply the number out, for example $2*2*2$ instead of 2^3 . The following example shows how to use the POW command.

```
pow_FW     IS FW1
pow2_FW    IS FW2

;raise 2 to the 3rd power
IF 1==1 THEN pow_FW = POW(2, 3) ;returns 8.00000000
;raise 2 to the 3.5th power
IF 1==1 THEN pow2_FW = POW(2, 3.5) ;returns 11.31370831
;raise 300 to the 1/8th power
IF 1==1 THEN pow3_FW = POW(300, 0.125) ;returns 2.04004693
```

Absolute value – ABS

Absolute value.

```
If 1==1 THEN W1 = -42,  
             W2 = ABS(W1) ; W2 = 42
```

Standard PLC Program Layout

A PLC program is laid out with all definitions at the top of the program until the first IF statement is used. From then on no more definitions are allowed and will generate an error at compile time. When the PLC program is executed it runs from top to bottom and executes every line except under certain circumstances, specifically Stages. Top to bottom execution can lead to logic errors where something is SET in one part of the program and then reset somewhere else. Wherever possible all conditions that could change an output should be gathered together.

The basic functionality described in this chapter is based on the basic DC3IOB PLC program.

The PLC program needs to have certain sections to achieve a minimum of real-world functionality as outlined in this chapter. The PLC program should always check all of the connections between the MPU, the PLC board including miniPLC boards and the drive board. In order for a Jog Panel or Keyboard Jogging to work they must be written into the PLC program.

Definitions are used to make reading the program easier and as such, the names used for the definitions should reflect the function and polarity when tripped, on or Green in PLC Diagnostics. This means that if a variable is Normally Closed it is named something like XminusLimitOk. The logic to check it becomes IF ! XMinusLimitOk THEN SET MEM9. If the variable is Normally Open, the naming should be inverted. For a Normally Open Lube Fault input the logic to check it becomes IF LubeFault THEN SET MEM90.

At a minimum a WatchDogStage, InitialStage and MainStage should be used in every PLC program. The basic PLC program has many more stages for things like spindle control, Lube timers, Checking System Variables, etc. The InitialStage is used to setup any timers and default states. The MainStage is set in the InitialStage which then resets itself and is never SET again while CNC11 is running. This means that the InitialStage will only run once for each time the CNC software starts. Exiting CNC11 and restarting it causes the InitialStage to run again.

Programming Conventions

It is helpful when debugging and reading a program to know what type a variable is without having to constantly search through the multiple uses of a name to get to the definition at the top of the program. Following is a table of basic suggestions of ways the types can be named to reduce confusion. Inputs and Outputs are typically named to indicate the purpose rather than applying an extra label to them. The basic idea is to put something like the code specific type name at the end of the declaration. Whether you put an underscore between the name and type, add the letters in all Caps, or capitalize the first letter is up to you.

Type	Example Name	Comments
Constants	AXIS_FLT_CLR_C	SCREAMING_SNAKE_CASE, end with _C
Input	EStopOk_I	PascalCase, Suffix with _I
Output	LubeOut_O	PascalCase, Suffix with _O
Memory Bit	SpinFault_M	PascalCase, Suffix with _M
Word	Axis3FiberOk_W	PascalCase, Suffix with _W
Double Word	BigCounter_DW	PascalCase, Suffix with _DW
Floating-point Word	SpindleRangeMultiplier_FW	PascalCase, Suffix with _FW
Double-Floating-point Word	PreciseNumber_DFW	PascalCase, Suffix with _DFW
Timer	CycloneStatus_T	PascalCase, Suffix with _T
One-Shot	SlowFast_PD	PascalCase, Suffix with _PD PD is Positive Differential, meaning rising edge
Stage	InitialStage	PascalCase, Suffix with Stage or _STG

Fast Stage	CountSomething_FSTG	PascalCase, Suffix with Stage or _FSTG
CNC System Variables	DoToolCheck_SV	PascalCase, Suffix with _SV, Prefix with Do or Select. Note: These are Defined CNC SystemVariables for easier reading of PLC.
Keyboard Keys	Kb_a_SV	PascalCase, Prefix with Kb_, Suffix with _SV Kb is short for Keyboard
M-Codes	M6_SV, PutbackComplete_SV	PascalCase, Ends with _SV Note: Typically is named after M-Code used such as M6, however can also describe action that it activates, such as PutbackComplete_SV or DustFootActivate_SV.
Skin Events	SkinSpinAutoMan_SV	PascalCase, Prefix with Skin, Suffix with _SV

Stages

While technically no Stages are required to be explicitly used for a program to compile, they should be used. The benefits include allowing debug of small sections of code by turning off other stages, reduced Program running time by running only certain things all the time, and compartmentalizing code for easier comprehension.

Stage names in the program are surrounded on lines above and below with a full width line of equal signs to designate the start of a stage. PascalCase is used to name Stages and the same naming should be used in the program. Type the Stage name 20 columns from the left.

Section Titles

Section titles within a stage have a full width line of minus signs, and also include the name of the section in that line or not as preference dictates. The name should start after 5 minus signs either way and optionally fill out the rest of the line with minus signs.

```
;-----INPUT DEFINITIONS-----
```

Section explanations longer than about 5 lines should have a full width line of minus signs before and after the section to show it is a block of comments. Also every line between the two bracketing lines should have a semi-colon to show it is part of the explanation.

Section explanations shorter than about 5 lines should just have a semi-colon in front of each line with no spaces.

```
;INP769 - INP784 encompass the MPU11 onboard input connections
;which are generally used for MPG and probing functions.
```

General Formatting

The compiler is not case sensitive so EstopOK, ESTOPOK, EsToPoK, and estopok are all references to the same Input. Convention states that capitalization be kept uniform for variable references, but it is not required by the compiler.

All lines should end after 79 columns to prevent printing issues and ensure readability on smaller monitors.

In the definition section the IS should line up with all others above and below in the section, not with the entirety of the defines. This increases readability while maximizing comment space. There should be two spaces between the longest variable name in the section and the IS keyword. Also, if possible, line up the comments after the definition.

Use only spaces in the PLC program, never use tabs unless you can figure out how to replace tabs with spaces in your editor. It is helpful to show non-printing characters as well to make sure trailing spaces are not causing extra wrapping of text.

System Variables with names like SV_STOP should not be redefined to a Memory Bit in order to avoid

When the software starts again, E-Stop must be cycled and then motion can be commanded again. SV_STOP is SET and the InitialStage is run whenever the software starts up again.

Checking PLC Fault and Software Ready

```
;=====
WatchDogStage
;=====

; Handle PLC executor faults. The only way to reset a PLC executor fault
; is to reboot the MPU11.
IF SV_PLC_FAULT_STATUS != 0
  THEN PLC_Fault_W = SV_PLC_FAULT_STATUS,
  PLCFaultAddr_W = SV_PLC_FAULT_ADDRESS,
  ErrorCode_W = PLC_EXECUTOR_FLT_MSG, MSG ErrorCode_W,
  SET PLCExecutorFault_M, RST SetErrorStage, SET SV_STOP

; Handle software exit.
IF !SV_PC_SOFTWARE_READY && (SV_PLC_FAULT_STATUS == 0)
  THEN SET SoftwareNotReady_M,
  SET SV_STOP,
  ErrorCode_W = SOFTWARE_EXIT_MSG

if SV_PC_SOFTWARE_READY && (SV_PLC_FAULT_STATUS == 0) THEN (SoftwareReadyPD)
IF SoftwareReadyPD && !SoftwareNotReady_M || !True THEN SET InitialStage

IF SoftwareReadyPD && SoftwareNotReady_M THEN RST SoftwareNotReady_M

;=====
MainStage
;=====
;This is just sample code to generate the errors, they should not actually be done in
;a ;real PLC program. To cause either of the errors uncomment the lines starting with IF

;divide by zero
;If 1==1 THEN Word2 = 10/0

;index out of bounds
;IF 1==1 THEN Word3 = 2500, Word4 = 2500
;IF 1==1 THEN Word5 = Word3 + Word4
;IF 1==1 THEN SET OUT[Word5]
```

Jog Panel and Keyboard Jogging

This is a significant part of the PLC program and as such is not transcribed from the [basic](#) program.

Nearly all Jog Panel functions that can be caused by key presses require Keyboard Jogging to be SET in Parameter 170 bit 0 and RST in Parameter 148 bit 1. The exceptions are Escape and Rapid Override.

There are some keys that require the Keyboard Jog Panel to be on screen as well as have Keyboard Jogging enabled. These functions are primarily limited to Jogging.

Axis Enable

The axes are checked for motion related fault conditions including a drive fault, stall error and fiber connections.

Fiber/Wire Connection Checking

There are two kinds of communication used on MPU11 systems. The DriveBus is used for communication related to motor control and can be either fibers 4 and 5 or the Drive Communication wiring for expanding the number axes that can be controlled. The PLCBus is on fibers 1 and 3 and is used for controlling I/O. If the Buses are being used, the connections must be checked to ensure communication problems are not clouding troubleshooting.

Checking the connections only needs to be done a few times a second to be effective. This means having a stage that is set by a Timer to read the information. Both of the following sections use the same logic to check for errors, but the first checks both connection methods and the second only checks the PLCBus. The third section needs to be added if an expansion board is added to the PLC.

Drive and PLCBus Checking

Presently the Optic4 and DC3IOB have DriveBus Fiber connections. The Optic4, DC3IOB and ALLIN1DC have Drive Communication In and/or Out Wire connections. The ALLIN1DC has an MPU11 on-board which means there are no visible Fiber connections, but the Drive and PLCBuses are still used and should be checked. If a system using 3rd party drives is used with more than four axes, the Optic4 must be used in conjunction with the GPIO4D and the DriveBus Fiber Checking must be done as well.

```
;=====
CheckCycloneStatusStage
;=====
; Due to amount of time it takes to retrieve data from the cyclone, this stage
; is only called few times per second to help reduce scan time of the main PLC
; program.

; The logic below is the equivalent to the following:
; IF true THEN BITTST SV_PC_CYCLONE_STATUS_2 0 Axis1FiberOk_M,
;           BITTST SV_PC_CYCLONE_STATUS_2 1 Axis2FiberOk_M,
;           BITTST SV_PC_CYCLONE_STATUS_2 2 Axis3FiberOk_M,
;           BITTST SV_PC_CYCLONE_STATUS_2 3 Axis4FiberOk_M,
;           BITTST SV_PC_CYCLONE_STATUS_2 4 Axis5FiberOk_M,
;           BITTST SV_PC_CYCLONE_STATUS_2 5 Axis6FiberOk_M,
;           BITTST SV_PC_CYCLONE_STATUS_2 6 Axis7FiberOk_M,
;           BITTST SV_PC_CYCLONE_STATUS_2 7 Axis8FiberOk_M
IF true THEN WTB SV_PC_CYCLONE_STATUS_2 Axis1FiberOk_M

; Generate some messages for fiber or wire to MPU11 having issues
IF SV_AXIS_VALID_1 && !SV_DRIVE_ONLINE_1 THEN ErrorCode_W = AXIS1_INFLT,
                                           SET DriveComFltIn_M
IF SV_AXIS_VALID_2 && !SV_DRIVE_ONLINE_2 THEN ErrorCode_W = AXIS2_INFLT,
                                           SET DriveComFltIn_M
IF SV_AXIS_VALID_3 && !SV_DRIVE_ONLINE_3 THEN ErrorCode_W = AXIS3_INFLT,
                                           SET DriveComFltIn_M
IF SV_AXIS_VALID_4 && !SV_DRIVE_ONLINE_4 THEN ErrorCode_W = AXIS4_INFLT,
                                           SET DriveComFltIn_M
IF SV_AXIS_VALID_5 && !SV_DRIVE_ONLINE_5 THEN ErrorCode_W = AXIS5_INFLT,
                                           SET DriveComFltIn_M
IF SV_AXIS_VALID_6 && !SV_DRIVE_ONLINE_6 THEN ErrorCode_W = AXIS6_INFLT,
                                           SET DriveComFltIn_M
IF SV_AXIS_VALID_7 && !SV_DRIVE_ONLINE_7 THEN ErrorCode_W = AXIS7_INFLT,
                                           SET DriveComFltIn_M
IF SV_AXIS_VALID_8 && !SV_DRIVE_ONLINE_8 THEN ErrorCode_W = AXIS8_INFLT,
                                           SET DriveComFltIn_M

; Generate some messages for fiber or wire to drive having issues
IF SV_AXIS_VALID_1 && SV_DRIVE_ONLINE_1 && SV_MASTER_ENABLE && !Axis1FiberOk_M
```

```

    THEN ErrorCode_W = AXIS1_OUTFLT, set DriveComFltOut_M
IF SV_AXIS_VALID_2 && SV_DRIVE_ONLINE_2 && SV_MASTER_ENABLE && !Axis2FiberOk_M
    THEN ErrorCode_W = AXIS2_OUTFLT, set DriveComFltOut_M
IF SV_AXIS_VALID_3 && SV_DRIVE_ONLINE_3 && SV_MASTER_ENABLE && !Axis3FiberOk_M
    THEN ErrorCode_W = AXIS3_OUTFLT, set DriveComFltOut_M
IF SV_AXIS_VALID_4 && SV_DRIVE_ONLINE_4 && SV_MASTER_ENABLE && !Axis4FiberOk_M
    THEN ErrorCode_W = AXIS4_OUTFLT, set DriveComFltOut_M
IF SV_AXIS_VALID_5 && SV_DRIVE_ONLINE_5 && SV_MASTER_ENABLE && !Axis5FiberOk_M
    THEN ErrorCode_W = AXIS5_OUTFLT, set DriveComFltOut_M
IF SV_AXIS_VALID_6 && SV_DRIVE_ONLINE_6 && SV_MASTER_ENABLE && !Axis6FiberOk_M
    THEN ErrorCode_W = AXIS6_OUTFLT, set DriveComFltOut_M
IF SV_AXIS_VALID_7 && SV_DRIVE_ONLINE_7 && SV_MASTER_ENABLE && !Axis7FiberOk_M
    THEN ErrorCode_W = AXIS7_OUTFLT, set DriveComFltOut_M
IF SV_AXIS_VALID_8 && SV_DRIVE_ONLINE_8 && SV_MASTER_ENABLE && !Axis8FiberOk_M
    THEN ErrorCode_W = AXIS8_OUTFLT, set DriveComFltOut_M

If !EstopOk THEN rst DriveComFltIn_M, rst DriveComFltOut_M
If DriveComFltOut_M || DriveComFltIn_M THEN set AxisFault_M

;check PLC status bit
IF TRUE THEN BitTst SV_PC_CYCLONE_STATUS_1 21 PLCBusExtDevEn_M

;check input fiber
IF !SV_PLC_BUS_ONLINE THEN ErrorCode_W = PLC_INFLT,
                        rst PLCBus_Oe_M, set PLCFault_M

;check output fiber
IF SV_PLC_BUS_ONLINE && PLCBus_Oe_M && !PLCBusExtDevEn_M
    THEN ErrorCode_W = PLC_OUTFLT, SET PLCFault_M

;clear PLC errors
IF PLCFault_M && SV_PLC_BUS_ONLINE && PLCBusExtDevEn_M && !EstopOk
    THEN RST PLCFault_M, ErrorCode_W = PLC_FLT_CLR, SET PLCBus_Oe_M

IF True THEN RST CheckCycloneStatusStage

```

PLCBus Checking only

The GPIO4D communicates to the MPU11 only over the PLCBus. If a system only has a GPIO4D then only the PLCBus should be checked. This section details the PLC Fiber checking only. The errors are checked for and set in the AxesEnableStage.

```

;=====
    CheckCycloneStatusStage
;=====

; Due to amount of time it takes to retrieve data from the cyclone, this stage
; is only called few times per second to help reduce scan time of the main PLC
; program.

;check PLC status bit
IF True THEN BITTST SV_PC_CYCLONE_STATUS_1 21 PLCBusExtDevEn_M

```

```

;check input fiber
IF !SV_PLC_BUS_ONLINE THEN ErrorCode_W = PLC_INFLT,
    RST PLCBus_Oe_M, SET PLCFault_M

;check output fiber
IF SV_PLC_BUS_ONLINE && PLCBus_Oe_M && !PLCBusExtDevEn_M
    THEN ErrorCode_W = PLC_OUTFLT, SET PLCFault_M

;clear PLC errors
IF PLCFault_M && SV_PLC_BUS_ONLINE && PLCBusExtDevEn_M && !EstopOk
    THEN RST PLCFault_M, ErrorCode_W = PLC_FLT_CLR, SET PLCBus_Oe_M

IF True THEN RST CheckCycloneStatusStage

```

MiniPLCBus Checking

This section should be added to any PLC program that uses the PLCADD1616 or ADD4AD4DA expansion boards. Specifically the checking should be added to the CheckCycloneStatusStage. Only the MiniPLCBus Online bits are checked in this section, but they should be added to the original Stage. Only the Expansion headers that are used should be checked. There is no equivalent to the SV_Axis_Valid_1 - _7 System Variables. Looking in the mpu_info.txt file will show what expansion boards are plugged in to the headers.

```

;add to Constant Defines
MINI_PLC_1_FLT_MSG IS (1 + 256*60); 15361

;add to CheckCycloneStatusStage
;check the first Expansion board
IF 1==1 THEN BITTST SV_PC_MINI_PLC_ONLINE 0 ADD1616ok1_M
IF !ADD1616ok1_M && PLCBus_Oe_M THEN
    ErrorCode_W = MINI_PLC_1_FLT_MSG, SET SetErrorStage, SET PLCFault_M

```

LubeTimers

There are two options for setting up automatic lubing in the basic PLC program. The first type is used for Lube pumps with no timer internally. This means that the PLC program must turn the output on and off to power the Lube pump correctly. The second type is for Lube pumps with internal timers. If Parameter 179 is set to a zero, then the pump timers will be used, otherwise the PLC program will keep track of the time that a job is running to determine when the next lube cycle should start.

Lube Pump Internal Timer

```

;=====
                                LubeUsePumpTimersStage
;=====

; METHOD 1 (SS == 0) For lube pumps with internal timers.
;

```

```

; When using this method, P179 should be set such that MMM is a
; value that is greater than the cycle time set on the internal timers and
; SS should be set to zero. How much greater MMM needs to be depends on the
; accuracy of the lube pump timers, but it is better to be on the long side
; to ensure proper operation.
;
; Example 1. The internal lube cycle interval is set to 60 minutes.
;         Set P179 = 7500. In this example, as long as the accuracy
;         of the lube timer interval causes the lube to turn on
;         within 75 minutes, it will work. Note that the amount of time
;         that lube is output is usually set with another timer control
;         on the lube pump and it does not factor into the setting of P179.
;
; It should be noted that lube pumps with internal timers may differ on how
; they operate.
;
; (a) For pumps that lube immediately when power is applied and then start timing
; until the next cycle, it is possible to run out of lube quickly on short job
; runs if, after the program has been run, lube power is removed.
;
; (b) For pumps that do not lube until it has been turned on for the interval time,
; it is possible that lube never gets applied if, after the short program has been run,
; lube power is removed.
;
; A short program or job run is defined as a job that finishes before
; the interval setting (60 minutes in the above example).
;
; For the above mentioned reasons, we want the power to be applied for at least
; the amount of time set by the interval timer, noting that if the user decides
; to engage the E-stop to remove power after short jobs, then they risk the
; above mentioned problems according to the type of pump.
;
; On the start of SV_PROGRAM_RUNNING, the lube pump turns on.
; The lube pump is turned off when a program has NOT been
; running continuously for MMM minutes or E-stop is engaged.
; The reason the lube pump is turned off after a program has NOT been
; running for MMM minutes is to prevent lubing when the user leaves for the
; weekend, leaving the machine on and E-stop disengaged.

```

```

IF (SV_PROGRAM_RUNNING || SV_MDI_MODE) THEN SET Lube, RST LubeM_T
IF !(SV_PROGRAM_RUNNING || SV_MDI_MODE) THEN LubeM_T = LubeM_W, SET LubeM_T
IF LubeM_T || !EStopOk THEN RST Lube

```

Lube Pump External Timer

```

;=====
;
;                               LubeUsePLCTimersStage
;=====
;
; METHOD 2 (SS != 0) For lube pumps that do not have internal timers.
;
; When using this method P179 should be set so the lube turns on
; every MMM minutes for SS seconds.
;
; Example 1.
;   To set the lube pump power to come on for 5 seconds
;   every 10 minutes, set P179 = 1005.
;
;                               MMMSS
; Example 2.
;   To set the lube pump power to come on for 30 seconds
;   every 2 hours, set P179 = 12030
;
;                               MMMSS
;
; This method will accumulate time while a program is running until
; it reaches MMM minutes, at which time it will apply power
; for SS seconds (unless E-stop is engaged) and then start over. It is
; possible with frequent use of E-stop that a lube cycle is cut short.
;

IF (SV_PROGRAM_RUNNING || SV_MDI_MODE) THEN LubeM_T = LubeM_W, SET LubeM_T
IF !(SV_PROGRAM_RUNNING || SV_MDI_MODE) THEN (StopRunningPD)
IF StopRunningPD THEN LubeAccumTime_W = LubeAccumTime_W + LubeM_T, RST LubeM_T
IF LubeM_T || (LubeAccumTime_W + LubeM_T > LubeM_W)
    THEN SET Lube, LubeS_T = LubeS_W, SET LubeS_T, RST LubeM_T, LubeAccumTime_W = 0
IF LubeS_T || !EStopOk THEN RST Lube, RST LubeS_T

```

Feedrate Override

Feedrate Override allows changing the master feedrate or commanded velocity if it is enabled. The basic feedrate override starts by reading the Feedrate potentiometer then scaling it to 0% to 200% from 0 to 256. The program then determines whether the Feedrate knob or the keyboard feedrate keys should be applied to CNC software. The maximum value is limited by parameter 39. Parameter 78 allows the feedrate value to be adjusted down if the spindle speed does not keep up with the command so CNC software gets to see if it wants to modify the Feedrate Override. Finally the PLC program has one last chance to change the new value coming back from the CNC software, though typically it should not.

There is now a feature that allows using keyboard and Knob override at the same time. By default the Knob is used, but if the Feedrate Override keyboard keys are used, then the Feedrate override displayed is based on the Keyboard value and not the Knob value. When the Knob is turned again more than 3%, however that value is displayed.

Spindle Functionality

Spindle DAC Output

Precise spindle speed is controlled through an inverter by sending an analog signal from the PLC to tell the inverter what speed to go at. The inverter must be calibrated to the analog signal with a tachometer or spindle encoder. This section only details setting up the PLC side of inverter control. Spindle Digital to Analog output or DAC is set by the PLC Program based on the commanded Spindle Speed from the 'S' command in G-Code, the Spindle Override value and the Spindle Gear Ratio. The DAC Spindle Speed command won't be seen by MPU11 until the DoSpindleStart_SV (SV_PLC_FUNCTION_37) goes from off to on or DoSpindleStop_SV (SV_PLC_FUNCTION_38) goes from on to off. This means that the Spindle Start Key cannot be held down at boot and have the spindle start as the software starts. This is a good thing. Both or either one of the Start or Stop can be used to control turning the Spindle on and off. Keep in mind that every time the Start or Stop changes state in the program, the DAC output will be updated, either turning on or off. This means that in most cases you should choose one of the two variables to use, rather than both, to avoid confusion.

The Spindle Analog Output bits are mapped to different places on different PLCs. The following table outlines the MPU11 board and the outputs for the Spindle Analog.

PLC	Spindle DAC Outputs
ALLIN1DC	241-252
DC3IOB	17-28
GPIO4D	305-316

Spindle Gear Ranges

The basic PLC programs have two ranges defined. Low Range and High Range. The High Range multiplier is taken from Parameter 33 and Low Range is taken from Parameter 65.

The example code is for a DC3IOB PLC program. The Inputs and Outputs for each of the standard PLC programs is detailed in [Appendix E](#).

MPG Operation

The MPG or Manual Pulse Generator can be used to supplement Jog buttons. This section is used to enable the MPG mode control in CNC software for the standard Centroid CNC MPG. This includes provisions for up to 4 axis in the basic PLC program with three step amounts. Note that when the MPG is active, with the way this is programmed, jogging will not work. It can be written into the PLC program that when a Jog Key is pushed to turn off MPG mode temporarily. Windup mode is used to make sure that the motor moves all of the commanded steps that the MPG sent. This is not desirable in x100 mode because turning the MPG fast enough will result in motion well after the MPG has stopped turning.

```

;=====
MPGStage
;=====
;
MPG Functions
;
Turn on/off Jog Panel MPG LED & on the MPG
IF MPGKey THEN (MpgPD)
IF MpgPD && MPGLLED THEN SET MPGManOfffflag_M
IF !SV_MPG_1_ENABLED || (MpgPD && !MPGLLED) THEN RST MPGManOfffflag_M

IF (MpgPD && !MPGLLED) || (SV_MPG_1_ENABLED && !MPGManOfffflag_M) &&
    !SV_PROGRAM_RUNNING THEN SET MPG_LED_OUT, SET MPGLLED

IF (!SV_MPG_1_ENABLED || (MpgPD && MPGLLED))
    || SV_PROGRAM_RUNNING THEN RST MPG_LED_OUT, RST MPGLLED

;x1, x10, x100 functions
;--X1
IF x1JogKey THEN (x1JogPD)
IF x1JogPD || OnAtPowerUp_M || X1_M || (MPG_Inc_X_1 && MPGLLED)
    THEN SET x1JogLED, RST x10JogLED, RST x100JogLED

;--X10
IF x10JogKey THEN (x10JogPD)
IF x10JogPD || X10_M || (MPG_Inc_X_10 && MPGLLED)
    THEN RST x1JogLED, SET x10JogLED, RST x100JogLED

;--X100
IF x100JogKey THEN (x100JogPD)
IF x100JogPD || X100_M || (MPG_Inc_X_100 && MPGLLED)
    THEN RST x1JogLED, RST x10JogLED, SET x100JogLED

IF !KbIncreaseJogInc_M && !KbDecreaseJogInc_M THEN RST X1_M, RST X10_M,
    RST X100_M

;--MPG 1 Enable
IF MPG_AXIS_1 || MPG_AXIS_2 || MPG_AXIS_3 || MPG_AXIS_4 ||
    MPG_AXIS_5 || MPG_AXIS_6 || MPG_AXIS_7 || MPG_AXIS_8
    THEN (SV_MPG_1_ENABLED)

```

```

;                Select axis to move
IF MPG_AXIS_1 THEN SV_MPG_1_AXIS_SELECT = 1
IF MPG_AXIS_2 THEN SV_MPG_1_AXIS_SELECT = 2
IF MPG_AXIS_3 THEN SV_MPG_1_AXIS_SELECT = 3
IF MPG_AXIS_4 THEN SV_MPG_1_AXIS_SELECT = 4
IF MPG_AXIS_5 THEN SV_MPG_1_AXIS_SELECT = 5

;                Select MPG 1 Multiplier
IF (MPG_Inc_X_100) THEN SV_MPG_1_MULTIPLIER = 100
IF (MPG_Inc_X_10) THEN SV_MPG_1_MULTIPLIER = 10
IF (MPG_Inc_X_1) THEN SV_MPG_1_MULTIPLIER = 1

;                Disable "Windup" mode IF x100 selected
IF (!MPG_Inc_X_100) THEN (SV_MPG_1_WINDUP_MODE)

```

Coolant Control

Coolant refers to the Flood and Mist control. Automatic coolant does not allow turning on the outputs unless the M7/M8 macros are used. Pushing the key to turn off Automatic coolant will then allow manual only control where a button must be pushed to turn on/off the coolant.

```

;--Coolant Functions

;--Toggle auto coolant mode
IF CoolAutoManKey || KbTogCoolAutoMan_M THEN (CoolantAutoManualPD)

IF (!CoolAutoManLED && CoolantAutoManualPD) || OnAtPowerUp_M
    THEN SET CoolAutoManLED

IF (CoolAutoManLED && CoolantAutoManualPD)
    THEN RST CoolAutoManLED

;--Report coolant mode to CNC11
IF CoolAutoManLED THEN (SelectCoolAutoMan)

;--Display coolant mode message
;changing to auto coolant mode ;9050 Auto Coolant Selected 2 + 50*256
IF (!CoolAutoManLED && CoolantAutoManualPD)
    THEN AsyncMsg_W = 12802, MSG AsyncMsg_W

```

```

;changing to manual coolant mode ;9051 Manual Coolant Selected 2 + 51*256
IF (CoolAutoManLED && CoolantAutoManualPD)
  THEN AsyncMsg_W = 13058, MSG AsyncMsg_W

;--Flood coolant on/off
IF ((CoolFloodKey || KbFloodOnOff_M) && !CoolAutoManLED) ||
  (M8 && CoolAutoManLED) || (DoCycleStart && M8 && CoolAutoManLED)
  THEN (CoolantFloodPD)

IF CoolantFloodPD && !CoolFloodLED THEN SET CoolFloodLED, SET Flood

IF SV_STOP || (CoolantFloodPD && CoolFloodLED) || (!M8 && CoolAutoManLED) ||
  (M8 && !CoolAutoManLED) || DoToolCheck THEN RST Flood, RST CoolFloodLED

IF CoolFloodLED THEN (SelectCoolantFlood)

;--Mist coolant on/off
IF ((CoolMistKey || KbMistOnOff_M)&& !CoolAutoManLED) || (M7 && CoolAutoManLED)
  || (DoCycleStart && M7 && CoolAutoManLED) THEN (CoolantMistPD)

IF (CoolantMistPD && !CoolMistLED) THEN SET Mist, SET CoolMistLED

IF SV_STOP || (CoolantMistPD && CoolMistLED) || (!M7 && CoolAutoManLED) ||
  (M7 && !CoolAutoManLED) || DoToolCheck THEN RST Mist, RST CoolMistLED

IF CoolMistLED THEN (SelectCoolantMist)

```

Probe Protection

There is some minimal protection built into the default PLC program to try and protect against crashing a probe. If the Mechanical probe trips while jogging, a probe fault is triggered. Jogging is not allowed in the direction that was being commanded when the probe tripped until the probe is cleared.

```

;-----
;           Probe protection while jogging
;-----
IF MechanicalProbe && !JogProbeFault_M && (DoAx1PlusJog || DoAx1MinusJog ||
  DoAx2PlusJog || DoAx2MinusJog || DoAx3PlusJog || DoAx3MinusJog ||
  DoAx4PlusJog || DoAx4MinusJog || DoAx5PlusJog || DoAx5MinusJog)
  THEN (JogProbeFaultPD)

```

```

IF MechanicalProbe && !JogProbeFault_M && FastSlowLED THEN SET LastProbeMode_M
IF MechanicalProbe && !JogProbeFault_M && !FastSlowLED THEN RST LastProbeMode_M

IF JogProbeFaultPD && !JogProbeFault_M THEN SET JogProbeFault_M, SET DoCycleCancel

IF JogProbeFault_M THEN ErrorCode_W = (PROBE_JOG_FAULT_MSG + 1),
    SET FastSlowLED

IF !MechanicalProbe && JogProbeFault_M && !LastProbeMode_M THEN RST FastSlowLED

IF !MechanicalProbe THEN RST JogProbeFault_M,
    RST Ax1PlusJogDisabled_M,
    RST Ax1MinusJogDisabled_M,
    RST Ax2PlusJogDisabled_M,
    RST Ax2MinusJogDisabled_M

```

PLC Optional Sections

This chapter covers optional sections of the PLC will go into detail about setting up the Debounce registers and inverting Inputs. There are several new features in CNC PLC programs, namely Input debounce and Input inversion. There is a standard value of 1.5 ms of debounce applied to all applicable PLC inputs and 18 ms debounce for the jog panel. No inputs are inverted by default. The PLC program can have debounce applied in excess of the standard value for a particularly noisy input.

Debounce or Invert Inputs

When a physical Input goes from on to off or vice-versa the PLC may see multiple transitions as the input turns on and off very quickly. Debounce is used to make sure that the Input is really on or really off. Once the Input is seen as on or off for the specified time, it is considered to be in that state and reported to the PLC program.

The PLCbus Inputs are Debounced at 1.5 ms by default, which is typically a decent value. Sometimes a switch is extra noisy and require more time to settle into a state. When that happens the debounce System Variables need to be adjusted. Debounce can be set for the first 240 PLCbus physical Inputs, 112 Jog Panel Inputs and 32 MPU11 Local I/O. They are all done with different System Variables, but the procedure is the same. The new Debounce values must be set every time the MPU11 boots, so a good place to do this is in the InitialStage.

The Debounce procedure scans at a certain rate, which determines the speed that an Input can be Debounced. The following chart lays out the scan rate of each of the Input types with Debounce. Keep in mind that the number of consecutive states of an Input required is determined by dividing the desired Debounce time by the scan rate. For example to get 1.5 ms of Debounce on a standard Input requires $0.0015/0.0000625 = 24$ scans. For reference 1 ms (millisecond) = 0.001 s and 1 μ s (microsecond) = 0.000001 s.

Note that there are two different updates rates for the PLC Inputs depending on where the Input is located. The Inputs directly on-board the DC3IOB and GPIO4D, for example are updated at 16kHz, whereas the Inputs on the PLCADD1616 are updated at 4kHz. However, the scan time for debounce (62.5 μ s) is the same for all PLC and PLC expansion devices. This means that even though the expansion devices update 4000 times per second they are checked at 16000 times per second. Therefore $16000/4000 = 4$ times the number of scans must be done on an expansion board. Instead of doing 24 scans for 1.5 ms of debounce 96 scans must be done.

The minimum Debounce Time is 0 which equates to the base Scan Rate and the maximum Debounce Time of 32768 equates to 2.048 s for PLC and MPU11 Local Inputs and 24.576 s.

Always use Debounce Time 2 or greater when making a custom time because Debounce Time 1 is used by most Inputs by default and Debounce Time 0 is always 0 and cannot be modified.

System Variable Name	Scan Rate (μ s)	Applies to
SV_PLC_DEBOUNCE_1 to _64	62.5	PLC Inputs
SV_JOG_LINK_DEBOUNCE_1 to _32	750	Jog Panel Inputs
SV_LOCAL_DEBOUNCE_1 to _13	62.5	MPU11 On-board Inputs

The default values for all of the Debounce Inputs is listed below.

	Default Debounce Setting Value	Default Debounce Time Value	Default Debounce Time (ms)
PLC Inputs	1	24	1.5
Jog Panel Inputs	1	24	18
MPU11 Local I/O	1*	24	1.5

*Input 770 for the DSP Probe Defaults to 0. Do not change this value.

Each group of System Variables is broken up into two sections. The majority of the System Variables are used for Debounce Setting to apply to the Inputs and the last few are for setting up the Debounce Time. This means

that there is not a separately named SV for the time and selecting what Input is used, just a different number. The following table illustrates which System Variables are for Inputs and which are for setting up the time.

System Variable Range	Function
SV_PLC_DEBOUNCE_1 to _60	Select Debounce time to use for each Input
SV_PLC_DEBOUNCE_61 to _64	Choose number of scans to Debounce
SV_JOG_LINK_DEBOUNCE_1 to _28	Select Debounce time to use for each Input
SV_JOG_LINK_DEBOUNCE_29 to _32	Choose number of scans to Debounce
SV_LOCAL_DEBOUNCE_1 to _9	Select Debounce time to use for each Input
SV_LOCAL_DEBOUNCE_10 to _13	Choose number of scans to Debounce

For each of the three types of Debounced Input there are seven different times that can be used in total. This means that any Input can have one of seven different times applied to it for Debounce.

Each of the System Variables to setup Inputs are broken up into four 8 bit sections devoted to one Input each. The layout for SV_PLC_DEBOUNCE_1 is illustrated in the following table, but all of the System Variables for setting up Inputs are divided the same way.

SV_PLC_DEBOUNCE_1 Bits	Function
0-7	Input 1 Setting
8-15	Input 2 Setting
16-24	Input 3 Setting
25-31	Input 4 Setting

Each of the 8-bit Input Setting Bytes is laid out according to the following table.

Input Setting Bit Number	Function
0	Debounce Time Select 1-7 in binary
1	1 = Time 1
2	...
3	111 = Time 7
4	reserved
5	reserved
6	reserved
7	Invert Input
8	Force Input On

In order to figure out which Debounce System Variable to use, divide the Input number by four and add one. PLC Input 211 is on $211 / 4 = 52.75$ plus one gives 53.75 for SV_PLC_DEBOUNCE_53. The fractional component tells which of the four input bytes in the System Variable should be set to get the correct Input. In the example above 0.75 is $\frac{3}{4}$ which points to Byte 3 out of 4 or bits 16-24.

Setting up the Debounce Time System Variables is more strait forward because there are only four System Variables that can be setup for each Input type. Each of the System Variables is split into two 16-bit values with the lowest one in each category being unused. That is why only Debounce times from 1 to 7 are allowed to be customized. Debounce Time 0 is always set to 0 and cannot be modified. The following table illustrates the seven available Debounce Times by System Variable.

Debounce Time Sys. Vars.	Debounce Time
SV_PLC_DEBOUNCE_61	0 – 15 always set to 0 16 – 31 Debounce Time 1
SV_PLC_DEBOUNCE_62	0 – 15 Debounce Time 2 16 – 31 Debounce Time 3
SV_PLC_DEBOUNCE_63	0 – 15 Debounce Time 4 16 – 31 Debounce Time 5
SV_PLC_DEBOUNCE_64	0 – 15 Debounce Time 6 16 – 31 Debounce Time 7

There are several ways to setup the Debounce values. One option is to calculate out the bit values in decimal for all 32-bits and add them all up. Another option is to create each 8-bits for the Debounce Time or Debounce Setting in a temp Word and Left Shift the value up to the correct place in the Word then add it to the Debounce System Variable. As an example the following program reads sets up INP6 with a debounce time of 13 ms and inverts the input. The values are recorded here in binary to show the bits changing. Note that these bits are written left to right and Msb to Lsb whereas WTB writes bits Lsb to Msb left to right.

Example Input Debounce Setup Program

```
;Original INP6 Debounce Setting Word Value
;0000 0001 0000 0001 0000 0001 0000 0001
;Desired Debounce Setting bits for INP6
;0000 0000 0000 0000 0000 0101 0000 0000
;modified Debounce Setting Word with new bits added in
;0000 0001 0000 0001 0000 0101 0000 0001
;Original Debounce Time Word Value
;0000 0000 0001 1000 0000 0000 0001 1000
;Debounce Time after removing the 16 bits that will be replaced
;0000 0000 0000 0000 0000 0000 0001 1000
;Temp Debounce Time Word with new value shifted into place
;0000 0000 1101 0000 0000 0000 0000 0000
;Final Debounce Time Word with new time value
;0000 0000 1101 0000 0000 0000 0001 1000

;////////////////////////////////////////
;Program: debounce.src
;Purpose: Example Debounce Setup for INP6
;      Set Time 5 for 13 ms., Invert Input
;Date: 20-APR-2010
;////////////////////////////////////////

;--Variable Defines
Debounce_2_Bits_M      IS MEM1  ;display the bits from the Word
```

Debounce_63_Bits_M IS MEM35 ;display the new bits for the Word

Bit0_M IS MEM73 ; Deb Time Select Bit 0

Bit1_M IS MEM74 ; Deb Time Select Bit 1

Bit2_M IS MEM75 ; Deb Time Select Bit 2

Bit3_M IS MEM76 ; reserved

Bit4_M IS MEM77 ; reserved

Bit5_M IS MEM78 ; reserved

Bit6_M IS MEM79 ; Invert Input

Bit7_M IS MEM80 ; Force Input On

Original_Deb_W IS W1 ;Value read from Deb SV

Temp_Deb_W IS W2 ;Calculated Word value

Final_Deb_W IS W3 ;final combined Word value

Orig_Deb_Time_W IS W4 ;Value read from Deb SV

Shift_Deb_Time_W IS W5 ;value with high 16 bits removed

Temp_Deb_Time_W IS W6 ;Calculated Word value

Final_Deb_Time_W IS W7 ;final combined Word value

InitialStage IS STG1

MainStage IS STG2

;reading INP6 means reading from SV_PLC_DEBOUNCE_?

; ? = $6/4 + 1 = 2.5$

;choosing which Byte to set requires looking at the remainder

;the remainder, $0.5 = 2/4$ or byte two of four must be set

;use Time 5 to setup the Debounce Time

;5 decimal = 101 binary

;looking at the table gives SV_PLC_DEBOUNCE_63 high 16 bits should be used

;=====

InitialStage

;=====

IF 1==1 THEN Original_Deb_W = SV_PLC_DEBOUNCE_2 ;read the Settings SV

IF 1==1 THEN Orig_Deb_Time_W = SV_PLC_DEBOUNCE_63 ;read the Time SV

IF 1==1 THEN Final_Deb_W = Original_Deb_W

```

;zero out the bits that are going to be replaced
IF 1==1 THEN BITRST Final_Deb_W 8
IF 1==1 THEN BITRST Final_Deb_W 9
IF 1==1 THEN BITRST Final_Deb_W 10
IF 1==1 THEN BITRST Final_Deb_W 11
IF 1==1 THEN BITRST Final_Deb_W 12
IF 1==1 THEN BITRST Final_Deb_W 13
IF 1==1 THEN BITRST Final_Deb_W 14
IF 1==1 THEN BITRST Final_Deb_W 15

;Setup the 8 bits for the Setting Byte
IF 1==1 THEN SET Bit0_M ;
    ,RST Bit1_M ;binary 5 for Time 5
    ,SET Bit2_M ;
    ,RST Bit3_M ;unused
    ,RST Bit4_M ;unused
    ,RST Bit5_M ;unused
    ,RST Bit6_M ;do not Invert the Input
    ,RST Bit7_M ;do not Force On

;move the 8 bits to a Word
IF 1==1 THEN BTW Temp_Deb_W Bit0_M 8
;shift the values to the 2nd byte as calculated above
IF 1==1 THEN LSHIFT Temp_Deb_W 8

;copy the new 8 bits to the final word
IF 1==1 THEN Final_Deb_W = Final_Deb_W + Temp_Deb_W

;calculate the Debounce scans required for 13 ms
;onboard PLC INP so  $0.0130/0.0000625 = 208$  scans
IF 1==1 THEN Temp_Deb_Time_W = 208

;shift the bits up to the high 16 bits for the word
IF 1==1 THEN LSHIFT Temp_Deb_Time_W 16

;shift off the high bits from the original Debounce
;registers
IF 1==1 THEN Shift_Deb_Time_W = Orig_Deb_Time_W

```

```

IF 1==1 THEN LSHIFT Shift_Deb_Time_W 16
IF 1==1 THEN RSHIFT Shift_Deb_Time_W 16

;combine the low 16 bits and high 16 bits
IF 1==1 THEN Final_Deb_Time_W = Temp_Deb_Time_W + Shift_Deb_Time_W

;save the values back to the SV
IF 1==1 THEN SV_PLC_DEBOUNCE_2 = Final_Deb_W
IF 1==1 THEN SV_PLC_DEBOUNCE_63 = Final_Deb_Time_W

IF 1==1 THEN RST InitialStage, SET MainStage

;=====
MainStage
;=====
;write out bits of final Word settings for verification
IF 1==1 THEN WTB Final_Deb_W Debounce_2_Bits_M 32
           , WTB Final_Deb_Time_W Debounce_63_Bits_M 32

```

Setting Inputs High or Low for Testing

The Debounce registers that store the settings for the Inputs can also be used to invert, at the hardware level, the state of the input. This means that a Normally Closed Input can be turned into a Normally Open Input.

Note that on the DC3IOB Inputs 1-6 and 11 are acted on at the hardware level before the Force and Invert bits are looked at. This means that Normally Open Limits and E-Stop still cannot be used.

There is also the ability to Force an Input to be SET. This should really never be used outside of debugging on the bench due to the possibility of masking an input from indicating an error has occurred. Inputs can be Forced off or RST by setting both the Invert and Force on bits.

Looking at the Table above that explains the layout of the Setting Bytes shows that Bit 7 and 6 are used for Forcing the Input and Inverting the Input respectively. Time 1 should always be set if no custom Debounce time is going to be used.

In the following example the same input as above is going to just be Forced On and Inverted, effectively forcing the input off all the time. This can be verified in PLC Diagnostics by toggling INP6 and seeing that it does not change.

```

;Original INP6 Debounce Setting Word Value
;0000 0001 0000 0001 0000 0001 0000 0001
;Desired Debounce Setting bits for INP6
;0000 0000 0000 0000 0000 0101 0000 0000
;modified Debounce Time Word with new bits added in
;0000 0001 0000 0001 1100 0001 0000 0001

```

```

;////////////////////////////////////////
;Program: invert.src
;Purpose: Example invert/force Setup for INP6
;////////////////////////////////////////
;--Variable Defines
Orig_Deb_Bits_0 IS OUT1 ;
Debounce_Bits_M IS MEM1 ;display the bits from the Word
Bit0_M          IS MEM73 ; Deb Time Select Bit 0
Bit1_M          IS MEM74 ; Deb Time Select Bit 1
Bit2_M          IS MEM75 ; Deb Time Select Bit 2
Bit3_M          IS MEM76 ; reserved
Bit4_M          IS MEM77 ; reserved
Bit5_M          IS MEM78 ; reserved
Bit6_M          IS MEM79 ; Invert Input
Bit7_M          IS MEM80 ; Force Input On

Original_Deb_W IS W1      ;Value read from Deb SV
Temp_Deb_W     IS W2      ;Calculated Word value
Final_Deb_W     IS W3      ;final combined Word value

InitialStage    IS STG1
MainStage       IS STG2

;begin program

;=====
InitialStage
;=====
IF 1==1 THEN Original_Deb_W = SV_PLC_DEBOUNCE_2 ;read the Settings SV
IF 1==1 THEN Final_Deb_W = Original_Deb_W

;zero out the bits that are going to be replaced
IF 1==1 THEN BITRST Final_Deb_W 8
IF 1==1 THEN BITRST Final_Deb_W 9
IF 1==1 THEN BITRST Final_Deb_W 10
IF 1==1 THEN BITRST Final_Deb_W 11
IF 1==1 THEN BITRST Final_Deb_W 12
IF 1==1 THEN BITRST Final_Deb_W 13

```

```

IF 1==1 THEN BITRST Final_Deb_W 14
IF 1==1 THEN BITRST Final_Deb_W 15

;Setup the 8 bits for the Setting Byte
IF 1==1 THEN SET Bit0_M ;
    ,RST Bit1_M ;binary 1 for Time 1
    ,RST Bit2_M ;
    ,RST Bit3_M ;unused
    ,RST Bit4_M ;unused
    ,RST Bit5_M ;unused
    ,SET Bit6_M ;Invert the Input
    ,SET Bit7_M ;Force On the Input

;move the 8 bits to a Word
IF 1==1 THEN BTW Temp_Deb_W Bit0_M 8
;shift the values to the 2nd byte as calculated above
IF 1==1 THEN LSHIFT Temp_Deb_W 8

;copy the new 8 bits to the final word
IF 1==1 THEN Final_Deb_W = Final_Deb_W + Temp_Deb_W

;save the values back to the SV
IF 1==1 THEN SV_PLC_DEBOUNCE_2 = Final_Deb_W

;write out bits of final Word settings for verification
IF 1==1 THEN WTB Final_Deb_W Debounce_Bits_M 32
IF 1==1 THEN WTB Original_Deb_W Orig_Deb_Bits_O 32

IF 1==1 THEN JMP MainStage
;no MainStage shown because there is nothing to do there for this example

```

Compiler Errors

There are many ways that a program can be changed so that it does not function as intended. Always start at the top of the list of errors because fixing earlier problems may fix repeat errors later on. For example if you fail to define a variable that is used throughout the program, every reference to that variable will generate an error.

If the compiler hangs or doesn't generate any output after ten seconds it is stuck on something and will not compile the program most likely. Press Ctrl.+c to cancel compilation and undo the last changes if the problem just started occurring. If you think the changes are valid double check the usage and then send the file before and after to Tech. Support.

There is a limit of about 1000 characters per line that the compiler can digest so do not go above that limit. Typically you should stick to 80 characters so that the document can be printed and not cause the printout to look different than on your monitor.

The easiest to fix are compiler related syntax or expression errors. The first to sections details what potential errors mean.

Warnings

Already Defined

This message does not cause compilation to fail, but you should address this problem because using two different variables to do two different things will cause very hard to detect logical errors.

Direct PLC Reference

This Warning occurs when you test or use a specific data type directly by name. An example is using MEM1 instead of defining Test_M IS MEM1 and using Test_M in the program.

General Errors

These errors specify problems with the program that are more serious than an undefined variable. They are not related to the code in the Program, but the way that the compiler was called.

Malformed Command Line

This error means the the attempt to compile the program failed because no source file was specified.

Unrecognized Command Line Option

An invalid switch was used on the command line. Presently there are two options for switches. Neither of them should be used except for testing and debugging the compiler itself. The output is not usable as a PLC Program. The options are -i for outputting a CSV version of the program to be used as an include file and -d is for verbose debugging of what the compiler does with each token in the program. This may be used when there is a suspected problem with the compiler.

Error Opening File

The specified source file was not found in the directory that mpucomp was called from. There are two ways to compile the program. The easiest is to open a command prompt and change directories to the folder where the source file is located. Then you can issue the command `mpucomp(.exe) source.src mpu.plc`. On Windows you must have mpucomp.exe in the same folder as the source file unless you add c:\cncm\bin to your PATH. In Linux this has been done for you so mpucomp does not need to be in the same folder as the source file. The other way of compiling a program is to call mpucomp and give the absolute path to the source file and/or destination file. For example, the source file is in /cncroot/c and the command line indicates the current

folder is /cncroot. To compile the source file and get the PLC program to the correct folder *mpucomp* /cncroot/c/source.src /cncroot/c/cncm/mpu.plc must be executed.

Syntax Errors

These are errors related to specific lines in the PLC program. Usually the offending line and column number is listed along with the specific error. Sometimes the row is one farther down the program than it should be so look around the area specified.

Compilation Failed

This message occurs when there are problems in the source file. It is displayed as the last message after other error messages. This message will not appear if there are too many errors.

Too Many Errors

This message is displayed when more than 25 errors occur at compile time. This can easily happen if you forget to define a Label and use it often in the program.

Undefined Label

This message occurs when a valid variable name is used without it being defined in the definition section of the PLC program.

[THEN, Word Type, Stage, Output, Parenthesis] expected

This message is generated when something is expected, but is not found. A Parenthesis is expected on the right if you put on one the left of a variable or expression.

```
IF INP1) THEN SET OUT1
```

End Of File Expected

This message can be generated many different ways, but the meaning is that a line was not written correctly. It can be caused because the IF at the start of a line was not used after the first IF statement in the program.

[Data Type] Out of Bounds

This message can occur on any Data Type when using the [Indexing](#) ability in the compiler. The error is generated when you make a constant reference to a Data Type number that cannot be accessed. For example if you try IF 1==1 THEN SET OUT[500000000] you will get this error. It is possible to fool the compiler and index out of range. See [Internal PLC Fault Checking](#) for how to detect this occurring and why it is so catastrophic.

Invalid Action

This error occurs when doing something that is not allowed. This includes trying to store a value into a Bit data type or a Constant define and failing to put a start location for a Range.

```
CONST1 IS 1                ;constant value
IF 1==1 THEN CONST1 = 10    ;cannot re-assign constant values
IF 1==1 THEN MEM1 = 10      ;cannot store Word values into Bits
IF 1==1 THEN .. OUT20       ;must include starting point for Range
IF 1==1 THEN SET W1         ;SET cannot be applied to Words
```

Rung Expected IF

The Keyword IF was not found after a valid STG reference.

STG2

```
1==1 THEN (OUT1)
```

Rung Expected THEN

The Keyword THEN is missing from the line specified.

```
IF 1==1 SET OUT1
```

JMP Expected STG or FSTG

If you call the JMP command and do not specify what Stage to Jump to, this message is displayed.

System Variable is Read-Only

Attempting to modify a Read-only System Variable will result in this message displayed. The Read-only variables are ones that CNC11 has write control of [here](#).

MSG Expected Word Reference

This error occurs when you try to use the MSG command with no Word variable or with an invalid data type. If no data type is specified, then the error will actually show up on the next line of the program and not show the MSG reference at all in most cases. This occurs because the compiler does not assume that a line feed is the end of a command. For example the first IF statement spanning two lines in the following code will compile whereas the second on only one line will not.

```
IF 1==1 THEN MSG
```

```
W1
```

```
IF 1==1 THEN MSG INP1
```

SMSG Expected String Reference

The SMGS functionality expects a string value and anything else causes this error. A string is defined as text between double quotes.

```
IF 1==1 THEN SMSG W1 ;not a string
```

BCD/BIN Expected Word Reference

If you fail to put a Word type variable right after the BCD and BIN command this error occurs.

```
IF 1==1 THEN BIN MEM1 32
```

BCD/BIN Cannot Use Bracketed Reference

Indexes are not allowed on Words when doing the BCD or BIN commands.

```
IF 1==1 THEN BCD W[10]
```

WTB Expected Word Reference

If you fail to put a Word type variable right after the WTB command this error occurs.

```
IF 1==1 THEN WTB MEM1 32
```

WTB Expected OUT/MEM Reference

If the Memory or Output location is not specified, this error occurs.

```
IF 1==1 THEN WTB W1 33
```

WTB Number of Bits Must be 1-32

When using the WTB command you can specify a number of bits to write. That value must be from 1 to 32. The incorrect value is shown after the error.

```
IF 1==1 THEN WTB W1 MEM1 33
```

BTW Expected Word Reference

If you fail to put a Word type variable right after the BTW command this error occurs.

```
IF 1==1 THEN BTW MEM1 32
```

BTW Expected INP/OUT/MEM Reference

If the Input, Memory or Output location is not specified, this error occurs.

```
IF 1==1 THEN WTB W1 33
```

BTW Number of Bits Must be 1-32

When using the BTW command you can specify a number of bits to write. That value must be from 1 to 32. The incorrect value is shown after the error.

```
IF 1==1 THEN BTW W1 MEM1 33
```

BITSET/RST Bit Must be 0-31

When using the BITSET or BISTRST commands you must specify the bit number to SET or RST. That value must be from 0 to 31. The incorrect value is shown after the error.

```
IF 1==1 THEN BITSET W1 43
```

BITSET/RST Expected an Integer Value

Bits can only be changed by positive Constant Integer numbers so using a floating-point number, negative or Word value to try and change one will fail.

```
IF 1==1 THEN BISTRST W1 1.5
```

```
IF 1==1 THEN BITSET W1 W2
```

```
IF 1==1 THEN BITSET W1 -10
```

BITSET/RST Word Indexing Not Allowed

Using the Index ability is not allowed when doing a BITSET/RST command.

```
IF 1==1 THEN BITSET W[W2] 2
```

BITSET/RST Expected Word Reference

If you fail to put a Word type variable right after the BITSET or BISTRST command this error occurs.

```
IF 1==1 THEN BITSET MEM1 4
```

BITTST MEM Indexing Not Allowed

Using the Index ability is not allowed when doing a BITTST command.

```
IF 1==1 THEN BITTST W2 2 MEM[5+1]
```

BITTST Expected MEM

The last of three arguments to a BITTST must point to a Memory Bit and it was not found or the wrong type was specified.

```
if 1==1 then BITTST w1 2 OUT1
```

BITTST Expected Word Reference

If you fail to put a Word type variable right after the BITTST command this error occurs.

```
IF 1==1 THEN BITTST INP1 5 MEM1
```

BITTST Bit Must be 0-31

When using BITTST you must specify the bit number to check in the Word. That value must be from 0 to 31. The incorrect value is shown after the error.

```
IF 1==1 THEN BITTST W1 43 MEM1
```

BITTST Expected Integer Value

Bits can only be changed by positive Constant Integer numbers so using a floating-point number, negative or Word value to try and change one will fail.

```
IF 1==1 THEN BITTST W1 1.5 MEM1
```

```
IF 1==1 THEN BITTST W1 W2 MEM1
```

```
IF 1==1 THEN BITTST W1 -10 MEM1
```

BITTST Word Indexing Not Allowed

Using the Index ability is not allowed when doing a BITTST command.

```
IF 1==1 THEN BITTST W[2+5] 2
```

LSHIFT/RSHIFT Bit Must be 0-31

When using the LSHIFT/RSHIFT commands you must specify the number of bits to shift. That value must be from 0 to 31. The incorrect value is shown after the error.

```
IF 1==1 THEN LSHIFT W1 99
```

LSHIFT/RSHIFT Expected Integer Value

Bits can only be changed by positive Constant Integer numbers so using a floating-point number, negative or Word value to try and change one will fail.

```
IF 1==1 THEN LSHIFT W1 1.5
```

```
IF 1==1 THEN LSHIFT W1 W2
```

```
IF 1==1 THEN LSHIFT W1 -10
```

LSHIFT/RSHIFT Word Indexing Not Allowed

Using the Index ability is not allowed when doing a LSHIFT/RSHIFT command.

```
IF 1==1 THEN LSHIFT W[10+2] 2
```

LSHIFT/RSHIFT Expected Word Reference

If you fail to put a Word type variable right after the LSHIFT/RSHIFT command this error occurs.

```
IF 1==1 THEN LSHIFT MEM1 4
```

Constant Integer Expression Label Not Found

If you reference a constant when defining other constants, but the referenced constant does not exist yet you will get this message. The following example will generate this error.

```
CONST2 IS 2*CONST3
```

Constant Integer Expression Expected Right Parenthesis

This error is generated when a left parenthesis is used to declare a constant, but the right parenthesis is left off.

```
CONST1 IS (10+2
```

Constant Integer Factor Expected

Constant math can only be done on Integer values. Putting a Floating-point value in a constant define that does math will give this error.

```
CONST1 IS (10+2.2)
```

Constant Integer Expression Label Does Not Reference an Integer

If you create a Constant that stores a floating-point value and then use it in the declaration of an Integer Constant you will get this message.

```
CONST1 IS 11.2
```

```
CONST2 IS (CONST1/2)
```

Constant Integer Expression Label not Found

This error only occurs when a previous Constant expression definition is invalid and it is referenced by another Constant expression definition.

```
CONST1 IS (11.2) ;invalid because of parentheses, only constant numbers allowed
```

```
CONST2 IS (CONST1/2);invalid CONST1 causes another error.
```

Relational Operator Expected

A comparison of data types that need a Relational Operator used on them to evaluate to true or false is needed rather than the used operator.

```
IF W1 = W2 THEN SET OUT1 ;assignment not allowed before THEN
```

```
IF !W2 THEN SET OUT2 ;Logical Operator not allowed on Words
```

System Variable Bits Cannot be Used With '..'

The Range selector cannot be used on any System Variables.

```
IF 1==1 THEN SET SV_ENABLE_AXIS_1 .. SV_ENABLE_AXIS_8
```

Range Extension Expected OUT/MEM/STG/FSTG/T

When specifying a Range, the end of the Range was not specified. This error will be shown on the next line after the error actually occurs. One of the listed types must be used to finish the Range. It must be the same type that is on the left of the Range select.

```
IF 1==1 THEN SET OUT1 ..
```

Range Error. End is Before Start

The Range selectors were specified out of order. They must be used from lower to high bits.

```
IF 1==1 THEN SET OUT10 .. OUT1
```

SET/RST Expected OUT/MEM/STG/FSTG/T/Modifiable SV

This message is generated when you try to SET or RST something that the action cannot be applied to or nothing is supplied. Inputs, One-Shots, Words, and Word type System Variables are all examples of things that cannot be SET or RST.

```
IF 1==1 THEN SET
```

```
IF 1==1 THEN SET INP1
```

```
IF 1==1 THEN SET PD1
```

Coil Expected PD/OUT/MEM/Modifiable SV

This error occurs when trying to use Coils on Data Types that cannot be SET/RST. This includes Word types and Timers. There is no point turning on a Timer to check that only lasts one pass of the PLC program. Words can have bits in them SET and RST using BITSET and BITRST.

```
IF INP1 THEN (T2)
```

Expected Right Bracket

This message occurs when a left bracket is used and a right one is not found.

```
if 1==1 then set out[10+3
```

```
if 1==1 then set out[10+3)]
```

Expected Left Bracket

This error occurs because a variable that can use Indexing had a space after the variable name and no bracket was found. The compiler then expects to see a Left Bracket. You may not be doing an Index reference at all, but merely had a space where there should not be one.

```
IF 1==1 THE SET OUT 10]
```

```
IF 1==1 THEN BTW W 3] 10
```

Bad Definition

The attempt at defining a variable is not written correctly. Brackets and negative numbers are not allowed. Constant defines are used for messaging mostly.

Const is [10] ; brackets cannot be used in the definition section of a PLC program

Number Expected Right Parenthesis

A right Parenthesis was expected in a math calculation in the Program.

```
IF 1==1 THEN W1 = (10 + 50
```

Numerical Factor Expected Right Parenthesis

Any command such as ABS, SIN, etc. is listed and then the calculation to be done is in parentheses afterwards. If the parenthesis on the right is missing, this error occurs.

```
if 1==1 then W1 = abs (10*5
```

Numerical Factor Expected Left Parenthesis

Any command such as ABS, SIN, etc. is listed and then the calculation to be done is in parentheses afterwards. If the parenthesis on the left is missing, this error occurs.

```
if 1==1 then FW1 = sin (3.14159265/180
```

ATAN2/POW Expected Right Parenthesis

In either an ATAN2 or POW operation, the right parenthesis was left off.

```
IF 1==1 THEN FW1 = ATAN2 (3.141592/180, 3.141592
```

ATAN2/POW Expected Comma

In either an ATAN2 or POW operation, the right parenthesis was left off.

```
IF 1==1 THEN FW1 = POW (1 1)
```

ATAN2/POW Expected Left Parenthesis

In either an ATAN2 or POW operation, the right parenthesis was left off.

```
IF 1==1 THEN FW1 = ATAN2 3.141592/180, 3.141592)
```

Expected Right Parenthesis

This error occurs when using a Coil on an Output or Memory Bit when there is a left parenthesis and not a right.

```
IF 1==1 THEN (OUT1
```

Boolean Factor Expected Right Parenthesis

In a Boolean check using parentheses, the right parenthesis was left off.

```
IF (INP1 THEN SET OUT1
```

Assignment Error

This error occurs when a value is getting stored to a variable and it cannot be completed. One example is to Index to a floating-point variable.

```
IF INP1 THEN W[10.4] = 1
```

Application Examples

These examples strive to give examples of using some of the various PLC functionality that isn't used in a standard PLC program. The examples are not meant to be useful in what is accomplished, but rather in how it is accomplished.

Toggle an Output Every Second

Pseudo-code

```
In InitialStage Second_On_T = 1000, Second_Off_T = 1000, SET Second_Off_T
IF Second_On_T THEN RST Second_On_T, SET Second_Off_T, RST OUT1
IF Second_Off_T THEN RST Second_Off_T, SET Second_On_T, SET OUT1
```

Aux Key Jogging

Pseudo-code

```
IF Aux2 THEN Y+_Jog
IF Aux8 THEN Y-_Jog
IF Aux4 THEN X-_Jog
IF Aux6 THEN X+_Jog
IF Aux3 THEN Z+_Jog
IF Aux9 THEN Z-_Jog
IF Aux1 THEN W+_Jog
IF Aux7 THEN W-_Jog
```

Aux Key Override of M-Code

Pseudo-code

```
IF M-Code && (CNC_PROG_RUNNING || AUX14) THEN SET M-Code_Stage
```

Wait One Second Before Jogging on Key Press

Pseudo-code

```
IF !(all jog keys) THEN RST Jog_T
IF any jog key THEN SET Jog_T
IF Jog_T THEN DO Ax#_Jog
```

Interpret Enter Key as Cycle Start in MDI*

*Note that Centroid does not recommend doing this. It is merely an example of what is possible as far as customizing the User experience. Often, after typing a command in MDI it is natural to hit Enter to try to run the line, but that merely causes a prompt to push Cycle Start to appear. With this change that will not happen. This functionality should only be allowed when in MDI to prevent unexpected motion.

Pseudo-code

```
IF MDI_MODE && AllowKBJog && Kb_Enter_Key THEN Cycle_Start
```

Count Machine On Time

Use a Timer to count one day and then increment a word. Update P171 every 30 min. to keep track of the value. Read the value of P170 in the Initial stage and increment values to it.

Custom M-Codes

Using M94/M95 Bits

Pick an unused M94/M95 bit and assign it the name of the custom M-Code. Create an mfuncXXX.mac file that has the same number as the assigned M-Code. Put whatever you want in the M-Code and if the PLC program needs to do something, put code in to check for MXXX being SET or RST.

Using One M94/M95 Bit and a Parameter

In the custom M-Code SET the bit with M94 and G10 a parameter in the 171-177 range to a different binary value for each M-Code. Use 1, 2, 4, 8, 16, 32, etc. so that you can use BITTST. Alternatively, you can just use decimal numbers and check Parameter == 1, ==2, ==3, etc.

Customizing Standard M-Codes

In the basic MPU11 PLC programs there are four “customized” M-Codes by default. They are for Spindle directions and Coolant. The reason that they are considered custom is that rather than using the built in CNC software functionality, files were created to extend the original functionality. Now there is a message printed on screen that halts execution if an auto spindle or coolant function is called and prompts the user to put the control in auto mode. This can be done for many of the M-Codes to expand the usability of or modify the given feature. The following example shows how the M3 and M4 M-Codes were changed.

Automatic Spindle On/Off – M3/M4

When a default M3 is commanded the Spindle should start spinning in the clockwise direction. If the control is in manual spindle mode though, the spindle will not turn on and program execution will continue like everything is fine. This is bad, so a warning should be given to the user and execution paused. The same holds true for Automatic Coolant which is why the custom M7 and M8 are needed.

```
;-----  
;M3 macro  
; Displays message to select auto spindle mode if it is not SET  
;-----  
;if searching or backplotting a program, skip the macro  
IF #4202 || #4201 THEN GOTO 200  
M95 /2 ; turn off CCW spindle  
M94 /1 ;turn on CW spindle  
  
;if AutoSpindle i.e. OUT1058 (a.k.a. JP02) is set, exit the Macro  
IF #61058 THEN GOTO 200  
G4 P.1  
#140 = 1.5  
N100  
  
; If not in AutoSpindle Mode display a message and wait  
IF !#61058 THEN M225 #140 "Please Select Auto Spindle To Continue!"  
G4 P.5  
IF !#61058 THEN GOTO 100  
N200
```

Troubleshooting and Changing PLC Programs

This chapter contains some suggestions for debugging problems in your PLC program.

Write Down and Think Through Changes to the Program

Before making any changes to a PLC Program you should always take a report. Write out as specifically as possible the exact functionality that is to be implemented. This potentially includes timeouts, fault conditions, interactions with other Inputs and Outputs, messages that will need to be shown to the User, etc.

PLC Diagnostic Screen

When at the main screen, push Alt.-i to cause the PLC Diagnostic screen to appear. It appears over running jobs so it should not be left on all the time, especially if there are user prompts in the program. The red and green dots represent Inputs, Outputs, Memory and Stage Bits. Red means the Bit is off or open and Green means the Bit is closed or on. This is true in all cases for the Bits. Keep in mind that setting the Invert or Force [Debounce](#) bits will change the way the Inputs are reported to the PLC program and thus the PLC Diagnostic screen.

The screen shows the state of Bits 80 at a time for the entire range available for that Bit type and 12 Words per page. All 1312 available Inputs and Outputs are shown as are all 1024 Memory Bits and 256 Stages. Presently the 44 Words that are sent up to CNC11 are shown as well. This means that all the Jog Panel buttons and LEDs as well as Spindle DAC outputs can be seen without mirroring them to the 1 to 80 range. This is true as of CNC11 3.0 rev84 beta release.

Future improvements include adding FW, DW, DFW, FSTG, and Timer display to the screen.

PLC Bit-State Dump

When the PLC Diagnostic screen is showing, pushing the Ctrl.- i key combination will cause the state of all the Bit variables to be stored to a file called plcstate.txt. This file is overwritten every time the operation is performed. This feature will be implemented in the future.

DUMP

Use the [DUMP](#) PLC command to print all of the values of the first 64 Word type variables to disk. This can be called at any time to check status words or ATC bin position in a certain stage. Make sure to call this infrequently because it does take quite a long time to write to disk compared to the time to execute the PLC program once.

Echo to a Memory Bit

Transient signals that are on and then quickly off may not show up reliably on the PLC Diagnostic screen long enough to see because the display only updates 30 times/second. In this case, to verify that the bit is changing at all you can SET a Memory Bit if the Bit in question ever gets SET or RST. It is a good idea to incorporate the ability to RST these diagnostic Bits into an unused Aux Key so that you do not have to power off the system to RST them.

Use Stages

Group new features into a Stage so that it can be turned on and off to check for problems. Use more stages for more complex features to narrow down where you need to troubleshoot.

Communication In/Out Faults

DriveBus

These faults occur when the communication on the DriveBus is disrupted. The DriveBus goes out on fibers 4 and 5 from the MPU11 and across the Drive Communication In/Out wire connections on the ALLIN1DC, Optic4 and DC3IOB. If any of these connections exist and are checked in the PLC program correctly, they will help diagnose connection problems.

PLCBus

The PLCBus also should be checked in the PLC program to make sure that communication is in a good state. There should be checks for the Fibers 1 and 3 and miniPLCBus connections if expansion boards are used.

Appendix A: Example PLC program

ALLIN1DC DC system example

This is the basic PLC program using an ALLIN1DC drive and PLC.

```
;-----  
; File:          allin1dc-basic.src  
; Programmer:    Scott Pratt, Keith Dennison, Marc Leonard  
; Date:          7 April 2010  
; Purpose:       PLC for MPU11 and allin1dc + keyboard jog  
; Requires:      Requires CNC11 R83+  
;  
; $Id: allin1dc-basic.src 510 2011-03-02 11:05:28Z marc $  
;  
; Changes: 101123 KD - Added Aux 10, 11, 12 assignments to the gray Aux keys.  
;  
; NOTE Changes to keyboard jogging bmp needed. See MEM400+  
;      "Invalid key" messages will need to be suppressed  
;  
; Mpu11 based systems have the ability to invert, force and/or select a custom  
; debounce time on PLC inputs 1-240 using SV_PLC_DEBOUNCE_1-SV_PLC_DEBOUNCE_64.  
; Jog Panel inputs are modified in the same manner using SV_JOG_LINK_DEBOUNCE_1  
; -SV_JOG_LINK_DEBOUNCE_64. See system variable section for more information.  
;  
; The Mpu11 board includes connections for several types of auxiliary I/O.  
; 4 digital "high speed" inputs (INP769-772) typically used for probe/TT1  
; related functions, 3 auxiliary digital inputs (INP784-786), 11 Digital inputs  
; used for MPG increment and axis selection and 2 auxiliary digital outputs  
; (Out770-771).  
;  
; ALLIN1DC Physical I/O: While each ALLIN1DC that is installed reserves (maps)  
; 16 inputs and 16 Outputs, only 16 inputs and 9 outputs are accessible through  
; hardware.  
;  
; Digital Inputs: The ALLIN1DC provides 16 inputs, 10 of which are available for  
; general purpose use. The first 6 inputs (1-6) are dedicated for limit switch  
; use and must be either wired to a NC limit switch or defeated. All 16 inputs  
; can be configured (in banks of 4) for 5, 12 or 24VDC operation in either a  
; sourcing or sinking configuration.  
;  
; Analog input: The ALLIN1DC provides a single 12 bit analog input which is  
; mapped to inputs 241-252. LSB = 241. This input can be configured for the  
; following input:  
;  
; 1. 0 - 5VDC  
; 2. 0 - 10VDC  
; 3. -5 - +5VDC  
; 4. -10 - +10VDC  
;  
; Please see the ALLIN1DC manual for configuration information  
;  
; Outputs: The ALLIN1DC has 9 relay contact closure outputs. Outputs 1-7 are  
; are SPST type relays while Outputs 8 & 9 are SPDT type relays.
```

```
; Analog outputs: The 12 bit 0-10VDC analog output on the ALLIN1DC
; is mapped to outputs 241-252. NOTE: The spindle speed command that comes down
; from the PC (SV_PC_DAC_SPINDLE_SPEED) is a 16 bit integer value from 0-65535
; that must be converted to a 12 bit value from 0-4095 by the PLC. The PLC
; handles gear ranges by looking at the state of inputs and reading parameters
; (or hard coded values) to determine the ratio needed for adjusting the spindle
; speed display system variable
```

```
;-----
;                               CONSTANT DEFINITIONS
;-----
```

```
PLC_EXECUTOR_FLT_MSG IS 257; (1+256)
```

```
AXIS1_INFLT      IS 1282;(2+256*5) Fiber to MPU11 has a problem
AXIS2_INFLT      IS 1538;(2+256*6)
AXIS3_INFLT      IS 1794;(2+256*7)
AXIS4_INFLT      IS 2050;(2+256*8)
AXIS5_INFLT      IS 2306;(2+256*9)
AXIS6_INFLT      IS 2562;(2+256*10)
AXIS7_INFLT      IS 2818;(2+256*11)
AXIS8_INFLT      IS 3074;(2+256*12)
```

```
AXIS1_OUTFLT     IS 3330;(2+256*13) Fiber to axis drive has a problem
AXIS2_OUTFLT     IS 3586;(2+256*14)
AXIS3_OUTFLT     IS 3842;(2+256*15)
AXIS4_OUTFLT     IS 4098;(2+256*16)
AXIS5_OUTFLT     IS 4354;(2+256*17)
AXIS6_OUTFLT     IS 4610;(2+256*18)
AXIS7_OUTFLT     IS 4866;(2+256*19)
AXIS8_OUTFLT     IS 5122;(2+256*20)
```

```
AXIS_FLT_CLR     IS 5378;(2+256*21)
```

```
PLC_INFLT        IS 5634;(2+256*22)
PLC_OUTFLT       IS 5890;(2+256*23)
PLC_FLT_CLR      IS 6146;(2+256*24)
```

```
SPINDLE_FAULT_MSG IS 7681;(1+256*30)
PROBE_FAULT_MSG   IS 8705;(1+256*34)
```

```
KB_JOG_MSG       IS 8962;(2+256*35)
```

```
LUBE_FAULT_MSG   IS 9217;(1+256*36)
LUBE_WARNING_MSG IS 9218;(2+256*36)
PROBE_JOG_FAULT_MSG IS 9473;(1+256*37)
```

```
MIN_SPEED_MSG    IS 9730;(2+256*38)
SOFTWARE_EXIT_MSG IS 9985;(1+256*39)
```

```
MSG_CLEARED_MSG  IS 25345;(1+256*99)
```

```
;-----
;                               INPUT DEFINITIONS
;                               Closed = 1 (green)  Open = 0 (red)
;-----
```

```
Ax1_MinusLimitOk IS INP1
```

```

Ax1_PlusLimitOk      IS INP2
Ax2_MinusLimitOk     IS INP3
Ax2_PlusLimitOk      IS INP4
Ax3_MinusLimitOk     IS INP5
Ax3_PlusLimitOk      IS INP6
;spare               IS INP7
;spare               IS INP8
LubeOk               IS INP9 ;Lube is "ok" when input is closed (*)
SpindleInverterOk    IS INP10 ;Inverter is "ok" when input is closed (*)
EStopOk              IS INP11
SpinLowRange         IS INP12
;spare               IS INP13
;spare               IS INP14
;spare               IS INP15
;spare               IS INP16

```

;If a PLC expansion board (PLCADD1616) is used, the additional inputs will
begin at input 17.

```

; (*) If SpindleInverterOk or LubeOk is moved to a different input, then
;     the P178 inversion logic at the start of MainStage will also need
;     to be updated, to invert the correct input.

```

```

;-----
; INP769 - INP784 encompass the MPU11 onboard input connections
; which are generally used for MPG and probing functions.
;-----

```

```

MechanicalProbe      IS INP769
DSPProbe              IS INP770
ProbeDetect           IS INP771
ProbeAux              IS INP772
MPG_Inc_X_1           IS INP773
MPG_Inc_X_10          IS INP774
MPG_Inc_X_100         IS INP775
MPG_AXIS_1            IS INP776
MPG_AXIS_2            IS INP777
MPG_AXIS_3            IS INP778
MPG_AXIS_4            IS INP779
MPG_AXIS_5            IS INP780
MPG_AXIS_6            IS INP781
MPG_AXIS_7            IS INP782
MPG_AXIS_8            IS INP783

```

```

;-----
;
;           ALLIN1DC PLC Output Definitions
;     Logic 1 = OUTPUT ON (Green), 0 = OUTPUT OFF (Red)
;-----

```

```

NoFaultOut           IS OUT1 ;SPST Type
Lube                  IS OUT2 ;SPST Type
Flood                 IS OUT3 ;SPST Type
Mist                  IS OUT4 ;SPST Type
InverterResetOut      IS OUT5 ;SPST Type
Clamp                 IS OUT6 ;SPST Type - M10 On, M11 Off & Aux7
SpindleEnableOut      IS OUT7 ;SPST Type
SpindleDirectionOut   IS OUT8 ;SPDT Type
;                     IS OUT9 ;SPDT Type

```

;Outputs 10-16 are unavailable

; These bits control the actual analog hardware output on the ALLIN1DC.

; Output = 12bit (0-4095) 0-10VDC.

SpinAnalogOutBit0	IS OUT241
SpinAnalogOutBit1	IS OUT242
SpinAnalogOutBit2	IS OUT243
SpinAnalogOutBit3	IS OUT244
SpinAnalogOutBit4	IS OUT245
SpinAnalogOutBit5	IS OUT246
SpinAnalogOutBit6	IS OUT247
SpinAnalogOutBit7	IS OUT248
SpinAnalogOutBit8	IS OUT249
SpinAnalogOutBit9	IS OUT250
SpinAnalogOutBit10	IS OUT251
SpinAnalogOutBit11	IS OUT252

MPG_LED_OUT	IS OUT769
-------------	-----------

;-----
;
; Memory Bit Definitions
;-----

PLCExecutorFault_M	IS MEM1	
SoftwareNotReady_M	IS MEM2	; 0 = okay, 1 = CNC11 not running/ready
MPGManOffFlag_M	IS MEM3	
Ax1PlusJogDisabled_M	IS MEM11	
Ax1MinusJogDisabled_M	IS MEM12	
Ax2PlusJogDisabled_M	IS MEM13	
Ax2MinusJogDisabled_M	IS MEM14	
MasterEnable_M	IS MEM40	; 1 = enabled (echo of SV_MASTER_ENABLE)
PLCBus_Oe_M	IS MEM41	; 1 = okay, 0 = incoming PLC fiber problem
PLCBusExtDevEn_M	IS MEM42	; 1 = okay, 0 = PLC reports bad output fiber
ADD16160k1_M	IS MEM43	
Stop_M	IS MEM47	; 0 = okay, 1 = fault (echo of SV_STOP)
Stall_M	IS MEM48	; 0 = okay, 1 = stall (echo of SV_STALL_ERROR)
LubeFault_M	IS MEM49	; 0 = okay, 1 = lube fault
PLCFault_M	IS MEM50	; 0 = okay, 1 = PLC fault
AxisFault_M	IS MEM51	; 0 = okay, 1 = drive or drive fiber problem
DriveComFltIn_M	IS MEM52	; 0 = okay, 1 = incoming drive fiber problem
DriveComFltOut_M	IS MEM53	; 0 = okay, 1 = outgoing drive fiber problem
ProbeFault_M	IS MEM54	; 0 = okay, 1 = tried to start spindle w/probe
JogProbeFault_M	IS MEM55	; 0 = okay, 1 = tripped probe while jogging
SpindleFault_M	IS MEM56	; 0 = okay, 1 = spindle drive fault
OtherFault_M	IS MEM57	
KbJpActive_M	IS MEM60	; aka SV_PC_VIRTUAL_JOGPANEL_ACTIVE
Axis1FiberOk_M	IS MEM70	
Axis2FiberOk_M	IS MEM71	
Axis3FiberOk_M	IS MEM72	
Axis4FiberOk_M	IS MEM73	
Axis5FiberOk_M	IS MEM74	
Axis6FiberOk_M	IS MEM75	

Axis7FiberOk_M	IS MEM76
Axis8FiberOk_M	IS MEM77
ProbeMsgSent_M	IS MEM78
true	IS MEM81
SpinLowRange_M	IS MEM82
SpinHighRange_M	IS MEM85
SpindlePause_M	IS MEM86
DisableKbInput_M	IS MEM102 ;If 1, disable kb jogging
AllowKbInput_M	IS MEM103 ;If 1, allow kb jogging
JogOverOnly_M	IS MEM105
KbOverOnly_M	IS MEM106
UsingFeedrateKnob_M	IS MEM117
WaitingForSleepTimer_M	IS MEM118
X1_M	IS MEM119
X10_M	IS MEM120
X100_M	IS MEM121
OnAtPowerUp_M	IS MEM200
LimitTripped	IS MEM208
LastProbeMode_M	IS MEM210
InvLubeOk_M	IS MEM300 ; P178 Bit 0 (1) ; add 1 to parm 178 if ; lubeOk is NO
InvSpinInverterOk_M	IS MEM301 ; P178 Bit 1 (2) ; Add 2 to parm 178 if ; InvSpinInverterOk_M is NO
KbCycleStart_M	IS MEM400 ; "alt" + "s"
KbCycleCancel_M	IS MEM401 ; escape
KbToolCheck_M	IS MEM402 ; "Ctrl" + "t"
KbTogSingleBlock_M	IS MEM403 ; "Ctrl" + "b"
KbIncreaseJogInc_M	IS MEM404 ; "Insert"
KbDecreaseJogInc_M	IS MEM405 ; "Delete"
KbIncFeedOver_M	IS MEM406 ; "ctrl" + "keyboard "+" ("=")
KbDecFeedOver_M	IS MEM407 ; "ctrl" + "keyboard "-"
KbFeedOver100_M	IS MEM450 ; "ctrl" + "\"
KbTogIncContJog_M	IS MEM408 ; "ctrl" + "i"
KbTogFastSlowJog_M	IS MEM409 ; "ctrl" + "f"
KbJogAx1Plus_M	IS MEM411 ; right arrow + KbJpActive_M
KbJogAx1Minus_M	IS MEM412 ; left arrow + KbJpActive_M
KbJogAx2Plus_M	IS MEM413 ; up arrow + KbJpActive_M
KbJogAx2Minus_M	IS MEM414 ; down arrow + KbJpActive_M
KbJogAx3Plus_M	IS MEM415 ; page up + KbJpActive_M
KbJogAx3Minus_M	IS MEM416 ; page down + KbJpActive_M
KbJogAx4Plus_M	IS MEM417 ; "home"+ KbJpActive_M
KbJogAx4Minus_M	IS MEM418 ; "end" + KbJpActive_M
KbAux1Key_M	IS MEM419 ; "ctrl" + "F1"
KbAux2Key_M	IS MEM420 ; "ctrl" + "F2"
KbAux3Key_M	IS MEM421 ; "ctrl" + "F3"
KbAux4Key_M	IS MEM422 ; "ctrl" + "F4"
KbAux5Key_M	IS MEM423 ; "ctrl" + "F5"
KbAux6Key_M	IS MEM424 ; "ctrl" + "F6"
KbAux7Key_M	IS MEM425 ; "ctrl" + "F7"
KbAux8Key_M	IS MEM426 ; "ctrl" + "F8"
KbAux9Key_M	IS MEM427 ; "ctrl" + "F9"
KbAux10Key_M	IS MEM428 ; "ctrl" + "F10"

KbAux11Key_M	IS MEM429 ; "ctrl" + "F11"
KbAux12Key_M	IS MEM430 ; "ctrl" + "F12"
KbTogRapidOver_M	IS MEM431 ; "ctrl" + "r"
KbTogSpinAutoMan_M	IS MEM432 ; "ctrl" + "a"
KbSpinCW_M	IS MEM433 ; "ctrl" + "c"
KbSpinCCW_M	IS MEM434 ; "ctrl" + "w"
KbSpinStart_M	IS MEM435 ; "ctrl" + "s"
KbSpinStop_M	IS MEM436 ; "ctrl" + "q"
KbFloodOnOff_M	IS MEM437 ; "ctrl" + "n"
KbMistOnOff_M	IS MEM451 ; "ctrl" + "k"
KbTogCoolAutoMan_M	IS MEM438 ; "ctrl" + "m"
KbFeedHold_M	IS MEM439 ; space bar
KbIncSpinOver_M	IS MEM440 ; "ctrl" + ">" (.)
KbDecSpinOver_M	IS MEM441 ; "ctrl" + "<" (,)
KbSpinOver100_M	IS MEM442 ; "ctrl" + "?" (/)

;-----
 ; Jog panel keys are referenced as JPI1 through JPI256. Alternatively,
 ; jog panel inputs can also be referenced as INP1057-INP1312.
 ;-----
 ; Definitions follow JOGBOARD layout top to bottom, left to right

SpinOverPlusKey	IS JPI1 ; Row 1 Column 1
SpinAutoManKey	IS JPI2 ; Row 1 Column 2
Aux1Key	IS JPI3 ; Row 1 Column 3
Aux2Key	IS JPI4 ; Row 1 Column 4
Aux3Key	IS JPI5 ; Row 1 Column 5
SpinOver100Key	IS JPI6 ; Row 2 Column 1
SpinCWKey	IS JPI7 ; Row 2 Column 2
Aux4Key	IS JPI8 ; Row 2 Column 3
Aux5Key	IS JPI9 ; Row 2 Column 4
Aux6Key	IS JPI10 ; Row 2 Column 5
SpinOverMinusKey	IS JPI11 ; Row 3 Column 1
SpinCCWKey	IS JPI12 ; Row 3 Column 2
Aux7Key	IS JPI13 ; Row 3 Column 3
Aux8Key	IS JPI14 ; Row 3 Column 4
Aux9Key	IS JPI15 ; Row 3 Column 5
SpinStopKey	IS JPI16 ; Row 4 Column 1
SpinStartKey	IS JPI17 ; Row 4 Column 2
Aux10Key	IS JPI18 ; Row 4 Column 3
Aux11Key	IS JPI19 ; Row 4 Column 4
Aux12Key	IS JPI20 ; Row 4 Column 5
CoolAutoManKey	IS JPI21 ; Row 5 Column 1
CoolFloodKey	IS JPI22 ; Row 5 Column 2
CoolMistKey	IS JPI23 ; Row 5 Column 3
Aux13Key	IS JPI24 ; Row 5 Column 4
Aux14Key	IS JPI25 ; Row 5 Column 5
IncrContKey	IS JPI26 ; Row 6 Column 1
x1JogKey	IS JPI27 ; Row 6 Column 2
x10JogKey	IS JPI28 ; Row 6 Column 3
x100JogKey	IS JPI29 ; Row 6 Column 4
MPGKey	IS JPI30 ; Row 6 Column 5

```

Ax4PlusJogKey      IS JPI31 ; Row 7 Column 1
UnusedR7C2Key      IS JPI32 ; Row 7 Column 2
Ax2PlusJogKey      IS JPI33 ; Row 7 Column 3
UnusedR7C4Key      IS JPI34 ; Row 7 Column 4
Ax3PlusJogKey      IS JPI35 ; Row 7 Column 5

UnusedR8C1Key      IS JPI36 ; Row 8 Column 1
Ax1MinusJogKey     IS JPI37 ; Row 8 Column 2
FastSlowKey        IS JPI38 ; Row 8 Column 3
Ax1PlusJogKey      IS JPI39 ; Row 8 Column 4
UnusedR8C5Key      IS JPI40 ; Row 8 Column 5

Ax4MinusJogKey     IS JPI41 ; Row 9 Column 1
UnusedR9C2Key      IS JPI42 ; Row 9 Column 2
Ax2MinusJogKey     IS JPI43 ; Row 9 Column 3
UnusedR9C4Key      IS JPI44 ; Row 9 Column 4
Ax3MinusJogKey     IS JPI45 ; Row 9 Column 5

CycleCancelKey     IS JPI46 ; Row 10 Column 1
SingleBlockKey     IS JPI47 ; Row 10 Column 2
ToolCheckKey       IS JPI48 ; Row 10 Column 3
FeedHoldKey        IS JPI49 ; Row 10 Column 4
CycleStartKey      IS JPI50 ; Row 10 Column 5

;-----
;                               Feedrate Override Knob
;-----
JpFeedOrKnobBit0   IS JPI193
JpFeedOrKnobBit1   IS JPI194
JpFeedOrKnobBit2   IS JPI195
JpFeedOrKnobBit3   IS JPI196
JpFeedOrKnobBit4   IS JPI197
JpFeedOrKnobBit5   IS JPI198
JpFeedOrKnobBit6   IS JPI199
JpFeedOrKnobBit7   IS JPI200
JpFeedOrKnobBit8   IS JPI201 ; Current jog panels send first 8 bits
JpFeedOrKnobBit9   IS JPI202 ; unused
JpFeedOrKnobBit10  IS JPI203 ; unused
JpFeedOrKnobBit11  IS JPI204 ; unused
JpFeedOrKnobBit12  IS JPI205 ; unused
JpFeedOrKnobBit13  IS JPI206 ; unused
JpFeedOrKnobBit14  IS JPI207 ; unused
JpFeedOrKnobBit15  IS JPI208 ; unused

;-----
;                               Jog Panel Output (LED) Definitions
;                               Jog Panel LED's can be addressed as JP01 - JP0256
;                               OR
;                               OUT1057 - OUT1312
;-----
; Definitions follow JOGBOARD layout top to bottom, left to right
;
SpinOverPlusLED     IS JP01 ; Row 1 Column 1
SpinAutoModeLED     IS JP02 ; Row 1 Column 2
Aux1LED             IS JP03 ; Row 1 Column 3
Aux2LED             IS JP04 ; Row 1 Column 4

```

Aux3LED	IS JP05 ; Row 1 Column 5
SpinOver100LED	IS JP06 ; Row 2 Column 1
SpindleCWLED	IS JP07 ; Row 2 Column 2
Aux4LED	IS JP08 ; Row 2 Column 3
Aux5LED	IS JP09 ; Row 2 Column 4
Aux6LED	IS JP010 ; Row 2 Column 5
SpinOverMinusLED	IS JP011 ; Row 3 Column 1
SpindleCCWLED	IS JP012 ; Row 3 Column 2
Aux7LED	IS JP013 ; Row 3 Column 3
Aux8LED	IS JP014 ; Row 3 Column 4
Aux9LED	IS JP015 ; Row 3 Column 5
SpinStopLED	IS JP016 ; Row 4 Column 1
SpinStartLED	IS JP017 ; Row 4 Column 2
Aux10LED	IS JP018 ; Row 4 Column 3
Aux11LED	IS JP019 ; Row 4 Column 4
Aux12LED	IS JP020 ; Row 4 Column 5
CoolAutoManLED	IS JP021 ; Row 5 Column 1
CoolFloodLED	IS JP022 ; Row 5 Column 2
CoolMistLED	IS JP023 ; Row 5 Column 3
Aux13LED	IS JP024 ; Row 5 Column 4
Aux14LED	IS JP025 ; Row 5 Column 5
IncrContLED	IS JP026 ; Row 6 Column 1
x1JogLED	IS JP027 ; Row 6 Column 2
x10JogLED	IS JP028 ; Row 6 Column 3
x100JogLED	IS JP029 ; Row 6 Column 4
MPGLEDD	IS JP030 ; Row 6 Column 5
Ax4PlusJogLED	IS JP031 ; Row 7 Column 1
UnusedR7C2LED	IS JP032 ; Row 7 Column 2
Ax2PlusJogLED	IS JP033 ; Row 7 Column 3
UnusedR7C4LED	IS JP034 ; Row 7 Column 4
Ax3PlusJogLED	IS JP035 ; Row 7 Column 5
UnusedR8C1LED	IS JP036 ; Row 8 Column 1
Ax1MinusJogLED	IS JP037 ; Row 8 Column 2
FastSlowLED	IS JP038 ; Row 8 Column 3
Ax1PlusJogLED	IS JP039 ; Row 8 Column 4
UnusedR8C5LED	IS JP040 ; Row 8 Column 5
Ax4MinusJogLED	IS JP041 ; Row 9 Column 1
UnusedR9C2LED	IS JP042 ; Row 9 Column 2
Ax2MinusJogLED	IS JP043 ; Row 9 Column 3
UnusedR9C4LED	IS JP044 ; Row 9 Column 4
Ax3MinusJogLED	IS JP045 ; Row 9 Column 5
CycleCancelLED	IS JP046 ; Row 10 Column 1
SingleBlockLED	IS JP047 ; Row 10 Column 2
; FOR JOGBRD REV?????, the LED outputs do not match Key inputs	
; The PLC program should activate all three of these when	
; it wants to turn on FeedHoldLED so that future hardware changes	
; to put them in the same order as their corresponding inputs will work.	
ToolCheckLED	IS JP050 ; Row 10 Column 3

```
FeedHoldLED      IS JP048 ; Row 10 Column 4
CycleStartLED    IS JP049 ; Row 10 Column 5
```

```
;-----
;               -----SYSTEM VARIABLES-----
;
; For a complete list of System Variables and their functions, please see the
; MPU11 PLC manual.
;-----

; MPU11 based systems provide the PLC with the ability to read/write to a
; limited number of "System Variables". While the use of System Variables
; greatly expands PLC functionality, it comes with additional responsibility on
; the part of the PLC programmer. Functionality that was once implemented as
; default behavior such as jogging, spindle speed, feedrate override, spindle
; gear ranges etc... is now implemented through System Variables in the PLC
; program. It is now the sole responsibility of the PLC program to provide a
; method to jog an axis, override the spindle speed or feedrates or even map a
; jog panel keypress to a specific function. Pressing a jog key or Aux key
; won't DO anything unless the PLC assigns an action to the keypress. All jog
; panel functions MUST be explicitly implemented in the PLC program.
;
;      ----IMPORTANT----
; Menu navigation in the CNC software requires that the escape key or Cycle
; Cancel key is used to back out of menus and screens. You must use the PLC
; program to map a jog panel key and/or a keyboard key to the Cycle Cancel
; System Variable (SV_PLC_FUNCTION_1 has been declared as "DoCycleCancel")
; in order to use the control. For example:
; The following lines map the escape key and Jog Panel Cycle Cancel key to
; produce a Cycle Cancel event:

; 1. Map escape keypress event to identifier to describe what key was pressed.
;   Kb_Escape      IS SV_PC_Keyboard_Key_1

; 2. Map MEM bit to identifier that describes what the keypress is used for.
;   KbCycleCancel_M IS MEM401

; 3. Logic to "set" KbCycleCancel_M anytime the escape key is pressed.
;   if Kb_Escape THEN(KbCycleCancel_M)

; 4. Logic to cancel job if the escape key or cycle cancel key is pressed.
;   IF (CycleCancelKey || KbCycleCancel_M) && SV_PROGRAM_RUNNING
;   THEN (DoCycleCancel)

; Some of the information made available to the PLC through System Variables:
; 1. Encoder positions: SV_MPU11_ABS_POS_1 - SV_MPU11_ABS_POS_7
; 2. Parameter values: SV_MACHINE_PARAMETER_1 - SV_MACHINE_PARAMETER_999
; 3. Spindle Speed command from PC: SV_PC_DAC_SPINDLE_SPEED
; 4. PC Keyboard Keypress: SV_PC_FUNCTION_1 - SV_PC_FUNCTION_127
; 5. ...

; Some of the functionality controlled by the PLC through System Variables:
; 1. Axis jogging: SV_PLC_FUNCTION_12 - SV_PLC_FUNCTION_23
; 2. "Final" Spindle speed reported to PC: SV_PLC_SPINDLE_SPEED -provides nearly
;    unlimited gear ranges
; 3. Feedrate (through override knob): SV_PLC_FeedrateKnob_W
; 4. Custom debounce, invert/force inputs: SV_PLC_DEBOUNCE_1-SV_PLC_DEBOUNCE_64
; 5. ...
```

```

;-----
;      PLC Input manipulation - SV_PLC_DEBOUNCE_1 - SV_PLC_DEBOUNCE_64
; The System Variables in this section are used to modify the characteristics
; of PLC inputs 1-240. Each input can be inverted, forced or assigned a custom
; debounce time.

;-----Debounce Times-----
; SV_PLC_DEBOUNCE_61 - SV_PLC_DEBOUNCE_64 are used to define up to seven custom
; debounce times which can be selected for each input.

; The 32 bit integer System Variables SV_PLC_DEBOUNCE_61 - SV_PLC_DEBOUNCE_64,
; are broken up into 8, 16 bit words, only 7 of which are used. The first word,
; the 16 MSB of SV_PLC_DEBOUNCE_61 is unused. Each 16 bit word can be used to
; store a debounce time of between 0-32767 (the MSB of each word is unused).
; Debounce times are in increments of 62.5 usecs which provides debounce times
; of up to ~2 secs.

;
;      SV_PLC_DEBOUNCE_61
;      Unused:Bits 0-15 (Selection 0)
;      15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

;
;      Debounce Time Selection #1
;      MSB 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16

;
;      SV_PLC_DEBOUNCE_62
;      Debounce Time Selection #2
;      15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

;
;      Debounce Time Selection #3
;      MSB 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16

;-----Configuring Input Behavior-----
; Each System Variable from SV_PLC_DEBOUNCE_1 - SV_PLC_DEBOUNCE_60 is a 32 bit
; integer word broken up into 4 bit words to control the behavior of 4 inputs.
; Inputs 1-4 are configured using SV_PLC_DEBOUNCE_1, inputs 5-8 are handled
; using SV_PLC_DEBOUNCE_2 and so on to SV_PLC_DEBOUNCE_60 which controls inputs
; 237-240

; As mentioned above, each 32 bit word defines the characteristics for 4 inputs.
; SV_PLC_DEBOUNCE_1 defines the characteristics of INP1, INP2, INP3 & INP4 and
; so on through SV_PLC_DEBOUNCE_60 which handles INP237, INP238, INP239&INP240.
; The behavior of an input is set as follows:

; Five new operators have been introduced to simplify bit operations:
; BitSet, BitRst, BitTst, LShift & Rshift. Below we will use bitset to
; invert an input. This is convenient to use when a device is normally
; open and the logic is written for a normally closed device. Inverting
; the input allows to reuse the existing logic rather than rewrite it.

; bitset and bitrst can not operate directly on SV_PLC_DEBOUNCE_# system
; variables, they can only operate on W32 variables. In order to use bitset and
; bitrst to manipulate the debounce variables you'll have to perform all
; operations on a w32 first:

; Declare a W32:
; Inputs_9_12_W IS W1

```

```

; use BITSET or BTRST
; if 1 == 1 THEN bitset Inputs_9_12_W 14 ;invert INP10 (bit14)

; Set Debounce system variable = to W32 variable
; if 1 == 1 THEN SV_PLC_DEBOUNCE_3 = Inputs_9_12_W

;-----System Variable = SV_PLC_DEBOUNCE_1-----
;
;           Inp4 = bits 31-24           Inp3 = bits 23-16
;           MSB 31 30 29 28 27 26 25 24| 23 22 21 20 19 18 17 16
;
;           Inp2 = bits 15-8           Inp1 = bits 7-0
;           15 14 13 12 11 10 9 8| 7 6 5 4 3 2 1 0LSB
;
;           Each 8 bit word from above
;           MSB      7      6      5      4      3      2      1      0      LSB
;           Force  Invert Spare  Spare  Spare  Debounce Select (7)
;                                     selects 1 of 7
;                                     debounce times
;                                     (zero is invalid)
; Force (bit 7): Set this bit to force the input to a 1* (closed)
; Invert(bit 6): Set this to invert an input
; Spare(bit5-3): Not used
; Debounce(bit 0-2): Selects one of the 7 preset debounce times defined in
;                   SV_PLC_DEBOUNCE_61 - SV_PLC_DEBOUNCE_64
;
; *If you wish to force an input to 0, set the both invert AND force bits
; for the input.

;-----
; PLC Jog Panel input manipulation - The System Variables in this section are
; used to modify the characteristics of the Jog Panel keys. The jog panel keys
; can be configured in the same manner as the PLC inputs and use debounce times
; as selected/set in SV_PLC_DEBOUNCE_61 - SV_PLC_DEBOUNCE_64.
;-----

;-----
; System variables: Jog Panel Functions
;-----
; Jog panel functions
;Invalid                IS SV_PLC_FUNCTION_0
DoCycleCancel           IS SV_PLC_FUNCTION_1
DoCycleStart            IS SV_PLC_FUNCTION_2
DoToolCheck             IS SV_PLC_FUNCTION_3
SelectSingleBlock       IS SV_PLC_FUNCTION_4
SelectX1JogInc          IS SV_PLC_FUNCTION_5
SelectX10JogInc         IS SV_PLC_FUNCTION_6
SelectX100JogInc        IS SV_PLC_FUNCTION_7
SelectUserJogInc        IS SV_PLC_FUNCTION_8
SelectIncContJog        IS SV_PLC_FUNCTION_9
SelectFastSlowJog       IS SV_PLC_FUNCTION_10
SelectMpgMode           IS SV_PLC_FUNCTION_11
DoAx1PlusJog            IS SV_PLC_FUNCTION_12
DoAx1MinusJog           IS SV_PLC_FUNCTION_13
DoAx2PlusJog            IS SV_PLC_FUNCTION_14
DoAx2MinusJog           IS SV_PLC_FUNCTION_15
DoAx3PlusJog            IS SV_PLC_FUNCTION_16

```

DoAx3MinusJog	IS SV_PLC_FUNCTION_17
DoAx4PlusJog	IS SV_PLC_FUNCTION_18
DoAx4MinusJog	IS SV_PLC_FUNCTION_19
DoAx5PlusJog	IS SV_PLC_FUNCTION_20
DoAx5MinusJog	IS SV_PLC_FUNCTION_21
DoAx6PlusJog	IS SV_PLC_FUNCTION_22
DoAx6MinusJog	IS SV_PLC_FUNCTION_23
DoAux1Key	IS SV_PLC_FUNCTION_24
DoAux2Key	IS SV_PLC_FUNCTION_25
DoAux3Key	IS SV_PLC_FUNCTION_26
DoAux4Key	IS SV_PLC_FUNCTION_27
DoAux5Key	IS SV_PLC_FUNCTION_28
DoAux6Key	IS SV_PLC_FUNCTION_29
DoAux7Key	IS SV_PLC_FUNCTION_30
DoAux8Key	IS SV_PLC_FUNCTION_31
DoAux9Key	IS SV_PLC_FUNCTION_32
DoAux10Key	IS SV_PLC_FUNCTION_33
SelectRapidOverride	IS SV_PLC_FUNCTION_34
SelectManAutoSpindle	IS SV_PLC_FUNCTION_35
DoSpindleStart	IS SV_PLC_FUNCTION_37
DoSpindleStop	IS SV_PLC_FUNCTION_38
DoAux11Key	IS SV_PLC_FUNCTION_39
DoAux12Key	IS SV_PLC_FUNCTION_40
;SelectCoolantMan	IS SV_PLC_FUNCTION_41 ;deprecated
;SelectCoolantAuto	IS SV_PLC_FUNCTION_42 ;deprecated
SelectCoolantFlood	IS SV_PLC_FUNCTION_43
SelectCoolantMist	IS SV_PLC_FUNCTION_44
DoFeedHold	IS SV_PLC_FUNCTION_45
SelectSpindleCCW	IS SV_PLC_FUNCTION_98
SelectSpindleCW	IS SV_PLC_FUNCTION_99
SelectCoolAutoMan	is SV_PLC_FUNCTION_104
DoIncreaseSpindleOr	IS SV_PLC_FUNCTION_106
DoDecreaseSpindleOr	IS SV_PLC_FUNCTION_107
SelectSpinOr100	IS SV_PLC_FUNCTION_108

```

;-----
; System variables: Keyboard jogging functions
;-----
;-----
; Keyboard Jogging Keys - The System Variables in this section inform the PLC
; that a PC keyboard keypress has occurred. Keep in mind that some key presses
; only come down while the keyboard jogging screen is enabled (alt-j) and that
; NONE of these keys not perform ANY default actions unless programmed to do so.
; The assignments provided below are for reference only. For an example of
; mapping a keyboard key press to an MPU11 action, see the logic assigned to
; KbCycleStart_M or KbCycleCancel_M.
;
;Note:
; Keypresses are sent down as individual keys. It is the responsibility of
; the PLC programmer to insure that a keypress is only acted on at the
; appropriate times.
; The "SV_PC_VIRTUAL_JOGPANEL_ACTIVE" system variable can be used to prevent
; a keypress from being acted on unless the keyboard jog screen is being
; displayed. NOTE The above,29 character sys variable is mapped to
; KbJpActive_M (MEM80) to make it a "little" shorter.....
;-----
Kb_a          IS SV_PC_KEYBOARD_KEY_60

```

Kb_b	IS SV_PC_KEYBOARD_KEY_79
Kb_c	IS SV_PC_KEYBOARD_KEY_77
Kb_d	IS SV_PC_KEYBOARD_KEY_62
Kb_e	IS SV_PC_KEYBOARD_KEY_41
Kb_f	IS SV_PC_KEYBOARD_KEY_63
Kb_g	IS SV_PC_KEYBOARD_KEY_64
Kb_h	IS SV_PC_KEYBOARD_KEY_65
Kb_i	IS SV_PC_KEYBOARD_KEY_46
Kb_j	IS SV_PC_KEYBOARD_KEY_66
Kb_k	IS SV_PC_KEYBOARD_KEY_67
Kb_l	IS SV_PC_KEYBOARD_KEY_68
Kb_m	IS SV_PC_KEYBOARD_KEY_81
Kb_n	IS SV_PC_KEYBOARD_KEY_80
Kb_o	IS SV_PC_KEYBOARD_KEY_47
Kb_p	IS SV_PC_KEYBOARD_KEY_48
Kb_q	IS SV_PC_KEYBOARD_KEY_39
Kb_r	IS SV_PC_KEYBOARD_KEY_42
Kb_s	IS SV_PC_KEYBOARD_KEY_61
Kb_t	IS SV_PC_KEYBOARD_KEY_43
Kb_u	IS SV_PC_KEYBOARD_KEY_45
Kb_v	IS SV_PC_KEYBOARD_KEY_78
Kb_w	IS SV_PC_KEYBOARD_KEY_40
Kb_x	IS SV_PC_KEYBOARD_KEY_76
Kb_y	IS SV_PC_KEYBOARD_KEY_44
Kb_z	IS SV_PC_KEYBOARD_KEY_75
Kb_spacebar	IS SV_PC_KEYBOARD_KEY_95
Kb_L_Shift	IS SV_PC_KEYBOARD_KEY_74
Kb_R_Shift	IS SV_PC_KEYBOARD_KEY_85
Kb_L_Alt	IS SV_PC_KEYBOARD_KEY_94
Kb_R_Alt	IS SV_PC_KEYBOARD_KEY_96
Kb_L_Ctrl	IS SV_PC_KEYBOARD_KEY_92
Kb_R_Ctrl	IS SV_PC_KEYBOARD_KEY_99
Kb_Ins	IS SV_PC_KEYBOARD_KEY_31
Kb_Home	IS SV_PC_KEYBOARD_KEY_32
Kb_End	IS SV_PC_KEYBOARD_KEY_53
Kb_PgDown	IS SV_PC_KEYBOARD_KEY_54
Kb_PgUp	IS SV_PC_KEYBOARD_KEY_33
Kb_Del	IS SV_PC_KEYBOARD_KEY_52
Kb_Back	IS SV_PC_KEYBOARD_KEY_30
Kb_Tab	IS SV_PC_KEYBOARD_KEY_38
Kb_Up	IS SV_PC_KEYBOARD_KEY_87
Kb_Down	IS SV_PC_KEYBOARD_KEY_101
Kb_Left	IS SV_PC_KEYBOARD_KEY_100
Kb_Right	IS SV_PC_KEYBOARD_KEY_102
Kb_Escape	IS SV_PC_KEYBOARD_KEY_1 ;Performs Cycle Cancel
Kb_F1	IS SV_PC_KEYBOARD_KEY_2
Kb_F2	IS SV_PC_KEYBOARD_KEY_3
Kb_F3	IS SV_PC_KEYBOARD_KEY_4
Kb_F4	IS SV_PC_KEYBOARD_KEY_5
Kb_F5	IS SV_PC_KEYBOARD_KEY_6
Kb_F6	IS SV_PC_KEYBOARD_KEY_7
Kb_F7	IS SV_PC_KEYBOARD_KEY_8
Kb_F8	IS SV_PC_KEYBOARD_KEY_9
Kb_F9	IS SV_PC_KEYBOARD_KEY_10
Kb_F10	IS SV_PC_KEYBOARD_KEY_11
Kb_F11	IS SV_PC_KEYBOARD_KEY_12
Kb_F12	IS SV_PC_KEYBOARD_KEY_13

Kb_Prt_Scrn	IS SV_PC_KEYBOARD_KEY_14
Kb_Scr1_Lck	IS SV_PC_KEYBOARD_KEY_15
Kb_Break	IS SV_PC_KEYBOARD_KEY_16
Kb_Num_Lock	IS SV_PC_KEYBOARD_KEY_34
Kb_1	IS SV_PC_KEYBOARD_KEY_18
Kb_2	IS SV_PC_KEYBOARD_KEY_19
Kb_3	IS SV_PC_KEYBOARD_KEY_20
Kb_4	IS SV_PC_KEYBOARD_KEY_21
Kb_5	IS SV_PC_KEYBOARD_KEY_22
Kb_6	IS SV_PC_KEYBOARD_KEY_23
Kb_7	IS SV_PC_KEYBOARD_KEY_24
Kb_8	IS SV_PC_KEYBOARD_KEY_25
Kb_9	IS SV_PC_KEYBOARD_KEY_26
Kb_0	IS SV_PC_KEYBOARD_KEY_27
Kb_10_Key_Div	IS SV_PC_KEYBOARD_KEY_35
Kb_10_Key_Mlt	IS SV_PC_KEYBOARD_KEY_36
Kb_10_Key_Sub	IS SV_PC_KEYBOARD_KEY_37
Kb_10_Key_0	IS SV_PC_KEYBOARD_KEY_103
Kb_10_Key_1	IS SV_PC_KEYBOARD_KEY_88
Kb_10_Key_2	IS SV_PC_KEYBOARD_KEY_89
Kb_10_Key_3	IS SV_PC_KEYBOARD_KEY_90
Kb_10_Key_4	IS SV_PC_KEYBOARD_KEY_71
Kb_10_Key_5	IS SV_PC_KEYBOARD_KEY_72
Kb_10_Key_6	IS SV_PC_KEYBOARD_KEY_73
Kb_10_Key_7	IS SV_PC_KEYBOARD_KEY_55
Kb_10_Key_8	IS SV_PC_KEYBOARD_KEY_56
Kb_10_Key_9	IS SV_PC_KEYBOARD_KEY_57
Kb_10_Key_Dec_Pt	IS SV_PC_KEYBOARD_KEY_104
Kb_10_Key_Plus	IS SV_PC_KEYBOARD_KEY_58
Kb_Num_Enter	IS SV_PC_KEYBOARD_KEY_91
Kb_L_Sq_Bracket	IS SV_PC_KEYBOARD_KEY_49
Kb_R_Sq_Bracket	IS SV_PC_KEYBOARD_KEY_50
Kb_Hyphen	IS SV_PC_KEYBOARD_KEY_28
Kb_Equals	IS SV_PC_KEYBOARD_KEY_29
Kb_Comma	IS SV_PC_KEYBOARD_KEY_82
Kb_Period	IS SV_PC_KEYBOARD_KEY_83
Kb_Slash	IS SV_PC_KEYBOARD_KEY_84
Kb_Backslash	IS SV_PC_KEYBOARD_KEY_86

```

;-----
; M functions - The System Variables in this section inform the
; PLC that an M function has been requested.
;-----
M3      IS SV_M94_M95_1 ;(Spindle CW)
M4      IS SV_M94_M95_2 ;(Spindle CCW)
M8      IS SV_M94_M95_3 ;(Flood On)
M10     IS SV_M94_M95_4 ; Clamp
M7      IS SV_M94_M95_5 ;(Mist)
;       IS SV_M94_M95_6 ;
;       IS SV_M94_M95_7 ;
;       IS SV_M94_M95_8 ;
;       IS SV_M94_M95_9 ;
;       IS SV_M94_M95_10;
;       IS SV_M94_M95_11;
;       IS SV_M94_M95_12;
;       IS SV_M94_M95_13;
;       IS SV_M94_M95_14;

```

```
;      IS SV_M94_M95_15;
;      IS SV_M94_M95_16;
```

```
;-----
;                               Word Definitions (int32)
;-----
```

ErrorCode_W	IS W1
TwelveBitSpeed_W	IS W2
LubeAccumTime_W	IS W3
KbOverride_W	IS W4
FeedrateKnob_W	IS W5
CycloneStatus_W	IS W6
FinalFeedOverride_W	IS W7
PLC_Fault_W	IS W8
PLCFaultAddr_W	IS W9
Last_FeedrateKnob_W	IS W10
AsyncMsg_W	IS W11
P148Value_W	IS W12
Lube_W	IS W21
LubeM_W	IS W22
LubeS_W	IS W23
Inputs_9_12_W	IS W28
P170Value_W	IS W30
P178Value_W	IS W31

```
;-----
;                               Word Definitions cont. (f32)
;-----
```

SpinRangeAdjust	IS FW1
RPMPerBit_FW	IS FW2
CfgMinSpeed_FW	IS FW3
CfgMaxSpeed_FW	IS FW4
TwelveBitSpeed_FW	IS FW5
SpinSpeedCommand_FW	IS FW6

```
;-----
;                               One Shot Definitions
;-----
```

IncrContPD	IS PD1
SlowFastPD	IS PD2
MpgPD	IS PD3
SingleBlockPD	IS PD4
FeedHoldPD	IS PD5
SpinAutoManPD	IS PD6
SpindlePlusPD	IS PD7
SpinOverMinusPD	IS PD8
SpinOver100PD	IS PD9
SpinStartPD	IS PD10
SpinStopPD	IS PD11
SpinCWPD	IS PD12
SpinCCWPD	IS PD13
F9PD	IS PD14
x1JogPD	IS PD15
x10JogPD	IS PD16
x100JogPD	IS PD17

Aux11KeyPD	IS PD18
RapidOverPD	IS PD19
CoolantAutoManualPD	IS PD21
CoolantFloodPD	IS PD22
CoolantMistPD	IS PD23
ToolCheckPD	IS PD24
JogProbeFaultPD	IS PD25
RigidTapPD	IS PD26
PCSpindleStartStopPD	IS PD30
PCSpindleManualPD	IS PD31
PCSpindleCWPD	IS PD32
PCSpindleCCWPD	IS PD33
StopRunningPD	IS PD35
SoftwareReadyPD	IS PD36
Aux1PD	IS PD41
Aux2PD	IS PD42
Aux3PD	IS PD43
Aux4PD	IS PD44
Aux5PD	IS PD45
Aux6PD	IS PD46
Aux7PD	IS PD47
Aux8PD	IS PD48
Aux9PD	IS PD49

```

;-----
;                               Timer Definitions
;-----
; 1000 = 1 second for all timers.
;
MsgClear_T           IS T1
SleepTimer           IS T2
CycloneStatus_T      IS T3
Initialize_T          IS T4
LubeM_T              IS T13
LubeS_T              IS T14

```

```

;-----
;                               Stage Definitions
;-----
WatchDogStage        IS STG1
InitialStage         IS STG2
JogPanelStage        IS STG3
MainStage            IS STG4
AxesEnableStage      IS STG5
SpindleStage         IS STG6
MPGStage             IS STG7
CheckCycloneStatusStage IS STG8
LoadParametersStage  IS STG9
KeyboardEventsStage  IS STG10
LubeUsePumpTimersStage IS STG13
LubeUsePLCTimersStage IS STG14

SetErrorStage        IS STG50
BadErrorStage        IS STG51

```

```

;#####
;                               Program Start

```

```

;#####

;=====
;                               WatchDogStage
;=====

; Handle PLC executor faults. The only way to reset a PLC executor fault
; is to reboot the MPU11.
IF SV_PLC_FAULT_STATUS != 0
    THEN PLC_Fault_W      = SV_PLC_FAULT_STATUS,
         PLCFaultAddr_W = SV_PLC_FAULT_ADDRESS,
         ErrorCode_W    = PLC_EXECUTOR_FLT_MSG, MSG_ErrorCode_W,
         SET PLCExecutorFault_M, RST SetErrorStage, SET SV_STOP

; Handle software exit.
IF !SV_PC_SOFTWARE_READY && (SV_PLC_FAULT_STATUS == 0)
    THEN SET SoftwareNotReady_M,
         SET SV_STOP,
         ErrorCode_W = SOFTWARE_EXIT_MSG

if SV_PC_SOFTWARE_READY && (SV_PLC_FAULT_STATUS == 0) THEN (SoftwareReadyPD)
IF SoftwareReadyPD && !SoftwareNotReady_M || !True THEN SET InitialStage

IF SoftwareReadyPD && SoftwareNotReady_M THEN RST SoftwareNotReady_M

;=====
;                               InitialStage
;=====
IF 1==1 THEN SET true,
    SET OnAtPowerUp_M,
    SET AxesEnableStage,
    SET MainStage,
    SET JogPanelStage,
    SET LoadParametersStage,
    SET MPGStage,
    SET PLCBus_Oe_M,
    RST DriveComFltIn_M,
    RST DriveComFltOut_M,
    RST PLCFault_M,
    CycloneStatus_T = 300,
    ErrorCode_W = MSG_CLEARED_MSG,
    RST BadErrorStage,
    SET SetErrorStage,
    Initialize_T = 1000, SET Initialize_T,
    RST InitialStage

;=====
;                               LoadParametersStage
;=====

; There are two methods of control for the lube pump and they are set by CNC11
; Machine Parameter 179, where the value is between 0 - 65535 and is formatted
; as MMMSS where MMM is a time in minutes and SS is a time in seconds.
;
; METHOD 1 (SS == 0) For lube pumps with internal timers.
; METHOD 2 (SS != 0) For lube pumps with no timers (controlled solely by PLC).

```

```

;
; Load lube pump times from P179 and convert to milliseconds.
IF true THEN Lube_W = SV_MACHINE_PARAMETER_179,
              LubeM_W = (Lube_W / 100) * 60000,
              LubeS_W = (Lube_W % 100) * 1000

; Set the appropriate stage according to method of control
IF LubeS_W == 0 THEN SET LubeUsePumpTimersStage, RST LubeUsePLCTimersStage
IF LubeS_W != 0 THEN SET LubeUsePLCTimersStage, RST LubeUsePumpTimersStage

IF True THEN P148Value_W = SV_MACHINE_PARAMETER_148, ; Misc Jogging Options
              P170Value_W = SV_MACHINE_PARAMETER_170, ; Enable Keyboard Jogging
              P178Value_W = SV_MACHINE_PARAMETER_178 ; PLC IO NO / NC Settings

IF True THEN BITTST P148Value_W 1 DisableKbInput_M,
              BITTST P170Value_W 0 AllowKbInput_M
IF DisableKbInput_M THEN RST AllowKbInput_M
If true THEN BitTst P170Value_W 1 JogOverOnly_M
If true THEN BitTst P170Value_W 2 KbOverOnly_M
if JogOverOnly_M && KbOverOnly_M THEN rst KbOverOnly_M

If true THEN BitTst P178Value_W 0 InvLubeOk_M
If true THEN BitTst P178Value_W 1 InvSpinInverterOk_M

;=====
;                               LubeUsePumpTimersStage
;=====

; METHOD 1 (SS == 0) For lube pumps with internal timers.
;
; When using this method, P179 should be set such that MMM is a
; value that is greater than the cycle time set on the internal timers and
; SS should be set to zero. How much greater MMM needs to be depends on the
; accuracy of the lube pump timers, but it is better to be on the long side
; to ensure proper operation.
;
; Example 1. The internal lube cycle interval is set to 60 minutes.
;           Set P179 = 7500. In this example, as long as the accuracy
;           of the lube timer interval causes the lube to turn on
;           within 75 minutes, it will work. Note that the amount of time
;           that lube is output is usually set with another timer control
;           on the lube pump and it does not factor into the setting of P179.
;
; It should be noted that lube pumps with internal timers may differ on how
; they operate.
;
; (a) For pumps that lube immediately when power is applied and then start timing
; until the next cycle, it is possible to run out of lube quickly on short job
; runs if, after the program has been run, lube power is removed.
;
; (b) For pumps that do not lube until it has been turned on for the interval time,
; it is possible that lube never gets applied if, after the short program has been run,
; lube power is removed.
;
; A short program or job run is defined as a job that finishes before
; the interval setting (60 minutes in the above example).
;

```

```
; For the above mentioned reasons, we want the power to be applied for at least
; the amount of time set by the interval timer, noting that if the user decides
; to engage the E-stop to remove power after short jobs, then they risk the
; above mentioned problems according to the type of pump.
```

```
;
; On the start of SV_PROGRAM_RUNNING, the lube pump turns on.
; The lube pump is turned off when a program has NOT been
; running continuously for MMM minutes or E-stop is engaged.
; The reason the lube pump is turned off after a program has NOT been
; running for MMM minutes is to prevent lubing when the user leaves for the
; weekend, leaving the machine on and E-stop disengaged.
```

```
IF (SV_PROGRAM_RUNNING || SV_MDI_MODE) THEN SET Lube, RST LubeM_T
IF !(SV_PROGRAM_RUNNING || SV_MDI_MODE) THEN LubeM_T = LubeM_W, SET LubeM_T
IF LubeM_T || !EStopOk THEN RST Lube
```

```
;=====
```

LubeUsePLCTimersStage

```
;=====
```

```
; METHOD 2 (SS != 0) For lube pumps that do not have internal timers.
```

```
;
; When using this method P179 should be set so the lube turns on
; every MMM minutes for SS seconds.
```

```
; Example 1.
```

```
; To set the lube pump power to come on for 5 seconds
; every 10 minutes, set P179 = 1005.
```

```
MMMSS
```

```
; Example 2.
```

```
; To set the lube pump power to come on for 30 seconds
; every 2 hours, set P179 = 12030
```

```
MMMSS
```

```
;
; This method will accumulate time while a program is running until
; it reaches MMM minutes, at which time it will apply power
; for SS seconds (unless E-stop is engaged) and then start over. It is
; possible with frequent use of E-stop that a lube cycle is cut short.
```

```
IF (SV_PROGRAM_RUNNING || SV_MDI_MODE) THEN LubeM_T = LubeM_W, SET LubeM_T
IF !(SV_PROGRAM_RUNNING || SV_MDI_MODE) THEN (StopRunningPD)
IF StopRunningPD THEN LubeAccumTime_W = LubeAccumTime_W + LubeM_T, RST LubeM_T
IF LubeM_T || (LubeAccumTime_W + LubeM_T > LubeM_W)
THEN SET Lube, LubeS_T = LubeS_W, SET LubeS_T, RST LubeM_T, LubeAccumTime_W = 0
IF LubeS_T || !EStopOk THEN RST Lube, RST LubeS_T
```

```
;=====
```

KeyboardEventsStage

```
;=====
```

```
; This stage handles functions that are required for menu navigation
; by CNC11, require multiple keypresses and/or need to be interlocked
; with SV_PC_VIRTUAL_JOGPANEL_ACTIVE and/or AllowKbInput_M. Regarding
; "AllowKbInput_M": This PLC program reads a bit from a system parameter,
; in this case bit 0 of SV_MACHINE_PARAMETER_170, and sets "AllowKbInput_M"
; if the bit is a "0". If the operator wishes to allow keyboard input
; to trigger PLC events, they must set parameter 170 to a "1"
```

```

; (or any odd number for that matter). It should be mentioned that
; the programmer will not want to interlock all keyboard keys with
; SV_PC_VIRTUAL_JOGPANEL_ACTIVE and/or AllowKbInput_M. For example:
; The "escape" key must be echoed by the PLC to CNC11 to aid in menu
; navigation. NOTE: For backward compatibility with CNC10, setting bit 1
; of SV_MACHINE_PARAMETER_148 OR clearing bit 0 of SV_MACHINE_PARAMETER_170
; will disable keyboard jogging.

;-----Not interlocked-----
; The code for cycle cancel has been moved to the main stage.
; It is commented out below but remains for reference
;Cycle Cancel
;if Kb_Escape THEN (KbCycleCancel_M)

;Rapidoverride: Ctrl-r
if Kb_r && (Kb_L_Ctrl || Kb_R_Ctrl) THEN (KbTogRapidOver_M)

;-----Interlocked with AllowKbInput_M-----
;KbCycle Start: alt-s
if Kb_s && (Kb_R_Alt || Kb_L_Alt) && AllowKbInput_M then (KbCycleStart_M)

;KbToolCheck_M: Ctrl-t
if Kb_t && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M then (KbToolCheck_M)

;KbTogSingleBlock_M: ctrl-b
if Kb_b && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M then (KbTogSingleBlock_M)

;KbTogSpinAutoMan_M: ctrl-a
if Kb_a && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M then (KbTogSpinAutoMan_M)

;KbSpinCW_M: ctrl-c
IF Kb_c && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M THEN SET KbSpinCW_M,
    RST KbSpinCCW_M

;KbSpinCCW_M: ctrl-w
IF Kb_w && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M THEN SET KbSpinCCW_M,
    RST KbSpinCW_M

;KbSpinStart_M: ctrl-s
if Kb_s && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M then (KbSpinStart_M)

;KbSpindle stop: Ctrl-q
if Kb_q && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M then (KbSpinStop_M)

;KbIncSpinOver_M: ctrl (>)
if Kb_Period && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M
    then (KbIncSpinOver_M)

;KbDecSpinOver_M: ctrl (<)
if Kb_Comma && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M
    then (KbDecSpinOver_M)

;KbSpinOver100_M: ctrl + /
IF Kb_Slash && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M
    THEN (KbSpinOver100_M)

;KbTogCoolAutoMan_M: Ctrl-m

```

```

if Kb_m && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M then (KbTogCoolAutoMan_M)

;KbFloodOnOff_M: Ctrl-n
if Kb_n && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M then (KbFloodOnOff_M)

;KbMistOnOff_M: Ctrl-k
if Kb_k && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M then (KbMistOnOff_M)

;KbTogIncContJog_M: "ctrl" + "i"
if Kb_i && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M then (KbTogIncContJog_M)

;KbTogFastSlowJog_M: "ctrl" + "f"
IF Kb_f && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M
  THEN (KbTogFastSlowJog_M)

;KbAux1Key_M: "ctrl" + "F1"
if Kb_F1 && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M then (KbAux1Key_M)

;KbAux2Key_M: "ctrl" + "F2"
if Kb_F2 && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M THEN (KbAux2Key_M)

;KbAux3Key_M: "ctrl" + "F3"
if Kb_F3 && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M THEN (KbAux3Key_M)

;KbAux4Key_M: "ctrl" + "F4"
if Kb_F4 && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M THEN (KbAux4Key_M)

;KbAux5Key_M: "ctrl" + "F5"
if Kb_F5 && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M THEN (KbAux5Key_M)

;KbAux6Key_M: "ctrl" + "F6"
if Kb_F6 && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M THEN (KbAux6Key_M)

;KbAux7Key_M: "ctrl" + "F7"
if Kb_F7 && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M THEN (KbAux7Key_M)

;KbAux8Key_M: "ctrl" + "F8"
if Kb_F8 && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M THEN (KbAux8Key_M)

;KbAux9Key_M: "ctrl" + "F9"
if Kb_F9 && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M THEN (KbAux9Key_M)

;KbAux10Key_M: "ctrl" + "F10"
if Kb_F10 && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M THEN (KbAux10Key_M)

;KbAux11Key_M: "ctrl" + "F11"
if Kb_F11 && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M THEN (KbAux11Key_M)

;KbAux12Key_M: "ctrl" + "F12"
if Kb_F12 && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M THEN (KbAux12Key_M)

;KbIncFeedOver_M: "ctrl" + "keyboard +" (actually "=")
IF Kb_Equals && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M
  THEN (KbIncFeedOver_M)

;KbDecFeedOver_M: "ctrl" + "keyboard -"
IF Kb_Hyphen && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M

```

```

THEN (KbDecFeedOver_M)

;KbFeedOver100_M: "ctrl" + "keyboard \"
IF Kb_Backslash && (Kb_L_Ctrl || Kb_R_Ctrl) && AllowKbInput_M
  THEN (KbFeedOver100_M)

;-----Interlocked with AllowKbInput_M && KbJpActive_M-----

;KbIncreaseJogInc_M: "insert"
if Kb_Ins && AllowKbInput_M && KbJpActive_M
  then (KbIncreaseJogInc_M)
if KbIncreaseJogInc_M && x1JogLED && !X1_M && !X10_M && !X100_M
  then set X10_M
if KbIncreaseJogInc_M && x10JogLED && !X1_M && !X10_M && !X100_M
  then set X100_M

;KbDecreaseJogInc_M: "delete"
if Kb_Del && AllowKbInput_M && KbJpActive_M
  then (KbDecreaseJogInc_M)
if KbDecreaseJogInc_M && x10JogLED && !X1_M && !X10_M && !X100_M
  then set X1_M
if KbDecreaseJogInc_M && x100JogLED && !X1_M && !X10_M && !X100_M
  then set X10_M

;KbJogAx1Plus_M: Right arrow
if Kb_Left && AllowKbInput_M && KbJpActive_M THEN (KbJogAx1Plus_M)

;KbJogAx1Minus_M: Right arrow
if Kb_Right && AllowKbInput_M && KbJpActive_M THEN (KbJogAx1Minus_M)

;KbJogAx2Plus_M: Up arrow
if Kb_Up && AllowKbInput_M && KbJpActive_M THEN (KbJogAx2Plus_M)

;KbJogAx1Minus_M: Down arrow
if Kb_Down && AllowKbInput_M && KbJpActive_M THEN (KbJogAx2Minus_M)

;KbJogAx3Plus_M: Page up
if Kb_PgUp && AllowKbInput_M && KbJpActive_M THEN (KbJogAx3Plus_M)

;KbJogAx3Minus_M: Page Down
if Kb_PgDown && AllowKbInput_M && KbJpActive_M THEN (KbJogAx3Minus_M)

;KbAx4PlusJog: "home"
if Kb_Home && AllowKbInput_M && KbJpActive_M
  then (KbJogAx4Plus_M)

;KbAx4MinusJog: "end"
if Kb_End && AllowKbInput_M && KbJpActive_M
  then (KbJogAx4Minus_M)

IF True THEN RST KeyboardEventsStage

;=====
;                               MPGStage
;=====

```

```

;                               MPG Functions
;       Turn on/off Jog Panel MPG LED & on the MPG
IF MPGKey then (MpgPD)
IF MpgPD && MPGLLED then set MPGManOffFlag_M
IF !SV_MPG_1_ENABLED || (MpgPD && !MPGLLED) then RST MPGManOffFlag_M

IF (MpgPD && !MPGLLED) || (SV_MPG_1_ENABLED && !MPGManOffFlag_M) &&
    !SV_PROGRAM_RUNNING THEN SET MPG_LED_OUT, SET MPGLLED

IF (!SV_MPG_1_ENABLED || (MpgPD && MPGLLED))
    || SV_PROGRAM_RUNNING THEN RST MPG_LED_OUT, RST MPGLLED

;x1, x10, x100 functions
;--X1
IF x1JogKey THEN (x1JogPD)
IF x1JogPD || OnAtPowerUp_M || X1_M || (MPG_Inc_X_1 && MPGLLED)
    THEN SET x1JogLED, RST x10JogLED, RST x100JogLED

;--X10
IF x10JogKey THEN (x10JogPD)
IF x10JogPD || X10_M || (MPG_Inc_X_10 && MPGLLED)
    THEN RST x1JogLED, SET x10JogLED, RST x100JogLED

;--X100
IF x100JogKey THEN (x100JogPD)
IF x100JogPD || X100_M || (MPG_Inc_X_100 && MPGLLED)
    THEN RST x1JogLED, RST x10JogLED, SET x100JogLED

if !KbIncreaseJogInc_M && !KbDecreaseJogInc_M then rst X1_M, rst X10_M,
    rst X100_M

;--MPG 1 Enable
IF MPG_AXIS_1 || MPG_AXIS_2 || MPG_AXIS_3 || MPG_AXIS_4 ||
    MPG_AXIS_5 || MPG_AXIS_6 || MPG_AXIS_7 || MPG_AXIS_8
    THEN (SV_MPG_1_ENABLED)

;                               Select axis to move
IF MPG_AXIS_1 THEN SV_MPG_1_AXIS_SELECT = 1
IF MPG_AXIS_2 THEN SV_MPG_1_AXIS_SELECT = 2
IF MPG_AXIS_3 THEN SV_MPG_1_AXIS_SELECT = 3
IF MPG_AXIS_4 THEN SV_MPG_1_AXIS_SELECT = 4
IF MPG_AXIS_5 THEN SV_MPG_1_AXIS_SELECT = 5

;                               Select MPG 1 Multiplier
IF (MPG_Inc_X_100) THEN SV_MPG_1_MULTIPLIER = 100
IF (MPG_Inc_X_10) THEN SV_MPG_1_MULTIPLIER = 10
IF (MPG_Inc_X_1) THEN SV_MPG_1_MULTIPLIER = 1

;                               Disable "Windup" mode IF x100 selected
IF (!MPG_Inc_X_100) THEN (SV_MPG_1_WINDUP_MODE)

;=====
;                               JogPanelStage
;=====
;--Select Incremental or Continuous Jog Mode
IF IncrContKey || KbTogIncContJog_M THEN (IncrContPD)

```

```

IF (IncrContPD && !IncrContLED) || OnAtPowerUp_M THEN SET IncrContLED
IF (IncrContPD && IncrContLED) THEN RST IncrContLED

;--Select Fast or Slow Jog Mode
IF FastSlowKey || KbTogFastSlowJog_M THEN (SlowFastPD)
IF (SlowFastPD && !FastSlowLED) || OnAtPowerUp_M
    THEN SET FastSlowLED
IF (SlowFastPD && FastSlowLED) THEN RST FastSlowLED

;--Single Block Mode

IF SingleBlockKey || KbTogSingleBlock_M THEN (SingleBlockPD)
IF SingleBlockPD && !SingleBlockLED && !SV_PROGRAM_RUNNING
    THEN SET SingleBlockLED
IF SingleBlockPD && SingleBlockLED THEN RST SingleBlockLED
IF SingleBlockLED THEN (SelectSingleBlock)

;--Toolcheck

IF (ToolCheckKey || KbToolCheck_M) && EstopOk THEN (ToolCheckPD)
IF ToolCheckPD THEN (DoToolCheck)

;--Feed Hold Mode
IF (FeedHoldKey || KbFeedHold_M) && SV_PROGRAM_RUNNING THEN (FeedHoldPD)
IF FeedHoldPD && !FeedHoldLED THEN SET FeedHoldLED
IF FeedHoldPD && FeedHoldLED && !SV_PROGRAM_RUNNING && !SV_MDI_MODE
    THEN RST FeedHoldLED
IF FeedHoldLED && (DoCycleStart || DoCycleCancel || ToolCheckPD)
    THEN RST FeedHoldLED
; (FeedHoldLED will be used later to signal MPU11 to do Feed Hold)

;--Feedrate Override Section
;-----
; Feedrate override works as follows:
;
; 1. The PLC reads the 8 bit value of the FeedrateKnob_W directly (0-255)
; 2. The PLC scales this value to a 0-200 value (0-200%) by dividing by
;    the knob value by 127.5 and then multiplying the result by 100
; 3. If keyboard joggin is not disabled (it is enabled by default), the PLC
;    determines whether the operator is using the keyboard override or
;    the FeedrateKnob_W to override the feedrate by watching which was changed
;    most recently. The most recently changed value is saved as
;    "FinalFeedOverride_W"
; 4. Parameter 39 in (From the "params" screen in CNC11 software) stores
;    a value which allows which the PLC program can use to limit the amount
;    of override applied to the programmed feedrate. This value is specified
;    as a percentage.
; 5. The PLC limits the override percentage by reading parameter 39 and, if
;    the feedrate override percentage as read from the knob is greater than
;    parameter 39, it sets the FinalFeedOverride_W value to the value of
;    parameter 39.
; 6. Once the override percentage has been determined and limited (if needed)
;    The PLC send this value up to the CNC11 software by setting
;    SV_PLC_FeedrateKnob_W = FinalFeedOverride_W
; 7. CNC11 reads SV_PLC_FEEDRATE_KNOB, factors in it's on own override based
;    on parameter 78 (see operators manual for more info on parm 78) and then
;    returns an override value to the PLC in the system variable

```

```

; SV_PC_FEEDRATE_PERCENTAGE
; 8. The PLC reads SV_PC_FEEDRATE_PERCENTAGE and (typically) echoes the system
; variable to SV_PLC_FEEDRATE_OVERRIDE which the MPU11 uses as the final
; determination of the feedrate override percentage.
;-----
; 1. The PLC reads the 8 bit value of the FeedrateKnob_W directly (0-255)
; NOTE: BTW = Bit To Word
; BTW reads the specified number of bits (if none is specified it defaults to 8)
; starting from a bit location and writes them to a word with the starting bit
; location being written to the LSB of the word used. Below, BTW reads the bit
; values from JpFeedOrKnobBit0 to JpFeedOrKnobBit7 and writes them into to the
; word "FeedrateKnob_W" which sets FeedrateKnob_W to a value of 0-255
;-----
IF true THEN FeedrateKnob_W = 0
if true THEN BTW FeedrateKnob_W JpFeedOrKnobBit0 8

;-----
; 2. Scale this value to a 0-200 value (0-200%)
;-----
IF true THEN FeedrateKnob_W = (FeedrateKnob_W/127.5)*100

;-----
; 3. Determine whether to use FeedrateKnob_W or KbOverride_W
;-----
; This section determines when to use the feedrate override value sent down
; by the jogpanel (FeedrateKnob_W) or the feedrate override as determined
; by the PLC monitoring the keyboard override keys (KbOverride_W).

;-----
; At powerup, default feedrate override is jog panel (FeedrateKnob_W)
; To use both keyboard or jogpanel overrides set p170 to 0 (default)
; To use jogpanel override only set p170 to 2
; To use keyboard only set p170 to 4
;-----
IF OnAtPowerUp_M && KbOverOnly_M || KbFeedOver100_M THEN KbOverride_W = 100
IF OnAtPowerUp_M && !KbOverOnly_M THEN set UsingFeedrateKnob_M,
                                KbOverride_W = FeedrateKnob_W,
                                Last_FeedrateKnob_W = FeedrateKnob_W

;-----Calculate keyboard feedrate override-----
; SleepTimer is used to limit the KbOverride_W update rate to 20% per sec
;-----
if AllowKbInput_M && KbIncFeedOver_M && !WaitingForSleepTimer_M
    THEN KbOverride_W = KbOverride_W + 1,
         RST UsingFeedrateKnob_M,
         SET WaitingForSleepTimer_M,
         SleepTimer = 50, SET SleepTimer

if AllowKbInput_M && KbDecFeedOver_M && !WaitingForSleepTimer_M
    THEN KbOverride_W = KbOverride_W - 1, rst UsingFeedrateKnob_M,
         set WaitingForSleepTimer_M, SleepTimer = 50, set SleepTimer

if SleepTimer THEN rst WaitingForSleepTimer_M, rst SleepTimer

;-----Switch to FeedrateKnob_W if it changes more than 3%-----
; Once it has changed by more than 3%, it will update as normal (1% increments)
; until it sees another KbOverride_W command at which point it will take

```

```

; another 3% change to re-activate the FeedrateKnob_W

if (abs>Last_FeedrateKnob_W - FeedrateKnob_W) >= 3) || UsingFeedrateKnob_M
    THEN FinalFeedOverride_W = FeedrateKnob_W, KbOverride_W = FeedrateKnob_W,
    Last_FeedrateKnob_W = FeedrateKnob_W, set UsingFeedrateKnob_M

;Limit keyboard override to parm 39. Allowing the FeedrateKnob_W to go past
;parm 39, but keeping the KbOverride_W limited keeps the "dead space"
;down and allows the PLC to respond to changes in the FeedrateKnob_W even if
;above 120. Overall override is still limited later but this gives better
;response in changing between KbOverride_W & the FeedrateKnob_W
if KbOverride_W > SV_MACHINE_PARAMETER_39
    THEN KbOverride_W = SV_MACHINE_PARAMETER_39

if !UsingFeedrateKnob_M && !JogOverOnly_M
    THEN FinalFeedOverride_W = KbOverride_W

;-----
; 4 & 5. Limit override percentage to value set in Parameter 39
;-----
;-----Limit final override percentage to parm 39-----
if FinalFeedOverride_W > SV_MACHINE_PARAMETER_39
    THEN FinalFeedOverride_W = SV_MACHINE_PARAMETER_39

if FinalFeedOverride_W < 0 THEN FinalFeedOverride_W = 0

;-----
; Override Controls
; It is important that the plc program only writes to SV_PLC_Feedrate_Knob once per pass
;-----
; Override control bit for the feedrate override
; 1 == feedrate override knob will effect feedrate
; 0 == override knob has NO effect on feedrate
IF !SV_PC_OVERRIDE_CONTROL_FEEDRATE_OVERRIDE THEN FinalFeedOverride_W = 100

;-----
; 6. Send override percentage to CNC11
;-----
;-----Send override to PC for modification if needed-----
if true THEN SV_PLC_Feedrate_Knob = FinalFeedOverride_W

;-----
; 7. Copy the feedrate override sent from the PC to the MPU11.
;-----
;-----
; Normally a number from 0.0-2.0, no limitations although V will not exceed
; Vmax. A negative number in here would be extremely bad.
;-----
IF true THEN SV_PLC_FEEDRATE_OVERRIDE = SV_PC_FEEDRATE_PERCENTAGE/100.0

;--MPU11 Jog Panel Key Functions
IF KB_F9 then (F9PD)
IF KbTogRapidOver_M || (F9PD && (SV_PROGRAM_RUNNING || SV_MDI_MODE))
    THEN (RapidOverPD)
IF RapidOverPD^ SelectRapidOverride THEN (SelectRapidOverride)
IF OnAtPowerUp_M THEN SET SelectRapidOverride
IF (CycleCancelKey || KbCycleCancel_M) && (SV_PROGRAM_RUNNING || SV_MDI_MODE)

```

```

THEN (DoCycleCancel)
IF (CycleStartKey || KbCycleStart_M) THEN (DoCycleStart)

IF (Ax1PlusJogKey || KbJogAx1Plus_M) && !Ax1PlusJogDisabled_M &&
  !(IncrContLED && FinalFeedOverride_W == 0) THEN (DoAx1PlusJog)
IF (Ax1MinusJogKey || KbJogAx1Minus_M) && !Ax1MinusJogDisabled_M &&
  !(IncrContLED && FinalFeedOverride_W == 0) THEN (DoAx1MinusJog)
IF (Ax2PlusJogKey || KbJogAx2Plus_M) && !Ax2PlusJogDisabled_M &&
  !(IncrContLED && FinalFeedOverride_W == 0) THEN (DoAx2PlusJog)
IF (Ax2MinusJogKey || KbJogAx2Minus_M) && !Ax2MinusJogDisabled_M &&
  !(IncrContLED && FinalFeedOverride_W == 0) THEN (DoAx2MinusJog)

IF (Ax3PlusJogKey || KbJogAx3Plus_M) &&
  !(IncrContLED && FinalFeedOverride_W == 0) THEN (DoAx3PlusJog)
IF (Ax3MinusJogKey || KbJogAx3Minus_M) &&
  !(IncrContLED && FinalFeedOverride_W == 0) THEN (DoAx3MinusJog)
IF (Ax4PlusJogKey || KbJogAx4Plus_M) &&
  !(IncrContLED && FinalFeedOverride_W == 0) THEN (DoAx4PlusJog)
IF (Ax4MinusJogKey || KbJogAx4Minus_M) &&
  !(IncrContLED && FinalFeedOverride_W == 0) THEN (DoAx4MinusJog)

IF (Aux1Key || KbAux1Key_M) THEN (DoAux1Key)
IF (Aux2Key || KbAux2Key_M) THEN (DoAux2Key)
IF (Aux3Key || KbAux3Key_M) THEN (DoAux3Key)
IF (Aux4Key || KbAux4Key_M) THEN (DoAux4Key)
IF (Aux5Key || KbAux5Key_M) THEN (DoAux5Key)
IF (Aux6Key || KbAux6Key_M) THEN (DoAux6Key)
IF (Aux7Key || KbAux7Key_M) THEN (DoAux7Key)
IF (Aux8Key || KbAux8Key_M) THEN (DoAux8Key)
IF (Aux9Key || KbAux9Key_M) THEN (DoAux9Key)
IF (Aux10Key || KbAux10Key_M) THEN (DoAux10Key)
IF (Aux11Key || KbAux11Key_M) THEN (DoAux11Key)
IF (Aux12Key || KbAux12Key_M) THEN (DoAux12Key)
IF x1JogLED THEN (SelectX1JogInc)
IF x10JogLED THEN (SelectX10JogInc)
IF x100JogLED THEN (SelectX100JogInc)
IF IncrContLED THEN (SelectIncrContJog)
IF FastSlowLED THEN (SelectFastSlowJog)
IF MPGLLED THEN (SelectMpgMode)
IF FeedHoldLED THEN (DoFeedHold)

;--Coolant Functions

;--Toggle auto coolant mode
IF CoolAutoManKey || KbTogCoolAutoMan_M THEN (CoolantAutoManualPD)

IF (!CoolAutoManLED && CoolantAutoManualPD) || OnAtPowerUp_M
  THEN SET CoolAutoManLED

IF (CoolAutoManLED && CoolantAutoManualPD)
  THEN RST CoolAutoManLED

;--Report coolant mode to CNC11
IF CoolAutoManLED THEN (SelectCoolAutoMan)

;--Display coolant mode message
;changing to auto coolant mode ;9050 Auto Coolant Selected 2 + 50*256

```

```

IF (!CoolAutoManLED && CoolantAutoManualPD)
  THEN AsyncMsg_W = 12802, MSG AsyncMsg_W

;changing to manual coolant mode ;9051 Manual Coolant Selected 2 + 51*256
IF (CoolAutoManLED && CoolantAutoManualPD)
  THEN AsyncMsg_W = 13058, MSG AsyncMsg_W

;--Flood coolant on/off
IF ((CoolFloodKey || KbFloodOnOff_M) && !CoolAutoManLED) ||
  (M8 && CoolAutoManLED) || (DoCycleStart && M8 && CoolAutoManLED)
  THEN (CoolantFloodPD)

IF CoolantFloodPD && !CoolFloodLED Then SET CoolFloodLED, Set Flood

IF SV_STOP || (CoolantFloodPD && CoolFloodLED) || (!M8 && CoolAutoManLED) ||
  (M8 && !CoolAutoManLED) || DoToolCheck THEN Rst Flood, Rst CoolFloodLED

IF CoolFloodLED THEN (SelectCoolantFlood)

;--Mist coolant on/off
IF ((CoolMistKey || KbMistOnOff_M)&& !CoolAutoManLED) || (M7 && CoolAutoManLED)
  || (DoCycleStart && M7 && CoolAutoManLED) THEN (CoolantMistPD)

IF (CoolantMistPD && !CoolMistLED) THEN SET Mist, SET CoolMistLED

IF SV_STOP || (CoolantMistPD && CoolMistLED) || (!M7 && CoolAutoManLED) ||
  (M7 && !CoolAutoManLED) || DoToolCheck THEN Rst Mist, Rst CoolMistLED

IF CoolMistLED THEN (SelectCoolantMist)

;--Spindle Control
;-----
; JOGBOARD SPINDLE CONTROL
; Spindle Auto Mode / Manual mode toggles via Auto/Man jog panel key
; CW/CCW jog keys determine spindle direction in manual mode
; M3/M4 system variables determine spindle direction in Auto mode
; Spindle can be stopped and restarted in auto mode using
; spin stop/start jog keys
;-----
;--Select Auto or Manual Spindle Operation Mode
;Triggers to Toggle Auto/Manual Spindle Mode
IF SpinAutoManKey || KbTogSpinAutoMan_M THEN (SpinAutoManPD)

;--Set spindle to auto mode on startup
IF (SpinAutoManPD && !SpinAutoModeLED) || OnAtPowerUp_M
  THEN SET SpinAutoModeLED

;--Set spindle to manual mode
if SpinAutoManPD && SpinAutoModeLED THEN rst SpinAutoModeLED

;--Report the Spindle mode to CNC11
IF SpinAutoModeLED THEN (SelectManAutoSpindle)

;--Set triggers to start and stop the spindle
; NOTE: SpindlePause_M allows the operator to start and stop the
; spindle with the spin start and stop keys while in a job. In
; this case, pressing the spindle start key will only restart

```

```

; the spindle if an M3 or M4 had previously been issued and is
; still active.

IF ((SpinStartKey || KbSpinStart_M) && !SpinAutoModeLED) ||
  (SpinAutoModeLED && (M3 || M4) && !SpindlePause_M) ||
  ((SpinStartKey || KbSpinStart_M) && ((M3 || M4) && SpinAutoModeLED))
  THEN (SpinStartPD), Rst SpindlePause_M

If (SpinAutoModeLED && (M3 || M4) && (SpinStopKey || KbSpinStop_M))
  THEN set SpindlePause_M

If (SpinStopKey || KbSpinStop_M) || (SpinAutoModeLED && !M3 && !M4) ||
  (SpinAutoManPD && SpindleEnableOut) || (SV_PC_RIGID_TAP_SPINDLE_OFF &&
  SpinAutoModeLED) THEN (SpinStopPD)

;--Adjust spindle override when entering manual or auto spin mode
;Set the override value to 100% when spin auto mode is first selected
IF SpinAutoManPD && !SpinAutoModeLED
  THEN SV_PLC_SPINDLE_KNOB = 100,
  SET SpinAutoModeLED

;Set the override value to 10% whenever manual mode is entered
IF SpinAutoManPD && SpinAutoModeLED
  THEN SV_PLC_SPINDLE_KNOB = 10, Rst SpinAutoModeLED

;--Set spindle direction
;-----Set Clockwise direction
IF ((KbSpinCW_M || SpinCWKey) && !SpinAutoModeLED) || (M3 && SpinAutoModeLED)
  || (M3 && DoCycleStart) then (SpinCWPD)
IF SpinCWPD then rst SpindleDirectionOut
IF !SpindleDirectionOut then (SpindleCWLED), (SelectSpindleCW)

;-----Set Counterclockwise direction
IF ((KbSpinCCW_M || SpinCCWKey) && !SpinAutoModeLED) || (M4 && SpinAutoModeLED)
  || (M4 && DoCycleStart) then (SpinCCWPD)
IF SpinCCWPD then set SpindleDirectionOut
IF SpindleDirectionOut then (SpindleCCWLED), (SelectSpindleCCW)

;-----
;                                     Turn spindle on/off
;-----
IF ProbeDetect && SpinStartPD THEN set ProbeFault_M

IF (SpindleEnableOut || SpinStartPD) &&
  !(SpinStopPD || SV_STOP || ProbeDetect)
  THEN (SpindleEnableOut)

IF !SpindleEnableOut THEN (DoSpindleStop)

;-----
;          SPINDLE OVERRIDE CONTROL
;          Jogboard (-, +, and 100% keys),
;          Keyboard "ctrl" + "<", "ctrl" + ">", "ctrl" + "/"
;-----
IF SpinOverPlusKey || KbIncSpinOver_M
  THEN SV_PLC_SPINDLE_KNOB = SV_PLC_SPINDLE_KNOB + 1
IF SpinOverMinusKey || KbDecSpinOver_M

```

```

    THEN SV_PLC_SPINDLE_KNOB = SV_PLC_SPINDLE_KNOB - 1
IF SpinOver100Key || KbSpinOver100_M || OnAtPowerUp_M
    THEN SV_PLC_SPINDLE_KNOB = 100

IF SV_PLC_SPINDLE_KNOB < 1 THEN SV_PLC_SPINDLE_KNOB = 1
IF SV_PLC_SPINDLE_KNOB > 200 THEN SV_PLC_SPINDLE_KNOB = 200

IF SV_PLC_SPINDLE_KNOB == 100 THEN
    (SpinOver100LED),
    (SelectSpinOr100)

IF SV_PLC_SPINDLE_KNOB < 100 THEN
    (SpinOverMinusLED),
    (DoDecreaseSpindleOr )

IF SV_PLC_SPINDLE_KNOB > 100 THEN
    (SpinOverPlusLED),
    (DoIncreaseSpindleOr)

;--Output 12-bit DAC value for spindle control
;-----
;           Read spindle range inputs and/or range M codes
;
; NOTE: SV_SPINDLE_LOW_RANGE & SV_SPINDLE_MID_RANGE M are used to report the
; selected spindle range to CNC11
;-----
;   hi      med-high  med-low  low
;   0        1        1      0 SV_SPINDLE_MID_RANGE M
;   0        0        1      1 SV_SPINDLE_LOW_RANGE

If SpinLowRange then set SpinLowRange_M
If !SpinLowRange then rst SpinLowRange_M

IF !SpinLowRange THEN (SpinHighRange_M),
                      RST SV_SPINDLE_LOW_RANGE,
                      rst SV_SPINDLE_MID_RANGE

if SpinLowRange_M && (SV_MACHINE_PARAMETER_65 > .01)
    THEN SpinRangeAdjust = SV_MACHINE_PARAMETER_65,
    SET SV_SPINDLE_LOW_RANGE,
    rst SV_SPINDLE_MID_RANGE

if SpinHighRange_M then SpinRangeAdjust = 1
if SpinRangeAdjust == 0 then SpinRangeAdjust = 1

;-----
;           Read commanded spindle speed, max & min
;
; ***NOTE*** SV_PC_CMDMANDED_SPINDLE_SPEED already has override
; factored in.
;-----
IF True THEN SpinSpeedCommand_FW = SV_PC_CMDMANDED_SPINDLE_SPEED,
          CfgMinSpeed_FW = SV_PC_CONFIG_MIN_SPINDLE_SPEED,
          CfgMaxSpeed_FW = SV_PC_CONFIG_MAX_SPINDLE_SPEED

;-----
; If commanded spindle speed is < Min Spin Speed * SpinRangeAdjust

```

```

; & commanded spindle speed > 0, force to commanded spindle speed
; = min spin speed value * SpinRangeAdjust.
;-----
IF (SpinSpeedCommand_FW > 0.0) &&
  (SpinSpeedCommand_FW < (CfgMinSpeed_FW * SpinRangeAdjust))
  THEN SpinSpeedCommand_FW = (CfgMinSpeed_FW * SpinRangeAdjust),
  ErrorCode_W = MIN_SPEED_MSG

;-----
; If SpinSpeedCommand_FW > Max Spin Speed * SpinRangeAdjust, force
; SpinSpeedCommand_FW = max spin speed value * SpinRangeAdjust.
;-----
IF SpinSpeedCommand_FW > (CfgMaxSpeed_FW * SpinRangeAdjust)
  THEN SpinSpeedCommand_FW = (CfgMaxSpeed_FW * SpinRangeAdjust)

;-----
; Convert Spindle "S" command to 12 bit value for output to DAC
;-----
; Commanded Spindle speed (includes override factor) is sent down from CNC11
; in SV_PC_CMDDED_SPINDLE_SPEED. This value needs to be converted to a
; 12 bit value (0-4095) where full scale = SV_PC_CONFIG_MAX_SPINDLE_SPEED.

; Calculate #RPM's per bit of resolution
IF CfgMaxSpeed_FW > 0.0 THEN RPMPerBit_FW = CfgMaxSpeed_FW/4095.0
IF CfgMaxSpeed_FW <= 0.0 THEN RPMPerBit_FW = 1.0

;Convert RPM to 12 bit value
IF True THEN TwelveBitSpeed_FW = SpinSpeedCommand_FW/RPMPerBit_FW

; Factor in gear range
IF true THEN TwelveBitSpeed_FW = (TwelveBitSpeed_FW/SpinRangeAdjust)

;Convert to integer word for DAC & I/O display
if true then TwelveBitSpeed_W = TwelveBitSpeed_FW

; Bound min to 0, max to 4095
IF TwelveBitSpeed_W < 0 THEN TwelveBitSpeed_W = 0
IF TwelveBitSpeed_W > 4095 THEN TwelveBitSpeed_W = 4095

; Output to DAC
If true then WTB TwelveBitSpeed_W SpinAnalogOutBit0 12

;Display calculated RPM value on PC
IF True THEN SV_PLC_SPINDLE_SPEED = SpinSpeedCommand_FW

;=====
; CheckCycloneStatusStage
;=====
; Due to amount of time it takes to retrieve data from the cyclone, this stage
; is only called few times per second to help reduce scan time of the main PLC
; program.

; The logic below is the equivalent to the following:
; IF true THEN BITTST SV_PC_CYCLONE_STATUS_2 0 Axis1FiberOk_M,
;                   BITTST SV_PC_CYCLONE_STATUS_2 1 Axis2FiberOk_M,
;                   BITTST SV_PC_CYCLONE_STATUS_2 2 Axis3FiberOk_M,
;                   BITTST SV_PC_CYCLONE_STATUS_2 3 Axis4FiberOk_M,

```

```

;          BITTST SV_PC_CYCLONE_STATUS_2 4 Axis5FiberOk_M,
;          BITTST SV_PC_CYCLONE_STATUS_2 5 Axis6FiberOk_M,
;          BITTST SV_PC_CYCLONE_STATUS_2 6 Axis7FiberOk_M,
;          BITTST SV_PC_CYCLONE_STATUS_2 7 Axis8FiberOk_M
IF true THEN WTB SV_PC_CYCLONE_STATUS_2 Axis1FiberOk_M

; Generate some messages for fiber or wire to MPU11 having issues
IF SV_AXIS_VALID_1 && !SV_DRIVE_ONLINE_1 THEN ErrorCode_W = AXIS1_INFLT, SET
DriveComFltIn_M
IF SV_AXIS_VALID_2 && !SV_DRIVE_ONLINE_2 THEN ErrorCode_W = AXIS2_INFLT, SET
DriveComFltIn_M
IF SV_AXIS_VALID_3 && !SV_DRIVE_ONLINE_3 THEN ErrorCode_W = AXIS3_INFLT, SET
DriveComFltIn_M
IF SV_AXIS_VALID_4 && !SV_DRIVE_ONLINE_4 THEN ErrorCode_W = AXIS4_INFLT, SET
DriveComFltIn_M
IF SV_AXIS_VALID_5 && !SV_DRIVE_ONLINE_5 THEN ErrorCode_W = AXIS5_INFLT, SET
DriveComFltIn_M
IF SV_AXIS_VALID_6 && !SV_DRIVE_ONLINE_6 THEN ErrorCode_W = AXIS6_INFLT, SET
DriveComFltIn_M
IF SV_AXIS_VALID_7 && !SV_DRIVE_ONLINE_7 THEN ErrorCode_W = AXIS7_INFLT, SET
DriveComFltIn_M
IF SV_AXIS_VALID_8 && !SV_DRIVE_ONLINE_8 THEN ErrorCode_W = AXIS8_INFLT, SET
DriveComFltIn_M

; Generate some messages for fiber or wire to drive having issues
IF SV_AXIS_VALID_1 && SV_DRIVE_ONLINE_1 && SV_MASTER_ENABLE && !Axis1FiberOk_M
THEN ErrorCode_W = AXIS1_OUTFLT, set DriveComFltOut_M
IF SV_AXIS_VALID_2 && SV_DRIVE_ONLINE_2 && SV_MASTER_ENABLE && !Axis2FiberOk_M
THEN ErrorCode_W = AXIS2_OUTFLT, set DriveComFltOut_M
IF SV_AXIS_VALID_3 && SV_DRIVE_ONLINE_3 && SV_MASTER_ENABLE && !Axis3FiberOk_M
THEN ErrorCode_W = AXIS3_OUTFLT, set DriveComFltOut_M
IF SV_AXIS_VALID_4 && SV_DRIVE_ONLINE_4 && SV_MASTER_ENABLE && !Axis4FiberOk_M
THEN ErrorCode_W = AXIS4_OUTFLT, set DriveComFltOut_M
IF SV_AXIS_VALID_5 && SV_DRIVE_ONLINE_5 && SV_MASTER_ENABLE && !Axis5FiberOk_M
THEN ErrorCode_W = AXIS5_OUTFLT, set DriveComFltOut_M
IF SV_AXIS_VALID_6 && SV_DRIVE_ONLINE_6 && SV_MASTER_ENABLE && !Axis6FiberOk_M
THEN ErrorCode_W = AXIS6_OUTFLT, set DriveComFltOut_M
IF SV_AXIS_VALID_7 && SV_DRIVE_ONLINE_7 && SV_MASTER_ENABLE && !Axis7FiberOk_M
THEN ErrorCode_W = AXIS7_OUTFLT, set DriveComFltOut_M
IF SV_AXIS_VALID_8 && SV_DRIVE_ONLINE_8 && SV_MASTER_ENABLE && !Axis8FiberOk_M
THEN ErrorCode_W = AXIS8_OUTFLT, set DriveComFltOut_M

If !EstopOk THEN rst DriveComFltIn_M, rst DriveComFltOut_M
If DriveComFltOut_M || DriveComFltIn_M THEN set AxisFault_M

;check PLC status bit
IF TRUE THEN BitTst SV_PC_CYCLONE_STATUS_1 21 PLCBusExtDevEn_M

;check input fiber
IF !SV_PLC_BUS_ONLINE THEN ErrorCode_W = PLC_INFLT,
rst PLCBus_Oe_M, set PLCFault_M

;check output fiber
IF SV_PLC_BUS_ONLINE && PLCBus_Oe_M && !PLCBusExtDevEn_M
THEN ErrorCode_W = PLC_OUTFLT, SET PLCFault_M

;clear PLC errors

```

```

IF PLCFault_M && SV_PLC_BUS_ONLINE && PLCBusExtDevEn_M && !EstopOk
  THEN RST PLCFault_M, ErrorCode_W = PLC_FLT_CLR, SET PLCBus_Oe_M

IF True THEN RST CheckCycloneStatusStage

;=====
;                               AxesEnableStage
;=====
; Since CNC11 v3.03r24, the MPU11 has managed axis enables
; directly. The PLC program has no further responsibility
; for SV_ENABLE_AXIS_n

;clear axis fault errors
IF AxisFault_M && !(DriveComFltOut_M || DriveComFltIn_M) && !EstopOk
  THEN ErrorCode_W = AXIS_FLT_CLR, RST AxisFault_M

;read the status bits
IF True THEN SET CycloneStatus_T
IF CycloneStatus_T THEN SET CheckCycloneStatusStage, RST CycloneStatus_T

;turn on drives if no drive or drive fiber errors
IF !DriveComFltIn_M && !DriveComFltOut_M THEN SET SV_MASTER_ENABLE
; (SV_MASTER_ENABLE will be turned off by stalls and other
; errors in the Fault-handling section of MainStage, below.)

;(If there is any drive or drive fiber error, then AxisFault_M will have
; been set previously, which will cause SV_STOP to be set, and SV_MASTER_ENABLE
; to be reset, later in MainStage)

;=====
;                               MainStage
;=====
;Do gather if commanded (uncomment and recompile for debugging purposes)
;IF Aux11Key THEN (Aux11KeyPD)
;If Aux11KeyPD THEN (SV_TRIGGER_PLOT_DUMP)

;read parameter 178 and check the Lube NO/NC state

; Get the previous value of the debounce configuration word
IF True THEN Inputs_9_12_W = SV_PLC_DEBOUNCE_3

; Invert input 9
IF InvLubeOk_M THEN BITSET Inputs_9_12_W 6
IF !InvLubeOk_M THEN BITRST Inputs_9_12_W 6

; Invert input 10
IF InvSpinInverterOk_M THEN BITSET Inputs_9_12_W 14
IF !InvSpinInverterOk_M THEN BITRST Inputs_9_12_W 14

; Write back to the debounce configuration word
IF true THEN SV_PLC_DEBOUNCE_3 = Inputs_9_12_W

;-----
;                               Probe protection while jogging
;-----

```

```

If MechanicalProbe && !JogProbeFault_M && (DoAx1PlusJog || DoAx1MinusJog ||
DoAx2PlusJog || DoAx2MinusJog || DoAx3PlusJog || DoAx3MinusJog ||
DoAx4PlusJog || DoAx4MinusJog || DoAx5PlusJog || DoAx5MinusJog)
THEN (JogProbeFaultPD)

IF MechanicalProbe && !JogProbeFault_M && FastSlowLED THEN SET LastProbeMode_M
IF MechanicalProbe && !JogProbeFault_M && !FastSlowLED THEN RST LastProbeMode_M

IF JogProbeFaultPD && !JogProbeFault_M THEN SET JogProbeFault_M, SET DoCycleCancel

IF JogProbeFault_M THEN ErrorCode_W = (PROBE_JOG_FAULT_MSG + 1),
SET FastSlowLED

IF !MechanicalProbe && JogProbeFault_M && !LastProbeMode_M THEN RST FastSlowLED

IF !MechanicalProbe THEN RST JogProbeFault_M,
RST Ax1PlusJogDisabled_M,
RST Ax1MinusJogDisabled_M,
RST Ax2PlusJogDisabled_M,
RST Ax2MinusJogDisabled_M

;--Clamp
IF M10 THEN (Clamp) ; cleared by M11 or by program not running

;--Process important Keyboard keys all the time
;--Cycle Cancel (ESC)
if Kb_Escape THEN (KbCycleCancel_M)
;KbFeedHold_M (spacebar)
if Kb_spacebar && AllowKbInput_M && SV_PROGRAM_RUNNING then (KbFeedHold_M)

if SV_PC_VIRTUAL_JOGPANEL_ACTIVE THEN (KbJpActive_M)

;Call KeyboardEvents stage if needed
IF Kb_L_Ctrl || Kb_R_Ctrl ||
Kb_L_Shift || Kb_R_Shift ||
Kb_R_Alt || Kb_L_Alt || KbJpActive_M
THEN SET KeyboardEventsStage

if (Kb_L_Ctrl || Kb_R_Ctrl || Kb_L_Shift || Kb_R_Shift || Kb_R_Alt || Kb_L_Alt)
&& (Kb_j || Kb_f || kb_a || kb_s) && !AllowKbInput_M
THEN ErrorCode_W = KB_JOG_MSG

;--Handle Faults
if !EstopOk || PLCFault_M || SV_STALL_ERROR || SpindleFault_M ||
LubeFault_M || AxisFault_M || ProbeFault_M || OtherFault_M THEN set SV_STOP

IF SV_Stop THEN RST SV_MASTER_ENABLE

IF !EstopOk THEN RST SV_STALL_ERROR,
RST LubeFault_M,
RST SpindleFault_M,
RST OtherFault_M,
RST ProbeFault_M,
RST ProbeMsgSent_M

IF Initialize_T && !LubeOk && !SV_PROGRAM_RUNNING
THEN SET LubeFault_M, ErrorCode_W = LUBE_FAULT_MSG

```

```

IF !LubeOk && SV_PROGRAM_RUNNING THEN ErrorCode_W = LUBE_WARNING_MSG

IF Initialize_T && !SpindleInverterOk
    THEN ErrorCode_W = SPINDLE_FAULT_MSG, SET SpindleFault_M
IF !EstopOk && !SpindleInverterOk THEN (InverterResetOut)

; Echo some system variables to memory bits, for troubleshooting only
IF SV_MASTER_ENABLE THEN (MasterEnable_M)
IF SV_STALL_ERROR THEN (Stall_M)
IF SV_STOP THEN (Stop_M)

IF !SV_STOP THEN (NoFaultOut)

IF EStopOk &&
    !(PLCFault_M || SV_STALL_ERROR || SpindleFault_M || LubeFault_M ||
    AxisFault_M || OtherFault_M || SoftwareNotReady_M || PLCExecutorFault_M)
    THEN RST SV_STOP

IF ProbeFault_M && !ProbeMsgSent_M
    THEN ErrorCode_W = PROBE_FAULT_MSG, SET ProbeMsgSent_M

;--M-Codes
;   Reset these M-codes if not in CNC Program Running mode
IF !(SV_PROGRAM_RUNNING || SV_MDI_MODE) THEN RST M3, RST M4, RST M8, RST M7, RST M10

;--turn off default setup variable
IF true THEN RST OnAtPowerUp_M

;=====
    SetErrorStage
;=====
IF !((ErrorCode_W % 256 == 1) || (ErrorCode_W % 256 == 2)) THEN JMP BadErrorStage
IF true THEN MSG ErrorCode_W
If ErrorCode_W != MSG_CLEARED_MSG Then MsgClear_T = 1000, set MsgClear_T

IF (!EstopOk && !SoftwareNotReady_M) ||
    ((ErrorCode_W != MSG_CLEARED_MSG) && (ErrorCode_W % 2 == 0) && MsgClear_T)
    THEN ErrorCode_W = MSG_CLEARED_MSG, RST MsgClear_T

;=====
    BadErrorStage
;=====
IF true THEN AsyncMsg_W = 2+256*100, MSG AsyncMsg_W, AsyncMsg_W = 0
IF True THEN RST BadErrorStage

```

Appendix B: Jog Panel Mapping

JogPanel Inputs and Outputs

The shorthand reference for JogPanel Inputs and Outputs JPI1-JPI256 and JPO1-JPO256 are remapping of Inputs and Outputs 1057-1312. Note that JPI stands for Jog Panel Input and JPO stands for Jog Panel Output so the letters after JP are the letters 'eye' and 'oh' capitalized.

Spindle + Key JPI1/INP1057 LED JPO1/OUT1057	Spindle Auto/Man Key JPI2/INP1058 LED JPO2/OUT1058	Aux1 Key JPI3/INP1059 LED JPO3/OUT1059	Aux2 Key JPI4/INP1060 LED JPO4/OUT1060	Aux3 Key JPI5/INP1061 LED JPO5/OUT1061
Spindle 100% Key JPI6/INP1062 LED JPO6/OUT1062	Spindle CW Key JPI7/INP1063 LED JPO7/OUT1063	Aux4 Key JPI8/INP1064 LED JPO8/OUT1064	Aux5 Key JPI9/INP1065 LED JPO9/OUT1065	Aux6 Key JPI10/INP1066 LED JPO10/OUT1066
Spindle - Key JPI11/INP1067 LED JPO11/OUT1067	Spindle CCW Key JPI12/INP1068 LED JPO12/OUT1068	Aux7 Key JPI13/INP1069 LED JPO13/OUT1069	Aux8 Key JPI14/INP1070 LED JPO14/OUT1070	Aux9 Key JPI15/INP1071 LED JPO15/OUT1071
Spindle Stop Key JPI16/INP1072 LED JPO16/OUT1072	Spindle Start Key JPI17/INP1073 LED JPO17/OUT1073	Aux10- Key JPI18/INP1074 LED JPO18/OUT1074	Aux11- Key JPI19/INP1075 LED JPO19/OUT1075	Aux12- Key JPI20/INP1076 LED JPO20/OUT1076
Coolant Auto/Man Key JPI21/INP1077 LED JPO21/OUT1077	Flood Key JPI22/INP1078 LED JPO22/OUT1078	Mist Key JPI23/INP1079 LED JPO23/OUT1079	Aux13- Key JPI24/INP1080 LED JPO24/OUT1080	Aux14- Key JPI25/INP1081 LED JPO25/OUT1081
INCR/CONT Key JPI26/INP1082 LED JPO26/OUT1082	X1 Key JPI27/INP1083 LED JPO27/OUT1083	X10 Key JPI28/INP1084 LED JPO28/OUT1084	X100 Key JPI29/INP1085 LED JPO29/OUT1085	MPG Key JPI30/INP1086 LED JPO30/OUT1086
Axis4+ Key JPI31/INP1087 LED JPO31/OUT1087	Blank- Key JPI32/INP1088 LED JPO32/OUT1088	Axis2+ Key JPI33/INP1089 LED JPO33/OUT1089	Blank- Key JPI34/INP1090 LED JPO34/OUT1090	Axis3+ Key JPI35/INP1091 LED JPO35/OUT1091
Blank- Key JPI36/INP1092 LED JPO36/OUT1092	Axis1- Key JPI37/INP1093 LED JPO37/OUT1093	SLOW/FAST Key JPI38/INP1094 LED JPO38/OUT1094	Axis1+ Key JPI39/INP1095 LED JPO39/OUT1095	Blank- Key JPI40/INP1096 LED JPO40/OUT1096
Axis4- Key JPI41/INP1097 LED JPO41/OUT1097	Blank- Key JPI42/INP1098 LED JPO42/OUT1098	Axis2- Key JPI43/INP1099 LED JPO43/OUT1099	Blank- Key JPI44/INP1100 LED JPO44/OUT1100	Axis3- Key JPI45/INP1101 LED JPO45/OUT1101
CYCLE CANCEL Key JPI46/INP1102 LED JPO46/OUT1102	SINGLE BLOCK Key JPI47/INP1103 LED JPO47/OUT1103	TOOL CHECK* Key JPI48/INP1104 LED JPO50/OUT1106	FEED HOLD* Key JPI49/INP1105 LED JPO48/OUT1104	CYCLE START* Key JPI50/INP1106 LED JPO49/OUT1105

Appendix C: Keyboard Jog Mapping

Notes on Keyboard Jogging

The following table lists the SV_PC_KEYBOARD_KEY_1 to _104 system variables and the keys that they represent on a 101-key or 104-key QWERTY US English traditional keyboard. These keys have not been tested with any international/multilingual keyboard.

There is a System Variable called SV_PC_VIRTUAL_JOGPANEL_ACTIVE that is SET when the Keyboard Jogging screen (**Alt.-J**) is visible from the main menu of CNC11. Keyboard Jogging should not allow axis jogging unless this system variable is set, otherwise the motors may move while cursoring around in the Load menu. The other standard Jog Panel functions can be allowed all the time or restricted based on safety requirements for the particular installation. By default, standard PLC programs with Keyboard Jogging allow everything except axis jogging when the Keyboard Jogging screen is not visible. This mirrors the functionality of the Jog Panel itself.

Keys left blank should not be used. The second character of the two-character keys is the **Shifted** key. This means that SV_PC_KEYBOARD_KEY_60 indicating the **a** or **A** has been pushed will look the same in a PLC program. It is the responsibility of the programmer to look for the **Shift** key separately. On many keyboards there is at least one **Windows**, **System**, **Print Screen**, **Scroll Lock** and **Pause/Break** key and sometimes multiple, but these keys are not accessible in the PLC program. There are usually two **Alt.**, **Shift** and **Control** keys as well on each keyboard and the PLC sees either one as the same key-press. The right hand key is reported as the left key being pressed so do not look for the right key. The number pad may not be present on every keyboard so it is not advised to write a Jogging PLC program looking for these keys to be used except as a custom program. The number pad keys are preceded with NP_ in the table below.

Keyboard Key Numbering Table

Index	Key	Index	Key	Index	Key	Index	Key
1	ESC	27	0)	53	END	79	bB
2	F1	28	-_	54	PAGEDOWN	80	nN
3	F2	29	=+	55	NP_7	81	mM
4	F3	30	BACKSPACE	56	NP_8	82	,<
5	F4	31	INSERT	57	NP_9	83	.>
6	F5	32	HOME	58	NP_PLUS	84	/?
7	F6	33	PAGEUP	59	CAPSLLOCK	85	Right SHIFT(74)*
8	F7	34	NUMLOCK	60	aA	86	\
9	F8	35	NP_DIVIDE	61	sS	87	UP Arrow
0	F9	36	NP_MULT	62	dD	88	NP_1
11	F10	37	NP_MINUS	63	fF	89	NP_2
12	F11	38	TAB	64	gG	90	NP_3
13	F12	39	qQ	65	hH	91	NP_ENTER
14		40	wW	66	jJ	92	Left CTRL
15		41	eE	67	kK	93	
16		42	rR	68	lL	94	Left ALT
17	`~	43	tT	69	;;	95	SPACEBAR
18	1!	44	yY	70	'"	96	Right ALT(94)*
19	2@	45	uU	71	NP_4	97	
20	3/#	46	iI	72	NP_5	98	
21	4\$	47	oO	73	NP_6	99	Right CTRL(92)*
22	5%	48	pP	74	Left SHIFT	100	LEFT Arrow
23	6^	49	[{	75	zZ	101	DOWN Arrow
24	7&	50	]}	76	xX	102	RIGHT Arrow
25	8*	51	ENTER	77	cC	103	NP_0
26	9(	52	DELETE	78	vV	104	NP_DP

* key event returns same index as the Left side key (shown in parenthesis)

Appendix D: System Variables

System Variable Types

The following table gives the shorthand version of the data types listed below in the Type column.

Type	Description
M	Memory bit
I32	32-bit signed Integer
I64	64-bit signed Integer
F32	32-bit Floating-point (float)
F64	64-bit Floating-point (double)

Notes on Certain System Variables

There are some System Variables that are written to or read from outside of the scope of execution of the PLC program. These System Variables should be changed in only one place in the PLC program to avoid odd effects and problems troubleshooting.

Externally Read System Variables

The table below lists the system variables that are read external to the PLC program execution and that could have different values if the PLC program changed them at several places. In general, these variables should only be updated once in a single pass. Rather than rely on stage activity or other complicated logic to ensure they are updated once per pass, it is best to write to a temporary variable and then use the temporary variable to set the system variable only once.

Bad Example:

```
IF EStop THEN SET SV_STOP
IF !EStop THEN RST SV_STOP
IF LowLube THEN SET SV_STOP
```

Good Example:

```
IF EStop THEN SET TEMP_SV_STOP
IF !EStop THEN RST TEMP_SV_STOP
IF LowLube THEN SET TEMP_SV_STOP
IF TEMP_SV_STOP THEN SET SV_STOP
IF !TEMP_SV_STOP THEN RST SV_STOP
```

System Variable	Notes
SV_DRIVE_CONTROL_x	Used to SET the auxiliary output of an OPTIC4 axis.
SV_STOP*	
SV_SPINDLE_RPM_MODE*	Used for "C" axis.
SV_PLC_FEEDRATE_KNOB*	
SV_PLC_FEEDRATE_OVERRIDE*	

SV_PLC_FUNCTION_34*	Rapid Override. This usage should be deprecated in favor of SV_PC_TOGGLE_RAPID_OVERRIDE
SV_PLC_OP_IN_PROGRESS	This SV should not be used. It is listed here for completeness.
SV_MPG_x_AXIS_SELECT	The change will not take effect unless MPG movement is stopped.
SV_MPG_x_MULTIPLIER	
SV_MPG_x_WINDUP_MODE	
SV_MPG_x_ENABLED	The change will not take effect unless MPG movement is stopped.
SV_MPG_x_OFFSET_MODE	The change will not take effect unless MPG movement is stopped.
SV_MPG_x_PLC_MPG_MODE	The change will not take effect unless MPG movement is stopped.

Most likely to have unwanted side effects.

Externally Written System Variables

The table below lists the system variables that are written external to PLC program execution and therefore a PLC program could read at multiple times in a single pass and get different values. Thus, these variables should only be read once in a single pass. Rather than rely on stage activity or other complicated logic to ensure they are read once per pass, it is best to store the value into a temporary variable at the start of execution and then refer to that temporary variable throughout the program.

Bad Example:

```
IF SV_STALL_ERROR THEN (MEM1)
IF !SV_STALL_ERROR THEN (MEM2)
```

Good Example:

```
IF TRUE THEN TEMP_SV_STALL_ERROR = SV_STALL_ERROR
IF TEMP_SV_STALL_ERROR THEN (MEM1)
IF !TEMP_SV_STALL_ERROR THEN (MEM2)
```

Note that in the bad example, there is no guarantee that MEM1 != MEM2.

System Variable	Notes
SV_DRIVE_STATUS_x*	Typically used with OPTIC4 that has no external drive fault input visible to the PLC program (GPIO4D has INP17-20).
SV_STALL_ERROR*	
SV_PC_FEEDRATE_PERCENTAGE*	
SV_FSIOx	
SV_M_FUNCTION	Not used in any program.
SV_PC_POWER_AXIS_x	
SV_MPU11_LASH_OFFSET_x	
SV_MPU11_ABS_POS_x	

SV_MPU11_EXPECTED_POS_x	
SV_PC_RIGID_TAP_SPINDLE_OFF	

Note that all system variables are readable by either the PLC or CNC software. The following tables indicate who, either the PLC or CNC software, can change the state of the Bits or Words. In some cases there are System Variables that CNC software and the PLC program can write to and it is so noted.

* = Guaranteed Version that the system variable is available in, if a version is denoted by a star, it means there may be cases where the system variable was available in some earlier versions of CNC software.

CNC Software Write-Controlled System Variables

Name	Type	Version	Description
1-bit Boolean System Variables (CNC → PLC)			
SV_M94_M95_1-128	M	V3.00.00	Used for M-Codes that require PLC interaction like M3, M4, M6, M7, M8, M10, M11 and custom M-Codes. They are set and reset from M & G-code programs using M94 and M95. These bits can also be controlled by the PLC program even though they are in the section labeled as Read Only for the PLC. Be aware that CNC11 has built-in default actions for some M-codes that control the first 16 of these variables. An example is to turn off auto coolant and spindle when not running a program. IF !SV_PROGRAM_RUNNING THEN RST M3, RST M4, RST M7, RST M8.
SV_PROGRAM_RUNNING	M	V3.00.00	1 = MDI mode or a job is in progress
SV_MDI_MODE	M	V3.00.00	1 = MDI mode active
SV_PC_OVERRIDE_CONTROL_FEEDRATE_OVERRIDE	M	V3.00.00	1 = FeedRate Override Knob is allowed to change the feedrate on axis motion
SV_PC_OVERRIDE_CONTROL_SPINDLE_OVERRIDE	M	V3.00.00	1 = Spindle Override keys or Knob change the spindle speed commanded
SV_PC_OVERRIDE_CONTROL_FEEDHOLD	M	V3.00.00	1 = Feedhold pauses the G-Code program
SV_PC_POWER_AXIS_1-8	M	V3.00.00	1 = indicates if the axis is powered to hold position ? = A,B,C,U,V,W,X,Y,Z
SV_?_AXIS_POWERED		V3.15.22	
SV_DRIVE_ONLINE_1-8	M	V3.00.00	1 = The drive for the axis is detected. ? = A,B,C,U,V,W,X,Y,Z
SV_?_AXIS_DRIVE_ONLINE		V3.15.22	
SV_LEGACY_JOG_PANEL_ONLINE	M	V3.00.00	1 = Legacy Jog Panel detected. Ex: Uniconsole-2
SV_PLC_BUS_ONLINE	M	V3.00.00	1 = Valid MPU11 PLC detected. Checked as part of Fiber Checking PLC Program section.
SV_JOG_LINK_ONLINE	M	V3.00.00	1 = Valid Jog Panel detected as Jogboard.
SV_PLC_IO2_ONLINE	M	V3.00.00	1 = IO2 Legacy PLC detected.
SV_AXIS_VALID_1-8	M	V3.00.00	1 = Motor Parameters screen axis label allows motion. Allowed labels are X, Y, Z, A, B, C, U, V, W. ? = A,B,C,U,V,W,X,Y,Z
SV_?_AXIS_VALID		V3.15.22	

SV_PC_RIGID_TAP_SPINDLE_OFF	M	V3.00.00	CNC software SETs this bit to signal that the spindle should be turned off when the rigid tap depth has been reached if Parameter 36, bit 4 is SET, otherwise it is not needed. This is usually not needed. It is NOT needed just because you have a custom mfunc5.mac file. CNC software ignores custom M5s during a rigid tap. It decides which bit to turn turn off (M3 or M4) based on the spindle direction. The only time that this system variable is needed is as follows: If turning off the spindle requires doing more than clearing M3 or M4. Note that CNC software will only clear one of M3 or M4 during a rigid tap, not both.
SV_PC_KEYBOARD_KEY_x (1-104)	M	V3.00.50	1 = Keyboard key pressed, 0 = Keyboard key not pressed. See Appendix C .
SV_PC_VIRTUAL_JOGPANEL_ACTIVE	M	V3.00.50	Indicates whether keyboard jogging is enabled i.e. the user has activated the keyboard (virtual) jog panel using ALT-J . Most things besides jogging are allowed without the screen being up by default.
SV_PC_HOME_SET	M	V3.00.73	1 = home set for all valid axes, check this to force slow jog before homing unless Parameter 148 bit 0 is set.
SV_PC_SOFTWARE_READY	M	V3.00.79	1 = CNC software is initialized and communicating with MPU normally 0 = CNC software exited normally or communication fault occurred. Intended to be used in a PLC program to disable certain functions when the CNC software is not running or there is another fault preventing normal communications. For example, on ATC machines, the carousel should not be rotated if the software is not running since the CNC software monitors the reported carousel position and saves changes to the job file.
SV_PC_TOGGLE_RAPID_OVERRIDE	M	V3.03.14	CNC software SETs this bit when the state of Rapid Override needs to change. The PLC should RSTs this bit after toggling Rapid Override.
SV_DOING_AUTO_TOOL_MEASURE	M	V3.03.42	This bit is set by the CNC software when performing an automatic tool measure with the TT1.
SV_JOG_PANEL_REQUIRED	M	V3.03.59	Indicates the setting of "Jog Panel Required" in the Control Configuration Menu of CNC software.
SV_LIMIT_TRIPPED	M	V3.03.59	1 = Any configured limit switch is tripped 0 = No configured limit switches are tripped Note: The PLC program is not required to perform any actions when this bit is set.
SV_JOB_IN_PROGRESS	M	V3.03.59	Set when CNC software is running a job or running an MDI command, but not while at the MDI prompt waiting for input.
SV_RPM_MODE_ZERO_SPEED_x (1-8)	M	V3.15.11	1 = AC1 drive axis is at zero speed in RPM mode
SV_RPM_MODE_ACTIVE_1-8	M	V3.15.11	1 = AC1 drive axis is in RPM mode
SV_?_AXIS_FIBER_OK	M	V3.15.22	These nine system variables are mapped to the bits

			in SV_PC_CYCLONE_STATUS_2 according to the axis labels (?) XYZABCUVW. They are intended to be a convenience when writing PLC programs so that they can handle axis changes more easily.
SV_SKIN_EVENT_1-255	M	V4.14.00*	These 255 system variable bits are used to provide generic events that can be SET/RST by skinning applications via the CNC Skinning API. The Centroid Virtual Control Panel (VCP) is a skinning application that uses them to communicate the status of virtual jog panel button states, among other things. As a convention, SV_SKIN_EVENT_1 – 50 should be mapped to the same fifty keys found on a real hardware jog panel, i.e., SV_SKIN_EVENT_1 corresponding to Spindle+ key, and then continuing left to right, top to bottom with SV_SKIN_EVENT_50 corresponding to the CYCLE_START key.
SV_USB_MPG_POWER	M	V4.16.00*	Set when the USB MPG is powered on. Note that in order for this to be set, a state change of the USB MPG needs to occur.
SV_ENCODER_INDEX_PULSE_1-21	M	V4.60.00*	Current state of encoder index pulses
SV_LATCHED_ENCODER_INDEX_PULSE_1-21	M	V4.60.00*	Latched state of encoder index pulses. Remain set until read.
SV_SCALE_INITIALIZED_AXIS_1-8	M	V4.60.00*	Set when a scale is initialized (axis homed). They turn off once an axis homing command is issued.
SV_SCALE_ENABLED_AXIS_1-8	M	V4.60.00*	Set if scale is enabled in Scale Menu of CNC software
SV_HOME_SET_AXIS_1-8	M	V4.60.00*	Set after initial homing and remain set even if CNC software resets the homing state.
SV_THC_TORCH_TOUCH	M	V5.00.00	MPU SETs this when torch touch is detected when the “Tin” input is triggered on a connected Centroid THC_TXRX Board. This information is passed on to the MPU through the Encoder Index Pulse Channel.
SV_THC_DIVE_DETECTED	M	V5.00.00*	MPU sets this when a voltage spike is detected based on P516 and P517. The PLC should react and turn off SV_THC_CONTROL_ENABLE in situations where it is valid.
SV_PC_EXT_USB_PANEL_INP_x (1-64)	M	V5.10.00	SET by CNC software for an external USB Panel to indicate a press of a button.
SV_PC_EXT_USB_PANEL_OUT_x (1-64)	M	V5.10.00	SET by PLC to activate outputs for an external USB Panel to activate indication LEDs.
SV_PC_TOGGLE RAPID FEED LINK	M	V5.20.00	CNC Software SETs this bit when the state of Feed/Rapid Link needs to change. The PLC should RST this bit after toggling Feed/Rapid Link.
SV_FORCE_PROBE_DETECTION_ON	M	V5.20.00	This Bit is set by CNC software when the detection of a probe is forced. Typically set when the probe tool # is called and in spindle. PLC should enable any probe protections it may have when this is set.
SV_PC_LASER_ON	M	V5.20.00	Bit is used to indicate the status of a crosshair laser. CNC software SETs this bit when the crosshair laser is requested to be on. PLC should SET this bit when

			the crosshair laser is on, and RST when the crosshair laser is off.	
SV_PC_DEFEAT_TRAVEL_LIMITS	M	V5.24.00	CNC software SETs this when software travel limits are defeated by M297.	
32-bit Integer System Variables (CNC → PLC)				
SV_PC_DAC_SPINDLE_SPEED	I32	V3.00.00	The DAC spindle speed as requested by the CNC software (value range is 0-65535)	
SV_M_FUNCTION	I32	V3.00.00	Set as part of M-code execution.	
SV_TOOL_NUMBER	I32	V3.00.00	Set as part of a tool change (M107), to indicate the requested tool number. In the case of “enhanced ATC” operation this is actually a request for a carousel bin location. (The LDT command of XPLCCOMP is deprecated and not used.)	
SV_ATC_CAROUSEL_POSITION	I32	V3.00.00	Sent by CNC software when it first starts up or as part of an “enhanced ATC” reset feature. (The LCP command of XPLCCOMP is deprecated and not used.)	
SV_ATC_TOOL_IN_SPINDLE	I32	V3.00.00	Sent by CNC software when it first starts up and the .job file is parsed or as part of an “enhanced ATC” reset feature. (The LTS command of XPLCCOMP is deprecated and not used.)	
SV_PC_FEEDRATE_PERCENTAGE	I32	V3.00.00	0-200% adjustment for axis motion control. The value is sent by CNC software for machine parameter 78 bit 1 checking and on-screen-display. This value is not needed if SV_PC_OVERRIDE_CONTROL_FEEDRATE_OVERRIDE is not SET.	
SV_PC_SPINDLE_OVERRIDE	I32	V3.00.00	Not currently used.	
SV_PC_CYCLONE_STATUS_x (1-2)	I32	V3.00.00	See Appendix H	
SV_PC_CYCLONE_STATUS_x (3-16)		V5.00.00		
SV_PC_MINI_PLC_ONLINE	I32	V3.00.00	Online bits for PLCADD1616 and other expansion PLC modules.	
			Bit	Function
			0	1 = miniPLC1 is online
			...	...
			15	1 = miniPLC16 online
16-31	Reserved			
SV_DRIVE_STATUS_x (1-8)	I32	V3.00.00	High Speed Drive Status word, separated by Drive type.	
SV_?_AXIS_DRIVE_STATUS		V3.15.22	? = A,B,C,U,V,W,X,Y,Z See Appendix I	
SV_MPU11_LASH_OFFSET_0-7	I32	V3.01.31	The current lash offset	
SV_FSI0_1-32	I32	V3.03.12	Fast Synchronous IO (see M300 commands in the Operator Manual)	
SV_STALL_REASON	I32	V3.05.07	Set whenever SV_STALL_ERROR is set. No Error = 0 position error = 1	

			full power without motion = 2 encoder differential error = 3 spindle slave position error = 4 OpticDirect C8 Error = 6 scale encoder differential error=15 encoder quadrature error = 16 scale encoder quadrature error = 17 standoff_error = 18 scale position error = 19 master enable turned off = 99
SV_STALL_AXIS	I32	V3.05.07	Set whenever SV_STALL_ERROR is set It will be set to the appropriate axis of the stall error. If the axis is not applicable then it will be set to 255.
SV_HSC_DRIVE_x_STATUS_y	I32	V3.09.31	These variables contain every bit of the status packets sent back by AC1 drives. There are a total of 64 variables, eight for each of the eight possible AC1 drives. Only SV_HSC_DRIVE_x_STATUS_4 is documented here as it contains PLC related information of greatest importance. SV_HSC_DRIVE_x_STATUS_4 bits: bit00 FatalError Indicates a serious error. The PLC program should monitor this bit on all AC1 drives and treat it as an emergency stop. This bit is cleared by the AC1 drive on the rising edge of SV_MASTER_ENABLE, but if the condition persists, the PLC program will not see a change. Therefore, if the FatalError bit is still on about a second after SV_MASTER_ENABLE has been turned back on (after a release of E-stop), then the fault should be thrown again. Other bits in this variable can be examined to determine the specific reason for the FatalError, but a PLC program does not need to handle every single error with a custom message as the state of all these bits can be viewed in the Log Options menu of CNC11, using the <F5> HSC function key. Also, this information is logged in the file hsc_status.txt and is included in a report. bit01 Warning Indicates a warning condition. Other bits in this variable can be examined to determine the specific reason for the warning. bit02 ErrorUVWInvalid bit03 ErrorUVWBadTransition bit04 ErrorUVWBadSize bit05 EncoderOk bit06 QuadratureError Note that EncoderOk and QuadratureError bits are already monitored by MPU11 and if set, will generate the appropriate encoder differential or quadrature error messages. bit07 EncoderMismatch bit08 LineVoltageOn

			bit09 OvercurrentHighSide bit10 OvercurrentLowSide bit11 OvervoltageMotor bit12 OvervoltageLine bit13 BrakeResistorMissing bit14 BrakeIGBTOpen bit15 MotorTemperatureSwitch bit16 HeatsinkTemperatureSwitch bit17 PlusLimit bit18 MinusLimit bit19 DriveShutdown bit20 BrakeOnTooMuch bit21 OvercurrentSensor bit22 WarningDriveHot bit23 ErrorDriveTooHot bit24 WarningMotorHot bit25 AccelTooGreat bit26 ADCOffsetOk bit27 ErrorMotorTooHot bit28 MoveSyncRunning bit29 StepRunning bit30 TuneRunning bit31 ErrorParameterChange
SV_SD_DRIVE_x_STATUS (1-5)	I32	V3.09.31	There are five of these variables that correspond to the status of legacy SD drives. Indicates a serious error. The PLC program should monitor this bit for all drives and treat it as an emergency stop condition. Other bits in this variable can be examined to determine the specific cause, but it is recommended that the PLC just echo this variable to W1-W44 so it can be viewed using line PLC diagnostics in CNC11. bit00 FatalErrorDetected bit01 ADMC300Online1 (should always be 1) bit02 ADMC300Online0 (should always be 0) bit03 IndexPulseDetected bit04 OvercurrentWarning bit05 PowerOn bit06 MoveRunning bit07 OvervoltageWarning bits08-15 unused bit16 ServoDriveErrorDetected bit17 OvervoltageDetected bit18 UndervoltageDetected bit19 CommutationEncoderError bit20 OvertemperatureDetected bit21 OvercurrentDetected bit22 DataTrasmitInError bit23 InvalidIndexPulse bit24 unused bit25 Lag (should not be reported) bit26 PositionError (should not be reported as MPU11 monitors and reports position errors) bit27 HighPowerNoMotion (should no be reported) bit28 EncoderBad (MPU11 monitors) bits29-31 unused

SV_DRIVE_TYPE_x (1-8)	I32	V3.13.02	The type of drive connected to the axis: 0 = None 1 = Legacy DC 2 = DC3IOB 3 = SD 4 = AC1 5 = OPTIC4 6 = GPIO4D 7 = DC1
SV_?_AXIS_DRIVE_TYPE		V3.15.22	8 = ALLINONEDC 9 = Optic Direct (YASINTF) 10 = RTK4 11 = ENCEXP 12 = OAK 13 = ACORN 14 = DRIVECOMM 15 = ECAT-Yaskawa 16 = ECAT-Estun 17 = ECAT-Nidec 18 = ECAT-MITSUBISHI 19 = ECAT-Delta 20 = ENCEXP2 21 = ECAT-Leadshine 22 = ECAT-LSServo 23 = ECAT-Glentek 24 = ACORNSIX 25 = ECAT-Generic
SV_DRIVE_VERSION_x (1-8)	I32	V3.13.02	The drive firmware version
SV_AXIS_LABEL_x (1-8)	I32	V3.15.22	The uppercase ASCII character value assigned to the axis. Common values are: A = 65 B = 66 C = 67 N = 78 U = 85 V = 86 W = 87 X = 88 Y = 89 Z = 90
SV_SKINNING_DATA_W_x (1-12)	I32	V3.15.22	Generic 32-bit integer data that can be set by CNC Skinning API functions and read by the PLC,
SV_?_AXIS_DRIVE_NUMBER	I32	V3.15.22	These nine system variables are mapped to machine parameter 300 (Axis 1 (?) Drive Number) through machine parameter 307 (Axis 8 (?) Drive Number). They are intended to be a convenience when writing PLC programs so that they can handle axis changes more easily. ? = A,B,C,U,V,W,X,Y,Z
SV_PC_CURRENT_WCS	I32	V4.14.00*	The current work coordinate system (1-18) in effect by the CNC software.
SV_USB_MPG_AXIS_SELECT	I32	V4.12.00*	Value indicating the wireless USB MPG axis select switch 0 = Off 1-6 selected axis (X, Y, Z, 4 th , 5 th , 6 th)

SV_USB_MPG_SCALE_SELECT	I32	V4.12.00*	Value indicating the wireless USB MPG scale selector knob 1 = x1 10 = x100 100 = x100 1000 = SPIN 10000 = FEED
SV_USB_MPG_BUTTON_STATE	I32	V4.12.00*	Contains bits for wireless MPG USB button states bit 0 = Reset (Cycle Cancel) bit 1 = Feed Hold bit 2 = Cycle Start bit 3 = Jog Plus bit 4 = Jog Minus bit 5 = SPIN Auto/Man bit 6 = SPIN On/Off bit 7 = Macro 1 bit 8 = Macro 2 bit 9 = Macro 3 bit 10 = Macro 4 bit 11 = Tool Check bit 12 = Set Zero (=0)
SV_USB_MPG_ENCODER_WHEEL	I32	V4.14.00*	The MPG wheel position. This system variable counts up and down and does not roll over.
SV_ETHER1616_ONLINE_BITS	I32	V4.20.00*	The online status of the Ether1616 devices. Used in conjunction with SV_MACHINE_PARAMETER_415 (Ether1616 Configured Bits) to detect changes.
ENCODER_DIFF_BITS	I32	V5.00.00*	bitmap of encoder differential errors
ENCODER_QUAD_BITS	I32	V5.00.00*	bitmap of encoder quadrature errors
SV_PC_MCU_STATUS_x (1-16)	I32	V5.00.00	See Appendix H
SV_PC_ERROR_COUNTER_x (1-40)	I32	V5.00.00	See Appendix J
SV_THC_VOLTAGE	I32	V5.00.00*	Current Voltage Reading from THC_TXRX Board.
SV_THC_ENCODER_STATUS	I32	V5.00.00*	Used to Report HC Encoder Status from THC_TXRX Board, Bit 0 is typically only used as it reports all error types. Bit 0 = General Error Bit 1 = Negative Delta Error Bit 2 = No B6/B7 Detected Error Bit 3 = B6/B7 Both Detected Error Bit 4 = Zero Delta Error
SV_POSITION_MODE_ERROR_AXIS_x (1-8)	I32	V5.00.00*	Same as in CNC software PID Encoder menu
SV_PC_EXT_USB_PANEL_W_x (1-16)	I32	V5.10.00	SET by CNC software to typically indicate a ration of a knob for Feed/Rapid/Spindle for an external USB Panel.
SV_PC_EXT_USB_PANEL_ONLINE_BITS	I32	V5.10.00	SET by CNC software when an External USB Panel is connected.
64-bit Integer System Variables (CNC → PLC)			
SV_MPU11_ABS_POS_x (0-7)	I64	V3.00.00	The Absolute Position of the axis in encoder counts. This is the same value that is reported to CNC software and is viewed in PID screen as AbsPos.

SV_MPU11_EXPECTED_POS_x (0-7)	I64	V3.01.31	The Expected Position of the axis in encoder counts. This is the current commanded position.
SV_STARTUP_TIME	I64	V3.11.12	A 12 decimal digit integer representing the date and time the CNC software was started, The format is YYMMDDHHmmSS,.
SV_ENCODER_POSITION_x (1-32)	I64	V5.08.00	Current Encoder Position.
32-bit Floating-point System Variables (CNC → PLC)			
SV_MACHINE_PARAMETER_0-999	F32	V3.00.00	Machine Parameter values as entered in CNC software. Note that these are 32-bit floating point values that have been converted from the 64-bit floating point values that are actually maintained by the CNC software, so there may be some loss of precision.
SV_PC_CMDDED_SPINDLE_SPEED	F32	V3.00.00	The commanded “S” value with Spindle Override factored in. Note that Parameters 65-67 for spindle range must be controlled in the PLC program.
SV_PC_CONFIG_MIN_SPINDLE_SPEED	F32	V3.00.00	The minimum spindle speed from the control configuration.
SV_PC_CONFIG_MAX_SPINDLE_SPEED	F32	V3.00.00	The maximum spindle speed from the control configuration.
SV_MEASURED_SPINDLE_SPEED	F32	V3.11.11	The measured spindle speed in RPM, taking into account the SV_SPINDLE_MID_RANGE and SV_SPINDLE_LOW_RANGE system variable settings.
SV_SKINNING_DATA_FW_1-11	F32	V3.15.22	Generic 32-bit floating point data that can be set by CNC Skinning API functions and read by the PLC,
SV_PC_MAXIMUM_CSS_SPEED	F32	V3.15.25	For Lathe, the maximum constant surface speed set by a G50 command.
SV_PC_MAX_SPINDLE_SPEED_FOR_TOOL	F32	V5.00.00	Maximum Allowed Spindle Speed for Tool In Spindle
SV_VELOCITY_RATIO	F32	V5.00.00*	Provides the current Velocity Ratio between Current Actual Velocity and Requested Feedrate Velocity. Value is 0 to 1. Note: From V5.00.00 to V5.20.00 only provides reliable ratio for non-smoothing moves. Since V5.22.00, will provide reliable ratio for both non-smoothed and smoothed motion.
SV_THC_VELOCITY_XY	F32	V5.00.00*	Provides the current XY velocity in encoder counts. Was used for smoothing ratio calculation but is no longer needed since V5.22.00. Deprecated
SV_MACHINE_POSITION_AXIS_x (1-8)	F32	V5.20.00	Current Machine Position Value for each axis (1-8) or (A,B,C,U,V,W,X,Y,Z) in machine units.
SV_?_AXIS_MACHINE_POSITION			
64-bit floating-point System Variables (CNC → PLC)			
SV_SKINNING_DATA_DFW_1-11	F64	V3.15.22	Generic 64-bit floating point data that can be set by CNC Skinning API functions and read by the PLC,

PLC Write-Controlled System Variables

Name	Type	Version	Description
1-bit boolean system variables (PLC → CNC)			
SV_SPINDLE_MID_RANGE	M	V3.00.00	This must be set for Rigid Tapping and display of spindle speed to function correctly. In combination with SV_SPINDLE_LOW_RANGE below, up to four ranges are supported.
SV_SPINDLE_LOW_RANGE	M	V3.00.00	This must be set for Rigid Tapping and display of spindle speed to function correctly. In combination with SV_SPINDLE_MID_RANGE above, up to four ranges are supported.
SV_PLC_FAULT	M	V3.00.00	Obsolete. Do not use.
SV_LUBRICANT_LOW	M	V3.00.00	Obsolete. Do not use.
SV_DRIVE_FAULT	M	V3.00.00	Obsolete. Do not use.
SV_SPINDLE_FAULT	M	V3.00.00	Obsolete. Do not use.
SV_STOP	M	V3.00.00	Bit that is SET by the PLC program on a critical error or when E-Stop is pushed to signal to the CNC software and MPU to prevent axis motion, spindle commands and ATC changes. The bit should be RST when E-Stop is released and there are no other errors.
SV_PLC_OP_IN_PROGRESS	M	V3.00.00	Obsolete. Do not use.
SV_PLC_FUNCTION_0	M	V3.00.00	Invalid (Do not use)
SV_PLC_FUNCTION_1	M	V3.00.00	Cycle Cancel
SV_PLC_FUNCTION_2	M	V3.00.00	Cycle Start
SV_PLC_FUNCTION_3	M	V3.00.00	Tool Check
SV_PLC_FUNCTION_4	M	V3.00.00	Select Single Block
SV_PLC_FUNCTION_5	M	V3.00.00	Select X1 Jog Mode
SV_PLC_FUNCTION_6	M	V3.00.00	Select X10 Jog Mode
SV_PLC_FUNCTION_7	M	V3.00.00	Select X100 Jog Mode
SV_PLC_FUNCTION_8	M	V3.00.00	Not Used (formally User Jog Inc Mode)
SV_PLC_FUNCTION_9	M	V3.00.00	Select Inc/Cont Jog Mode
SV_PLC_FUNCTION_10	M	V3.00.00	Select Fast/Slow Jog Mode
SV_PLC_FUNCTION_11	M	V3.00.00	Select Mpg Mode
SV_PLC_FUNCTION_12	M	V3.00.00	Axis 1 + Jog
SV_PLC_FUNCTION_13	M	V3.00.00	Axis 1 - Jog
SV_PLC_FUNCTION_14	M	V3.00.00	Axis 2 + Jog
SV_PLC_FUNCTION_15	M	V3.00.00	Axis 2 - Jog
SV_PLC_FUNCTION_16	M	V3.00.00	Axis 3 + Jog

Name	Type	Version	Description
SV_PLC_FUNCTION_17	M	V3.00.00	Axis 3 - Jog
SV_PLC_FUNCTION_18	M	V3.00.00	Axis 4 + Jog
SV_PLC_FUNCTION_19	M	V3.00.00	Axis 4 - Jog
SV_PLC_FUNCTION_20	M	V3.00.00	Axis 5 + Jog
SV_PLC_FUNCTION_21	M	V3.00.00	Axis 5 - Jog
SV_PLC_FUNCTION_22	M	V3.00.00	Axis 6 + Jog
SV_PLC_FUNCTION_23	M	V3.00.00	Axis 6 - Jog
SV_PLC_FUNCTION_24	M	V3.00.00	Aux1
SV_PLC_FUNCTION_25	M	V3.00.00	Aux2
SV_PLC_FUNCTION_26	M	V3.00.00	Aux3
SV_PLC_FUNCTION_27	M	V3.00.00	Aux4
SV_PLC_FUNCTION_28	M	V3.00.00	Aux5
SV_PLC_FUNCTION_29	M	V3.00.00	Aux6
SV_PLC_FUNCTION_30	M	V3.00.00	Aux7
SV_PLC_FUNCTION_31	M	V3.00.00	Aux8
SV_PLC_FUNCTION_32	M	V3.00.00	Aux9
SV_PLC_FUNCTION_33	M	V3.00.00	Aux10
SV_PLC_FUNCTION_34	M	V3.00.00	Select Rapid Override
SV_PLC_FUNCTION_35	M	V3.00.00	Select Man or Auto Spindle Mode
SV_PLC_FUNCTION_36	M	V3.00.00	Do not use
SV_PLC_FUNCTION_37	M	V3.00.00	Spindle Start
SV_PLC_FUNCTION_38	M	V3.00.00	Spindle Stop
SV_PLC_FUNCTION_39	M	V3.00.00	Aux11
SV_PLC_FUNCTION_40	M	V3.00.00	Aux12
SV_PLC_FUNCTION_41	M	V3.00.00	Deprecated. Do not use
SV_PLC_FUNCTION_42	M	V3.00.00	Deprecated. Do not use
SV_PLC_FUNCTION_43	M	V3.00.00	Select Coolant Flood
SV_PLC_FUNCTION_44	M	V3.00.00	Select Coolant Mist
SV_PLC_FUNCTION_45	M	V3.00.00	Feed Hold
SV_PLC_FUNCTION_46– SV_PLC_FUNCTION_97	M	V3.00.00	Do not use.
SV_PLC_FUNCTION_98	M	V3.00.00	Select Spindle CCW
SV_PLC_FUNCTION_99	M	V3.00.00	Select Spindle CW
SV_PLC_FUNCTION_100– SV_PLC_FUNCTION_103	M	V3.00.00	Do not use.
SV_PLC_FUNCTION_104	M	V3.00.00	Coolant Auto / Manual Mode
SV_PLC_FUNCTION_105	M	V3.00.00	Do not use
SV_PLC_FUNCTION_106	M	V3.00.00	Spindle Override +

Name	Type	Version	Description
SV_PLC_FUNCTION_107	M	V3.00.00	Spindle Override -
SV_PLC_FUNCTION_108	M	V3.00.00	Select Spindle Override / 100 %
SV_PLC_FUNCTION_109	M	V3.00.00	Escape Key (sent to the PC)
SV_PLC_FUNCTION_110	M	V3.00.00	Axis 7 Jog +
SV_PLC_FUNCTION_111	M	V3.00.00	Axis 7 Jog -
SV_PLC_FUNCTION_112	M	V3.00.00	Axis 8 Jog +
SV_PLC_FUNCTION_113	M	V3.00.00	Axis 8 Jog -
SV_PLC_FUNCTION_114– SV_PLC_FUNCTION_127	M	V3.00.00	Unused
SV_PLC_OVERRIDE_CONTROL_FEEDR ATE_OVERRIDE	M	V3.00.00	1 = Feedrate can be changed from commanded value by Feedrate Override
SV_PLC_OVERRIDE_CONTROL_SPIND LE_OVERRIDE	M	V3.00.00	1 = Spindle Speed can be changed from commanded value by Spindle Override
SV_PLC_OVERRIDE_CONTROL_FEEDH OLD	M	V3.00.00	1 = Feedhold is allowed
SV_ENABLE_AXIS_x (1-8)	M	V3.00.00	Obsolete. Do not use.
SV_MASTER_ENABLE	M	V3.00.00	PLC sets this bit to turn on the Master Enable to hardware devices (drives and PLCs)
SV_STALL_ERROR	M	V3.00.00	1 = MPU11 detected an error that the PLC program should handle. See SV_STALL_REASON and SV_STALL_AXIS for further information on the specific cause. PLC program should turn off all enables including the Master Enable when this error occurs. The PLC Program should RST this bit if the error occurred and E-Stop is pushed in. (ie clear on E-Stop)
SV_MPG_x_ENABLED (1-3)	M	V3.00.00	This MPG is enabled. This bit switches between MPG mode and Vector controlled mode for all axes in this group. When this is enabled the MPU11 will not process motion vectors from the PC or allow Jogging.
SV_MPG_x_ENABLED (4-9)		V5.10.00	
SV_MPG_x_WINDUP_MODE (1-3)	M	V3.00.00	The MPG is in windup mode. This means the MPG will move the total distance commanded by the MPG encoder input. This mode is typically enabled for x1 and x10 mode. This mode should be disabled for x100 mode. When windup mode is disabled the MPG will try to keep up, but will go off position if the MPG encoder counts too fast. When this happens CNC11 will display the message “MPG moving too fast”.
SV_MPG_x_WINDUP_MODE (4-9)		V5.10.00	
SV_MPG_x_OFFSET_MODE (1-9)	M	V3.00.00	The MPG is in Offset Mode. The MPG movement will be added to the current Expected Position instead of setting the Expected Position. In this mode the MPG is independent and is allowed to command motion while vectors are being processed.

Name	Type	Version	Description
SV_MPG_x_PLC_MPG_MODE (1-9)	M	V3.00.00	The MPG is in PLC Controlled Mode. The MPG encoder input will be read from SV_MPG_1_PLC_OFFSET instead of the actual encoder input. The plc program can change SV_MPG_1_PLC_OFFSET which will cause the mpg axis to move.
SV_TRIGGER_PLOT_DUMP	M	V3.00.52	This bit is intended for internal debugging purposes. When set, it will start a debug dump to be sent to CNC software. After CNC software receives all the debug data, it will launch the <code>plot.exe</code> program to display it. Without custom built CNC software, the debug dump has no useful data.
SV_RESET_PLC_STATS_MIN_MAX	M	V3.01.13	Resets the current Minimum/Maximum PLC executor statistics which are displayed on the PLC diagnostic screen (ALT-I)
SV_RESET_PLC_STATS_AVG	M	V3.01.13	Resets the current Average PLC executor statistics which are displayed on the PLC diagnostic screen (ALT-I)
SV_SPINDLE_RPM_MODE	M	V3.01.24	Used for C Axis Lathe. When this bit is set, the mpu11 will send the current value of SV_SPINDLE_DAC as the PID output to the drive for the last axis configured as a "C axis" When this mode is active, full power without motion and position errors are disabled for the axis.
SV_SCALE_INHIBIT_AXIS_x (1-8)	M	V3.07.17	When set, scale compensation for the axis is disabled until the bit is reset. When the scale compensation is disabled in this manner it will undo any previous corrections. The DRO will show the absolute position of the motor encoder when the scale is disabled.
SV_ENABLE_IO_OVERRIDE	M	V3.11.11	When set by the PLC prgram, CNC software will indirectly allow the inversion and forcing of PLC bits through the live PLC display (ALT-I on the main screen) by manipulating machine parameters 911-939, which the PLC program can use directly to set system variables that actually perform the state forcing and inversion.
SV_RPM_MODE_DIRECTION_x (1-8)	M	V3.15.11	Used with AC1 drives to set the direction for RPM mode.
SV_RPM_MODE_ENABLE_x (1-8)	M	V3.15.11	Used with AC1 drives to enable RPM mode.
SV_RPM_MODE_AXIS_ENABLE_x (1-8)	M	V3.15.11	Used with AC1 drives to enable the axis in RPM mode.
SV_DISABLE_STALL_DETECTION	M	V4.14.00*	When SET, will disable the detection of some stall errors, such as position errors and full power without motion errors.
SV_PLC_SET_AXIS_x_PART_ZERO (1-8)	M	V4.14.00*	When SET by a PLC program and the control is not running a job and at the main screen, will request CNC software to set the Part Zero for the axis.
SV_DAC_OUTPUT_ENABLE_x (1-8)	M	V4.20.00*	When set, enables the control of the analog voltage on the OpticDirect by setting the desired value using the SV_DAC_OUTPUT_VALUE_1-8 system variables.

Name	Type	Version	Description
SV_THC_CONTROL_ENABLE	M	V5.00.00*	SET by PLC to instruct CNC12 to Turn On Height Control Movement for Z-Axis
SV_THC_RESET_REQUEST	M	V5.00.00*	SET by PLC to instruct CNC12 to Reset Position after Height Control Movement for Z-Axis
SV_PLC_DISABLE_TRAVEL_LIMITS	M	V5.00.00	When SET by PLC, CNC will no longer stop at or error at software travel limits when jogging and/or MPG. Note: Does not prevent software travel limits while performing normal job processing.
SV_PLC_RESTART_FORWARD	M	V5.10.00	SET by PLC to instruct Traverse.exe to move forward in the job.
SV_PLC_RESTART_REVERSE	M	V5.10.00	SET by PLC to instruct Traverse.exe to move reverse in the job.
SV_ESTOP_PRESSED	M	V5.20.00	SET by PLC to instruct CNC Software that E-Stop was pressed.
SV_RESET_PRESSED	M	V5.20.00	SET by PLC to instruct CNC Software that Reset was pressed.
32-bit signed integer system variables (PLC → CNC)			
SV_PLC_FAULT_STATUS	I32	V3.00.00	Bitwise parameter of certain fault conditions: 0x00000001 DIV_BY_ZERO 0x00000002 OUT_OF_BOUNDS 0x00000004 INVALID_OPCODE See Internal PLC Fault Checking .
SV_PLC_FAULT_ADDRESS	I32	V3.00.00	Exact address in the PLC program where the above fault occurred. Should be monitored as above. See Internal PLC Fault Checking .
SV_PLC_SPINDLE_SPEED	I32	V3.00.00	If parameter 78 is not set to display the actual spindle speed, this value is the current spindle speed that is displayed on the screen.
SV_STOP_REASON	I32	V3.00.00	When motion stops a reason is given and that reason is stored here. Do not use at this time.
SV_PLC_CAROUSEL_POSITION	I32	V3.00.00	Bin position that the carousel is at. It is critical that the Carousel not be allowed to turn unless CNC software is running. Check the SV for Software running. When P160 == 0 : Used by CNC Software to determine Active Tool. Value input should be a BCD Value. When P160 != 0 : Used by CNC Software to determine carousel position, tool putback, and active tool. Value input should be in normal binary.
SV_PLC_FEEDRATE_KNOB	I32	V3.00.00	The feedrate knob as the PLC would have CNC11 see.
SV_PLC_SPINDLE_KNOB	I32	V3.00.00	Spindle Speed override percentage sent to the PLC
SV_SMSG_D_ARG_x (1-9)	I32	V3.00.00	Reserved for future use. Do not use.
SV_PLC_DEBOUNCE_x (1-64)	I32	V3.00.00	First 240 PLC inputs debounce configuration words
SV_JOG_LINK_DEBOUNCE_x (1-32)	I32	V3.00.00	Jog Panel Input debounce configuration words

Name	Type	Version	Description
SV_LOCAL_DEBOUNCE_x (1-13)	I32	V3.00.00	MPU11 Onboard/Local Inputs Debounce configuration
SV_MPG_x_AXIS_SELECT (1-3)	I32	V3.00.00	The current selected axis for the mpg (1-8)
SV_MPG_x_AXIS_SELECT (4-9)		V5.10.00	
SV_MPG_x_MULTIPLIER (1-3)	I32	V3.00.00	The mpg multiplier value normally (1, 10, 100)
SV_MPG_x_MULTIPLIER (4-9)		V5.10.00	
SV_MPG_x_PLC_OFFSET (1-3)	I32	V3.00.00	MPG Offset, for PLC controlled MPG Inputs. This allows the PLC to control the MPG through an Analog to Digital Input rather than an encoder. This mode is enabled when the PLCMPGmode bit is set.
SV_MPG_x_PLC_OFFSET (1-3)		V5.10.00	
SV_DRIVE_CONTROL_1-8	I32	V3.00.00	High Speed Drive Control word ? = A,B,C,U,V,W,X,Y,Z See Appendix K
SV_?_AXIS_DRIVE_CONTROL		V3.15.22	
SV_SPINDLE_DAC	I32	V3.01.25	Used for C Axis Lathe. When this bit is set, the MPU will send the current value of SV_SPINDLE_DAC as the PID output to the drive for the last axis configured as a "C axis" When this mode is active, full power without motion and position errors are disabled for the axis.
SV_NV_Wx (1-10)	I32	V3.07.02	Nonvolatile memory. These system variables can be used in the same manner as other I32 variables in the PLC program. Their values are retained even when power is off. A PLC programmer can expect any changes to these system variables to be written to non-volatile memory within 2ms.
SV_SYS_COMMAND	I32	V3.11.07	Setting this value to a non-zero, positive number, will cause the CNC software to launch a process and try to execute the Windows batch file named plc_system_command_n.bat, where n is the number the SV_SYS_COMMAND is set to.
SV_INVERT_INP1_16_BITS SV_INVERT_INP17_32_BITS SV_INVERT_INP33_48_BITS SV_INVERT_INP49_64_BITS	I32	V3.11.09	The lower 16 bits of each of these system variables is used to invert the indicated inputs, with the least significant bit mapped to the lower INP bit and the most significant bit mapped to the higher numbered INP.
SV_INVERT_INP65_80_BITS		V3.13.03	
SV_FORCE_INP1_16_BITS SV_FORCE_INP17_32_BITS SV_FORCE_INP33_48_BITS SV_FORCE_INP49_64_BITS	I32	V3.11.11	The lower 16 bits of each of these system variables is used to force the state of the indicated inputs, with the least significant bit mapped to the lower INP bit and the most significant bit mapped to the higher numbered INP. The input is forced on if the equivalent bit in the SV_INVERT_INP* system variable is clear and forced off if the equivalent bit in SV_INVERT_INP* system variable is set.
SV_FORCE_INP65_80_BITS		V3.13.03	
SV_FORCE_ON_OUT1_16_BITS SV_FORCE_ON_OUT17_32_BITS SV_FORCE_ON_OUT33_48_BITS SV_FORCE_ON_OUT49_64_BITS	I32	V3.11.11	The lower 16 bits of each of these system variables is used to force ON the indicated outputs, with the least significant bit mapped to the lower OUT bit and the most significant bit mapped to the higher numbered

Name	Type	Version	Description
SV_FORCE_ON_OUT65_80_BITS		V3.13.03	OUT bit. Note that the equivalent bit in the SV_FORCE_OFF* (see below) system variable must be clear.
SV_FORCE_OFF_OUT1_16_BITS SV_FORCE_OFF_OUT17_32_BITS SV_FORCE_OFF_OUT33_48_BITS SV_FORCE_OFF_OUT49_64_BITS	I32	V3.11.11	The lower 16 bits of each of these system variables is used to force OFF the indicated outputs, with the least significant bit mapped to the lower OUT bit and the most significant bit mapped to the higher numbered OUT bit. Note that the equivalent bit in the SV_FORCE_ON* (see above) system variable must be clear.
SV_FORCE_OFF_OUT65_80_BITS		V3.13.03	
SV_FORCE_ON_MEM1_16_BITS SV_FORCE_ON_MEM17_32_BITS SV_FORCE_ON_MEM33_48_BITS SV_FORCE_ON_MEM33_48_BITS SV_FORCE_ON_MEM65_80_BITS	I32	V3.15.22	The lower 16 bits of each of these system variables is used to force ON the indicated memory bits, with the least significant bit mapped to the lower MEM bit and the most significant bit mapped to the higher numbered MEM bit. Note that the equivalent bit in the SV_FORCE_OFF_MEM* (see below) system variable must be clear. Note that the forced state is implemented between PLC program passes, i.e., if the PLC program changes a forced memory bit during execution of a program, it will update the state of the memory bit.
SV_FORCE_OFF_MEM1_16_BITS SV_FORCE_OFF_MEM17_32_BITS SV_FORCE_OFF_MEM33_48_BITS SV_FORCE_OFF_MEM49_64_BITS SV_FORCE_OFF_MEM65_80_BITS	I32	V3.15.22	The lower 16 bits of each of these system variables is used to force OFF the indicated memory bit, with the least significant bit mapped to the lower MEM bit and the most significant bit mapped to the higher numbered MEM bit. Note that the equivalent bit in the SV_FORCE_ON_MEM* (see above) system variable must be clear. Note that the forced state is implemented between PLC program passes, i.e., if the PLC program changes a forced memory bit during execution of a program, it will update the state of the memory bit.
SV_SYS_MACRO	I32	V4.14.00*	Setting this system variable to a non-zero value while CNC software is at the main menu, will cause the CNC software to try to load and run the G-code program named MPGmacro#.mac located in the system directory. For example, if on a Mill system, setting SV_SYS_MACRO = 3 will attempt to run the G-code file named \cncm\system\MPGmacro3.mac. This can be set to a negative value.
SV_DAC_OUTPUT_VALUE_1-8	I32	V4.20.00*	Setting this system variable will control the analog output on an OpticDirect, provided that the corresponding SV_DAC_OUTPUT_ENABLE_1-8 variable is set.
SV_THC_TARGET_VOLTAGE	I32	V5.00.00*	SETS the Target Voltage in the range of 0-4095 when SV_THC_CONTROL_ENABLE is SET.
32-bit floating point system variables (PLC → CNC)			

Name	Type	Version	Description
SV_PLC_FEEDRATE_OVERRIDE	F32	V3.00.00	The Feedrate factor (for MPU11 motion control). Values can range from 0 to 2.0. A value of 1.0 results in no change to the G-code programmed or Jog Rate value. Note that the MPU11 will cap the Feedrate to the maximum set in Machine Setup. Care must be taken to never apply a negative value.
SV_SMSG_F_ARG_x (1-9)	F32	V3.00.00	Reserved for future use. Do not use.
SV_NV_Fwx (1-10)	F32	V3.07.02	Nonvolatile memory. These system variables can be used in the same manner as other F32 variables in the PLC program. Their values are retained even when power is off. A PLC programmer can expect any changes to these system variables to be written to non-volatile memory within 2ms.
SV_METER_x (1-16)	F32	V3.15.11	These system variables are set by the PLC program to values between -100.0 and 100.0 for use of the CNC software to display as a meter in the DRO. Only the first eight are used, the meters 9-16 are reserved for future implementation.
SV_?_AXIS_METER		V3.15.22	
SV_RPM_MODE_SPEED_REQUEST_1-8	F32	V3.15.11	For AC1 drives, the requested RPM speed
SV_SPINDLE_METER	F32	V3.15.22	This value is mapped to one of the SV_METER_x system variables, in particular the one that corresponds to the spindle axis.
SV_PLC_RAPID_FEEDRATE_OVERRIDE	F32	V5.08.00	The Rapidrate Factor (For MPU motion control). Values can range from 0 to 1.0 (0 not included). A value of 1.0 results in no change to the G-Code Programmed Rapid Rate value. Note: When set to 0, The MPU will revert to using the SV_PLC_FEEDRATE_OVERRIDE for the Rapid Rates. MPU will Cap the Rapid Rate to the Maximum Rate set in machine configuration.

Appendix E: PLC I/O Location

This chapter lays out the kinds of I/O available in general and specifically on the different PLC boards available.

Input Types

The Input tables in each of the manuals for these drives contain several headings that may need clarification as to their exact purpose. The first is Input and this directly corresponds to the value used in the PLC Program. Next, a heading called Type corresponds to the type of Input that can be wired in. Within this column there are several labels which need to be explained. Configurable indicates the voltage required for the Input can be customized. Typical Centroid 5, 12 and 24VDC SIP resistors can be used. 12VDC Optoisolated means that it must be 12VDC powering the Input and that it has signal isolation to prevent electrical noise getting onto the circuit board. 5VDC only means that 5V is the maximum signal voltage that can be wired to this input. It also does not provide any electrical isolation. Sourcing means that the input must be connected to the Input common to turn on. Lastly, ADC is an Analog-to-Digital Conversion. The ALLIN1DC currently is the only standard PLC that has this functionality available for use in the PLC program, though there is also an ADD4AD4DA expansion board that has four DAC outputs and four ADC inputs.

Output Types

The Output tables in the manuals also have similar columns including Type which again refers to the type of Output that can be wired. The first types are Relay Outputs that may or may not be fused depending on the specific Output. There are Single Throw and Double Throw relays. The Single Throw has two pins associated with the output and there is continuity between them only when the Output is SET. The Double Throw has three pins which are a so-called Output Common, Normally Open and Normally closed. There is continuity between the Normally Closed pin and the common when the Output is RST and continuity between the Normally Open and common when the Output is SET. The next type is a DAC Output. This is a Digital-to-Analog Converting output. The Bit number specified defines the range of discrete values that can be commanded. Open Collector is an output, usually limited to 5VDC that must be wired to an external relay for anything other than low current signals. It is much safer to use a relay and another power supply than assume the output will not draw too much current and damage the drive.

Warning: Always review the specific board manual that came with your system before changing any wiring.

ALLIN1DC

The ALLIN1DC has 16 inputs and 9 outputs that may be used in the PLC program. The MPG header has 14 Inputs and 3 outputs that can be used as General I/O in a customized program. Typically it is better to get a PLC1616ADD card rather than using the MPG header, however. The first six inputs are dedicated to Limit switch wiring and cannot be used for general purpose Inputs. Moreover the first two inputs are tied to the first axis, the 3rd and 4th inputs to the second axis and likewise for the third. The firmware on the drive shuts down motion in the direction of the Input that is tripped, regardless of CNC11 software Limit Switch setup. There is also an analog Input and Output with 12-bit resolution. For the schematic and I/O map, reference the ALLIN1DC manual on the [Dealer Support site](#) or with the literature you received in your kit.

DC3IOB

The DC3IOB has 30 Inputs and 31 Outputs usable in the PLC program. There is one analog Output used for Spindle control. The first six inputs are dedicated to Limit switch wiring and cannot be used for general purpose Inputs. Moreover the first two inputs are tied to the first axis, the 3rd and 4th inputs to the second axis and likewise for the third. The firmware on the drive shuts down motion in the direction of the Input that is tripped, regardless of CNC11 software Limit Switch setup. For the schematic and I/O map, reference the DC3IOB manual on the [Dealer Support site](#) or with the literature you received in your kit.

GPIO4D

The DC3IOB has 30 Inputs and 31 Outputs usable in the PLC program. There is one analog Output used for Spindle control and four analog Outputs that are not PLC accessible that are used for 3rd party drive control. The first six Inputs are typically dedicated to Limit switch wiring, but can be used for general purpose Inputs. Inputs and Outputs 17-20 are dedicated to the axis enable and fault inputs respectively. For the schematic and I/O map, reference the DC3IOB manual on the [Dealer Support site](#) or with the literature you received in your kit.

PLC Expansion

All of the PLCs in this chapter can expand the Inputs and Outputs with miniPLC expansion boards. There are differing numbers of available ports so be sure to check the board manual to determine the total I/O range. There are 16 Debouncable Input and Output slots each capable of controlling 16 individual I/O for each board type. Each new board starts at a multiple of 16. There are 32 non-Debouncable Input and Output slots with 16 controllable bits which may be grouped as individual I/O or combined to produce one DAC. The slot numbering starts at 0 and goes to 47.

Once the boards are plugged into the PLC and powered as directed by the main PLC board's manual, power on the system and look in the mpu_info.txt file in the cncm or cnct directory. This gives a breakdown of the Inputs and Outputs and what board is associated with them.

The order that the I/O appears is directly related to which ADD port it is plugged into. ADD1 is first and ADD4 is last. Depending on which is the master PLC the ADD boards come in on the next available slot.

See the individual board manuals for more in-depth documentation of PLC Expansion with examples. MiniPLC devices must be plugged in when power is off on the drive to which they are connecting.

This table shows what slot maps to what Input Range. To figure out where the I/O should start for a certain slot that is not shown here, multiply the slot number by 16 and add 1 to get the starting Input or Output number. Add 15 to that number to get the last number in the slot.

Slot Number	Input/Output Numbers
0	1-16
1	17-32
2	33-48
4	65-80
14	225-240
15	241-256
20	321-336
47	753-768

ALLIN1DC

The expansion for the ALLIN1DC starts at Slot 1 and 16 because the onboard I/O take up the first slot of Debounced and non-Debounced I/O.

DC3IOB

The expansion for the DC3IOB starts at Slots 2 and 15 for the Inputs and Slots 4 and 15 for the Outputs.

GPIO4D

The expansion for the GPIO4D starts at Slots 2 and 15 for the Inputs and Slots 2 and 20 for the Outputs.

Legacy IO2 (PLCIO2, RTK3) and Legacy RTK2 (RTK2, PLC5/15, PLC3/3)

Since CNC11 v3.10, there is support for programming legacy PLCs. To program a legacy PLC with an MPU11, the MPU11 must be fitted with the LEGACYADD daughter card, and the Control Configuration PLC type set to "IO2" or "RTK2".

For PLCIO2 and RTK3, INP59-INP62 have been moved to INP34-INP37 and OUT59-OUT62 have been moved to OUT49-OUT52.

Appendix F: What's New in CNC11

This chapter covers major differences between CNC10 PLC programs and CNC11/12 PLC programs.

There is Only One PLC Program

In CNC11 there is only one PLC program source file and there is a new compiler for that program. The new compiler is called mpucomp. The program should be compiled in the cncm or cnct directory which is how CNC10 PLC programs are compiled as well. Because everything is taken care of in one file, there is no longer the confusion of what happens in what file, however the file does tend to be much larger than in CNC10 due to the requirement handed to the PLC program for taking care of many basic operations. Essentially, if the PLC program does not tell CNC11 about something I/O related, it does not know about it, period.

The PLC Program has the Final Word

CNC11 was designed to allow the PLC program to have the final say in all things that affect motion and I/O. The exception is that the PLC program cannot enable the motors, only alarm if there is a fault. Feedrate Override is a good example of this paradigm. The PLC program reads the Feedrate knob and tells CNC11 what it found, then CNC11 passes down what it thinks the feedrate override value should be for Parameter 78 compliance. Again the PLC program can change the value before sending it to the Motion Controller, though typically this is not done because it can break intended functionality.

Spindle Speed DAC Command

In CNC10 the Spindle Gear range was taken care of before the analog voltage was sent out by software. In CNC11 the S command spindle speed is sent down exactly as entered in G-Code and the PLC program must deal with the DAC output and tell CNC11 what it sent out.

Direct Control of and Responsibility for Jogging

In CNC11 the PLC program sees every Keyboard Key and Jog Panel Key press and must tell CNC11 that it has occurred. For example, this allows the PLC program to do such things as wait for a jog key to be pushed for some time before allowing motion or not reporting that the key has not been pushed until some other condition is met.

Compiler/Language Differences

The following sections represent the differences between CNC10 and CNC11 PLC Programming paradigms and keywords.

PLC Inputs and Outputs

In CNC11 all inputs and outputs are only physical inputs and outputs. Inputs can only be read or **inverted** by the PLC program. The M94/M95 input bits, Spindle Range input bits, and status outputs from CNC10, for example, are now System Variables.

Green == 1 == closed == SET

Any Bit Variable in CNC11 is Green on the PLC Diagnostic screen if it is SET or closed and Red otherwise. This is true for Normally Closed or Normally Open Variables.

Keywords Removed

In CNC10 there were several keywords (LDT, LTS, LMT, LCP) which have been removed in CNC11. They are all

replaced by System Variables.

Timers

Timer units in CNC10 were 10 ms increments, but now in CNC11 the units are 1 ms This means that counting to 1000 in CNC10 takes 10 seconds whereas in CNC11 it would take 1 second.

Stages

In CNC11 when a Stage is reset all the variables inside retain their value, whereas in CNC10 all the variables are reset. Keep this in mind so that when a Jump is called or the Stage is reset manually, variables that you want off should be turned off explicitly before leaving the Stage. Be aware especially of One-Shots because in CNC10 when Jumping from one Stage to another all Coiled Variables are turned off whereas in CNC11 they are not. Read more on [Stages](#) in CNC11 here.

PLC Program Should Detect All Faults

In CNC10 PLC, Drive, Lube and Spindle Faults were detected by looking at what has become SV_STOP in CNC11 and printing the correct message based on several conditions. This is not allowed by convention in CNC11. The PLC programmer should take care of these errors with the PLC Messaging functionality.

Appendix G: Definitions of nonobvious Words

Bit

A Binary piece of information that can be on or off. For example, Inputs are Bits.

Integer Number

A positive or negative whole number. Examples include 15, -10, 0 and 1309921.

Floating-point Number

A positive or negative number with fractional components expressed to the right of a decimal point. Examples include 2.1453, -15.2, 0.0, and 104.999999.

Range

The span of numbers that can be represented by a variable.

Precision

A measure of the number of bits a variable can express. This directly relates to how big a number an integer variable can represent and how precise a number a floating-point variable can represent. A more precise (higher bit count) floating-point number is less prone to round-off error when comparing very large numbers with very small (ex: 1454394783237.35 – 0.00000001). More bits means longer processing time, so unless extreme accuracy is needed the 32-bit version of variables should be used.

Data Type

A classification for variables that defines what kind of information they can hold and the range of values they can represent. Further explanation for all the available data types can be found [here](#).

Define/Declare

Create an easier to understand identifier for any data type. These are conveniences for the programmer to allow easier understanding of logic in a PLC program. Data types can be set or checked directly, but a warning is issued by the compiler because it is considered bad practice and can cause confusion. Examples are:

```
Estop      IS INP11                      ;input 11 on DC systems is typically E-Stop
Shift_Key  IS SV_PC_KEYBOARD_KEY_74 ;pushing the shift key results in this System
                                           ;Variable being set high.
```

Variable

Any of the data types or definitions of data types used in the PLC program that can change its value while CNC11 and the PLC program is running. Examples include inputs, outputs, stages, renamed inputs by definition, etc.

Constant

Any literal number or definition of a number. Typically definitions of constants are written in all caps to differentiate from variables. Examples include:

10

2.5

ARM_FAULT_CODE IS 2561

Appendix H: Cyclone & MCU Status SV Information

The SV_PC_CYCLONE_STATUS_x variables contain the very same information that the cyclone processor it self uses, and thus contains information that may or may not be useful for the PLC. In addition, this also means that each board generation may uniquely use these variables differently. Though efforts have been made to keep them consistent.

	MPU13 (Hickory)	MPU12 (AcornSix)	MPU11 (MPU11, ALLIN1DC, OAK)
Bits	SV_PC_CYCLONE_STATUS_1		
0-20	Reserved		
21	PLCbus external device enabled	PLCbus external device enabled	PLCbus external device enabled
22	Reserved		
23	JOGLink external device enabled		
24-31	Reserved		
	SV_PC_CYCLONE_STATUS_2		
0-7	Drive Axis 1-8 Enabled	Drive Axis 1-8 Enabled	Drive Axis 1-8 Enabled
8-31	Reserved		
	SV_PC_CYCLONE_STATUS_3		
0-31	Reserved		
	SV_PC_CYCLONE_STATUS_4		
0-1	Reserved		
2	MCU Shutdown Detected	Reserved	Reserved
3-31	Reserved		
	SV_PC_CYCLONE_STATUS_5-16		
0-31	Reserved		

MPU13 Systems also have SV_PC_MCU_STATUS_x variables for the EtherCAT Processor.

Bits	SV_PC_MCU_STATUS_1-3
0-31	Reserved
Bits	SV_PC_MCU_STATUS_4
0-15	Reserved
16	EtherCAT is in OP Mode
17-31	Reserved
	SV_PC_MCU_STATUS_5-16
0-31	Reserved

Appendix I: Drive Status SV Information

ALLIN1DC/DC3IOB	
Bit	Description
0	Current Setting Low
1	Current Setting High
2	High power enable – FETs are installed to handle 15A current setting
3	Drive master/slave – 1 = communication on fibers, 0 = comm. On wires
4	Aux 1 jumper state – 1 = jumper block removed, 0 = jumper block in place
5	Aux 2 jumper state – 1 = jumper block removed, 0 = jumper block in place
6-15	Reserved

DC1	
Bit	Description
0	Current Setting Low
1	Current Setting High
2	High power enabled – FETs are installed to handle 15A current setting
3-4	Reserved
5	Spare jumper state – 1 = jumper block removed, 0 = jumper block in place
6	Plus Limit State
7	Minus Limit State
8-15	Reserved

Optic 4	
Bit	Description
0-10	Reserved
11	Quadrature Error: Incorrect encoder state transition
12	Direction Bit
13	Index Pulse
14	Differential error on encoder A or B channel
15	Drive fault from 3 rd party drive (1 = Fault, or no voltage at input)

Optic Direct	
Bit	Description
0-1	Reserved
2	Acceleration too high
3	Quadrature Generation Error
4	On board test point

5-7	Alarm code bit 3, 2, 1
8	/TGON – above speed
9	Holding Brake State
10	/S_RDY – Servo Ready
11	Quadrature error
12	Direction bit
13	Index Pulse
14	Differential error on encoder A or B channel
15	Drive fault from 3 rd party drive (1 = Fault, or no voltage at input)

Appendix J: Error Counter Information

	MPU11	MPU12/MPU13	Note
1	Jog external error count	Reserved	Expansion Side
2	Jog internal error count	Reserved	MPU Side
3	Jog external error count	Reserved	Expansion Side
4	Jog internal error count	Reserved	MPU Side
5	Shutdown count	Shutdown count	FPGA shutdowns
6	Drive internal errors	Reserved	MPU Side
7	Drive external errors 0	Reserved	Expansion Side
8	Drive external errors 1	Reserved	Expansion Side
9	Drive external errors 2	Reserved	Expansion Side
10	Drive external errors 3	Reserved	Expansion Side
11	Drive external errors 4	Reserved	Expansion Side
12	Drive external errors 5	Reserved	Expansion Side
13	Drive external errors 6	Reserved	Expansion Side
14	Drive external errors 7	Reserved	Expansion Side
15	PLCIO2 reception errors	Reserved	MPU side
16	SD drive errors	Reserved	Comm Errors
17	Missed interrupts	Missed interrupts	FPGA interrupts missed
18	Reserved	Homeless blocks	Data received before location assignment
19-32	Reserved	Reserved	
33	Missed interrupts	Missed interrupts	FPGA interrupts missed, counted on DSP from INT_MISS bit
34	Debug packet overflow	Debug packet overflow	
35	Raw debug buffer overflow	Raw debug buffer overflow	
36	Command buffer overflow	Command buffer overflow	
37	Rx buffer overflow	Rx buffer overflow	
38	Rx buffer full	Rx buffer full	
39	Rx wiznet buffer full	Rx wiznet buffer full	
40	Increments every pass	Increments every pass	Debug life sign

Appendix K: Drive Control SV Information

Allin1DC/DC3IOB/DC1	
Bits	Description
0-14	Reserved
15	Axis Enable*

Optic 4	
Bits	Description
0-12	Reserved
13	Tach. Direction Inversion**
14	Auxiliary Output
15	Axis Enable*

Optic Direct	
Bits	Description
0-6	Reserved
7	Position Error Clear
8	P-OT – Drive Limit
9	N-OT – Drive Limit
10	/P-CON – Control Method Switching
11	/P-CL – Positive Torque Limit
12	/N-CL – Negative Torque Limit
13	Absolute position send (not controllable on V0x0008+)
14	Alarm Reset
15	Axis Enable*

Notes:

* Controlled by MPU firmware, do not attempt to change.

** Controlled by parameters 200-207, do not attempt to change.